



Gruppo SkyNet
skynetunigroup@gmail.com

Code Guardian — Dal Proof of Concept all'MVP

*Proof of Concept degli agenti, progettazione backend MVP, progettazione frontend MVP
e infrastruttura AWS*

Versione	<i>1.0</i>
Uso	<i>Interno</i>
Destinatari	<i>Team di sviluppo</i>
Redattori	<i>Samuele Bertin, Marco Barbiero, Sara Ristovic e Dario Pirrone</i>
Verifica	<i>Marco Barbiero</i>
Approvazione	<i>Sara Ristovic</i>

Versioni del documento

Ver.	Data	Descrizione	Autore	Verificatore
1.1	2026/08/21	Aggiunta Parte V (Progettazione Backend MVP): autenticazione reale Argon2id + JWT HS256, cifratura credenziali AES-256-GCM + HKDF, evoluzione <code>CreateContextDto</code> (URL-first), sequenza di validazione in 8 passi su <code>POST /contexts</code> , meccanismo pause/ripresa <code>LangGraph</code> per l'agente <code>Changelog</code> , endpoint <code>POST /tasks/:id/input</code> , quinto evento <code>WebSocket task.inputRequired</code> , ADR BE-1-5. Allineamento DTO Frontend (<code>pendingInput kind</code> e <code>AuthTokenDto</code>) con contratto Backend MVP.	Sara Ristovic	Marco Barbiero
1.0	2026/08/19	Fusione in un documento unico e coerente: integrata la Progettazione Frontend (MVP) come Parte III-IV e la Progettazione Infrastruttura AWS (MVP) come Parte V-VI. Sostituita la progettazione dati basata su DocumentDB con MongoDB Atlas via AWS PrivateLink (§ 3, § 31). Rimossa la tabella dei sostituti locali AWS del PoC (superata dalla mappatura reale in § 28). Aggiornati i rimandi in § 2, § 7, § 8 e § 13 per riflettere l'evoluzione PoC → MVP. Aggiunti altri design pattern	Samuele Bertin	Marco Barbiero
0.6	2026/08/13	Aggiunta la progettazione dell'orchestratore di produzione (§ 12); aggiornati di conseguenza § 2, § 10 e § 13.	Samuele Bertin	Nome Cognome
0.5	2026/08/11	Correzione coerenza con il codice	Samuele Bertin	Nome Cognome
0.4.1	2026/07/26	Aggiunto Qwen come esempio concreto di provider LLM gestito e paragrafo dedicato all'uso dei dati per l'addestramento (RS.5), con l'asimmetria tra Bedrock e API diretta	Marco Barbiero	Nome Cognome
0.4.0	2026/07/26	Allineamento alla progettazione backend: integrazione GitHub read-only con tool-calling, provider LLM via API gestita, aggiunta di modello di dominio, contratto Report, endpoint interni e decisioni architetturali	Marco Barbiero	Nome Cognome
0.3.0	2026/07/24	Inizio ristesura completa del PoC: Backend e infrastruttura	Marco Barbiero	Nome Cognome
0.2.0	2026/07/23	Inizio ristesura completa del PoC: Frontend e tecnologie	Samuele Bertin	Nome Cognome
0.1.0	2026/07/06	Prima stesura: progettazione del PoC degli agenti	Nome Cognome	Nome Cognome

Indice

I	Proof of Concept degli Agenti	10
1	Introduzione e obiettivo	10
2	Perimetro del PoC: cosa implemento e cosa no	10
2.1	Cosa implemento	10
2.2	Cosa non implemento (in questa fase)	11
3	Vincoli, scelte tecnologiche e sostituti di AWS	12
4	Architettura del PoC	14
4.1	Il perimetro in un colpo d'occhio	14
4.2	Un solo scheletro, tre configurazioni	16
4.3	Modello di dominio	18
5	Il contratto di output condiviso	18
6	Integrazione con GitHub	20
6.1	Come l'agente legge dal repository	20
6.2	I quattro controlli su ogni chiamata	21
6.3	Autenticazione degli endpoint interni	21
6.4	Il caso dell'Agente Docs	21
7	Contratto delle API: comunicazione frontend-backend	22
7.1	Due piani di comunicazione distinti	23
7.2	API REST pubbliche (browser → backend)	23
7.3	Canale realtime (browser → backend)	25
7.4	Gli schemi dei DTO principali	25
7.5	API interne (agenti → backend)	26
8	Orchestrazione ed esecuzione	27
8.1	Instradamento deterministico	27
8.2	Ciclo di vita di una Task	27
8.3	Il giro completo end-to-end	28
9	Cosa implemento per ciascun agente	29
9.1	Agente Docs — documentazione inline	30
9.2	Agente Security — scansione vulnerabilità OWASP	31
9.3	Agente Changelog — changelog tecnico	31
9.4	Il filo comune	32
10	Decisioni architetturali (ADR)	33
11	Come validiamo il PoC	34
12	Estensioni di scalabilità dell'orchestratore (roadmap oltre l'MVP)	35
12.1	Obiettivo e perimetro	35
12.2	Requisiti	36
12.3	Architettura dei componenti	37
12.4	Modello di dominio esteso	38
12.5	Decisioni architetturali dell'orchestratore di produzione (ADR-5 – ADR-8)	40

12.6	Ciclo di vita del Batch	40
12.7	Retry e circuit breaker	41
12.8	Scaling: il pool di agenti	42
12.9	Dashboard di monitoraggio	42
12.10	Contratto API esteso	43
12.11	Migrazione dal PoC	43
12.12	Pattern architetturali aggiuntivi raccomandati	44
13	Sintesi e prossimi passi	46
II	Frontend MVP — Specifica Operativa (UX/UI)	46
14	Perimetro del documento	46
14.1	Obiettivo	46
14.2	Cosa progetto	46
14.3	Cosa non progetto (fuori perimetro MVP)	47
14.4	Mappatura Componenti PoC → Frontend MVP	47
15	Architettura dell'informazione	48
15.1	Ruoli e permessi	48
15.2	Mappa delle pagine e delle rotte	49
15.3	Credenziali e servizi esterni	51
16	Flussi utente per ruolo	51
16.1	Notazione	51
16.2	Flusso trasversale: accesso e configurazione iniziale	51
16.3	Selezione ed avvio multiplo delle operazioni (trasversale)	51
16.4	Flusso Sviluppatore — Agente Docs	52
16.5	Flusso Security Auditor / Team Lead — Agente Security	52
16.6	Flusso Project Manager — Agente Changelog	52
16.6.1	Dipendenza di esecuzione tra Changelog tecnico e di business	52
16.7	Flusso trasversale: consultazione dei report pregressi	54
16.8	Flusso trasversale: rilevamento di una credenziale non più valida	54
17	Wireframe descrittivi delle schermate	54
17.1	Notazione	54
17.2	Registrazione — <code>/register</code>	54
17.3	Login — <code>/login</code>	55
17.4	Servizi connessi — <code>/credentials</code>	55
17.5	Selezione del contesto — <code>/select</code>	56
17.6	Avvio operazioni — <code>/run</code>	56
17.7	Dashboard — <code>/tasks</code>	56
17.8	Dettaglio report — <code>/reports/:id</code>	57
17.9	Storico report — <code>/reports</code>	57
17.10	Intestazione applicativa	58
18	Design System e componenti condivisi	58
18.1	Stack di implementazione	58
18.2	Token visivi	58
18.3	Catalogo dei componenti condivisi	59
18.4	Stati interattivi comuni	60
18.5	Accessibilità (A11y)	60

19 Stato applicativo	60
19.1 Principio generale	60
19.2 <code>sessionStore</code> — Identità e credenziali	61
19.3 <code>selectionStore</code> — Contesto operativo	61
19.4 <code>tasksStore</code> — Stato vivo delle operazioni	61
19.5 Dati intenzionalmente non globali	63
19.6 Guardie di rotta	63
20 Contratto API consumato	63
20.1 Principi generali del contratto	63
20.2 Endpoint REST pubblici	64
20.3 Canale realtime (WebSocket)	65
20.4 Schemi dei DTO principali	66
20.5 API interne	68
21 Gestione degli errori	68
21.1 Due famiglie di errore	68
21.2 Pattern per gli errori di validazione e configurazione	68
21.3 Pattern per gli errori di esecuzione di un'operazione	68
21.3.1 Errore di orchestrazione o routing	69
21.4 Errore di esportazione (Report)	69
22 Tracciamento Requisito → Decisione Frontend	70
22.1 Decisioni Operative e Architetture	70
22.2 Requisiti mappati a livello di Flussi e Wireframe	71
22.3 Requisiti Fuori Perimetro Frontend	71
III Frontend MVP — Motivazioni e Decisioni Architetture	72
23 Principi Guida e Vincoli di Progetto	72
24 Decisioni Architetture (ADR-FE) e Motivazioni	73
24.1 Identità, Sessione e Ruoli	73
24.2 Integrazioni e Credenziali	74
24.3 Stack, Design System e Report	74
25 Compromessi e Rischi Accettati	74
26 Vincoli Ereditati dall'Infrastruttura AWS	76
26.1 Origine unica e assenza di policy CORS	76
26.2 Routing lato client e Custom Error Response	76
26.3 Configurazione a build-time per risorse statiche	76
26.4 Requisiti per connessioni WebSocket	76
26.5 Streaming dei file ed Esportazione PDF	77
26.6 Gestione degli errori di rete e Sicurezza Trasversale	77
IV Infrastruttura AWS — Manuale Operativo	78

27 Guida Operativa di Rilascio (Checklist)	78
27.1 Attività da avviare il Giorno 1	79
27.2 Rete e Sicurezza	79
27.3 Segreti e Chiavi	80
27.4 Dati e Caching	80
27.5 Immagini e Registro (ECR)	81
27.6 Calcolo e Orchestrazione (ECS)	81
27.7 Frontend e Distribuzione	82
27.8 Storage Applicativo	82
27.9 Osservabilità e Budget	82
28 Perimetro e Scope	83
28.1 Servizi AWS per Area (MVP)	83
28.2 Fuori Perimetro	83
28.3 Mappatura Componenti PoC → AWS	83
29 Rete e Sicurezza (VPC)	84
29.1 Parametri VPC base	84
29.2 Subnet e Routing	85
29.3 VPC Endpoints (PrivateLink)	85
29.4 Matrice Security Groups (Regole di Rete)	86
30 Compute e Orchestrazione (ECS su Fargate)	87
30.1 Cluster ECS e Task Definition	87
30.2 Gestione Immagini Container (Amazon ECR)	87
30.3 Application Load Balancer (ALB)	88
30.4 Service Discovery (AWS Cloud Map)	88
30.5 Flussi Applicativi End-to-End	88
31 Livello Dati e Caching	89
31.1 MongoDB Atlas (via AWS PrivateLink)	89
31.2 Amazon ElastiCache (Redis)	90
32 Integrazione Amazon Bedrock	91
32.1 Modelli e Accessi	91
32.2 Autenticazione e Policy IAM	92
32.3 Model Invocation Logging	92
33 Storage e Comunicazioni Esterne	92
33.1 Amazon S3 (Artefatti Applicativi)	92
34 Frontend Web Hosting	92
34.1 Amazon S3 (Hosting Statico)	93
34.2 Distribuzione Amazon CloudFront	93
34.3 Gestione SPA Routing e Build Vite	93
35 Segreti e Configurazione (Variabili d'Ambiente)	93
35.1 Mappatura Variabili PoC → AWS	93
35.2 Cifratura (AWS KMS)	94
36 Budget e Osservabilità Infrastrutturale	94
36.1 Metriche e Allarmi Base	94
36.2 Gestione Costi (AWS Budgets)	95

37 Tracciamento Requisito → Decisione AWS	95
V Infrastruttura AWS — Motivazioni e Decisioni Architettureali	97
38 Principi Guida e Vincoli di Progetto	97
39 Decisioni Architettureali (ADR) e Motivazioni	97
39.1 Identità, Accessi e Segreti	98
39.2 Rete, Sicurezza e Isolamento	98
39.3 Compute, Orchestrazione e Frontend	98
39.4 Scelte sul Livello Dati	99
39.5 Scelta dei Modelli LLM (Bedrock)	99
40 Compromessi e Rischi Accettati	99
41 Limiti di Piattaforma (Fargate e Prompt)	101
VI Progettazione Backend MVP	101
42 Perimetro e obiettivo	102
42.1 Cosa cambia dal PoC all'MVP	102
43 Schema MongoDB (Modello di Persistenza)	102
43.1 Diagramma Entità-Relazione	102
43.2 Collection <code>users</code>	104
43.3 Collection <code>servicecredentials</code>	104
43.4 Collection <code>analysiscontexts</code>	104
43.5 Collection <code>batches</code>	105
43.6 Collection <code>batchsteps</code>	106
43.7 Collection <code>tasks</code>	107
43.8 Collection <code>reports</code>	108
43.9 Indici e vincoli di integrità	110
44 Autenticazione e Autorizzazione	110
44.1 Data Transfer Object	110
44.2 Hashing delle password	111
44.3 JSON Web Token	111
45 Cifratura delle Credenziali di Servizio	111
45.1 Schema di cifratura	111
45.2 DTO delle credenziali	112
46 GitHub Client — Estensioni MVP	112
47 Creazione del Contesto di Analisi	113
47.1 Evoluzione del DTO	113
47.2 Sequenza di validazione in 10 passi	113
48 Contratto API MVP	114
48.1 Endpoint REST MVP	114
48.2 Canale realtime — 5 eventi WebSocket	115
48.3 Tipo <code>PendingInput</code> e <code>SubmitInputDto</code>	115

49	Meccanismo Input Utente: Pause e Ripresa dell'Agente Changelog	116
49.1	Soluzione architetturale	116
49.2	Implementazione LangGraph	117
49.3	Flusso completo end-to-end — CHANGELOG_BUSINESS (caso peggiore: tre sospensioni)	117
49.4	Perché questa soluzione	119
50	Decisioni Architetture Backend (BE-ADR)	119
51	Timeout e Limiti di Esecuzione	120
51.1	Gateway pubblico (RQ.6)	120
51.2	Timeout per operazione (<code>timeout_s</code>)	120
52	Catalogo delle Operazioni e Avvio	121
52.1	OperationDescriptorDto	121
52.2	Controlli prima dell'accodamento (POST <code>/tasks</code>)	121
53	Dettaglio degli Agenti	122
53.1	Agente Docs	122
53.1.1	DOCS_README (RF.82)	122
53.1.2	DOCS_INLINE (RF.83, RF.84)	122
53.1.3	DOCS_API (RF.86)	122
53.2	Agente Security	122
53.2.1	SECURITY_OWASP (RF.87–RF.91)	123
53.2.2	SECURITY_POLICY (RF.92, RF.93)	123
53.3	Agente Changelog	123
53.3.1	Flusso di input per le operazioni Changelog	123
53.3.2	CHANGELOG_TECHNICAL (RF.94–RF.96)	123
53.3.3	CHANGELOG_BUSINESS (RF.94–RF.105)	123
53.3.4	Leggibilità del Changelog (RQ.7)	124
54	Gestione degli Errori	124
54.1	Il meccanismo riusato dalla PoC (RF.65)	124
54.2	Codici d'errore (enumerazione chiusa)	124
54.3	Enumerazione <code>ErrorKind</code> lato agente (Python)	125
55	Report: Storico, Dettaglio ed Esportazione	125
55.1	Contratto <code>ReportDto</code> esteso (RF.54–RF.64)	125
55.2	Storico e sintesi (GET <code>/reports</code>)	127
55.3	Blocchi del corpo (RF.59–RF.62)	127
55.4	Esportazione PDF (RF.73–RF.74)	128

Elenco delle figure

1	Diagramma delle classi	15
2	Architettura a container del PoC	16
3	Modello di dominio persistito su MongoDB, corretto sul codice reale. Le sottoclassi <code>Payload</code> sono state rimosse (ridondanti con la figura 4); <code>AccessLog</code> e <code>ServiceCredential</code> riportano i campi effettivamente presenti negli schemi <code>Mon-goose</code>	19

4	Contratto Report	19
5	Facade GitHub read-only	20
6	Agente Docs e destinazione dell'output	22
7	Orchestrazione ed esecuzione	27
8	Ciclo di vita della Task	28
9	Giro completo end-to-end	29
10	Servizio agenti Python	30
11	Architettura dei componenti dell'orchestratore di produzione	38
12	Ciclo di vita di un Batch	41
13	Mappa di navigazione principale e reindirizzamenti condizionali del frontend	50
14	Diagramma di sequenza delle interazioni intermedie per l'Agente Changelog	53
15	Architettura degli store applicativi	60
16	Ciclo di vita di una Task (frontend)	62
17	VPC del MVP	85
18	Flussi applicativi principali	89

Parte I

Proof of Concept degli Agenti

1 Introduzione e obiettivo

Code Guardian è un sistema che automatizza audit, manutenzione e documentazione dei repository software. Nella sua forma completa è composto da un orchestratore centrale e da tre agenti intelligenti basati su modelli linguistici: l'agente **Docs** (README e documentazione tecnica), l'agente **Security** (scansione delle vulnerabilità OWASP e verifica delle policy interne) e l'agente **Changelog** (traduzione delle User Stories di uno Sprint in changelog tecnici e di business).

Nella prima stesura di questo documento il PoC era stato ridotto al solo core degli agenti, azionato da un driver a riga di comando che sostituiva sia l'orchestratore sia l'interfaccia utente. Un confronto con il professor Cardin ha messo in luce che questa riduzione lasciava fuori dal perimetro un rischio tecnico reale: non basta dimostrare che un agente basato su LLM legge materiale di progetto reale e produce un output strutturato, accurato e verificabile; occorre anche dimostrare che l'intero stack scelto per il prodotto si integri correttamente, perché diverse di queste tecnologie sono nuove per il team o comunque non ancora validate insieme.

A valle di quella revisione, il proponente ha inoltre respinto la seconda stesura del PoC perché non dimostrava a sufficienza l'integrazione tra le tecnologie scelte: la soglia richiesta è che **almeno l'80% delle tecnologie sia attraversato da un flusso end-to-end reale**, non semplicemente presente nel codice. Il criterio operativo che questo documento adotta è quindi netto: una tecnologia conta come integrata solo se un flusso osservabile e ripetibile la attraversa.

Per questo motivo il perimetro del PoC viene ampliato in due direzioni rispetto alla prima stesura. La prima: non si valida più il solo core dell'agente, ma una fetta verticale end-to-end, dall'interfaccia grafica fino alla persistenza, passando per il backend e per l'agente. La seconda, decisa in fase di progettazione backend e recepita in questa versione del documento: **l'agente dialoga effettivamente con GitHub**, leggendo codice e issue dal vivo attraverso il backend, invece di operare su un clone locale o su fixture. È proprio questa la giunzione che le stesure precedenti non dimostravano.

Restano fuori dal perimetro le parti rimandate a una fase successiva: l'orchestratore centralizzato completo, l'autenticazione, il deploy in produzione su AWS e l'apertura automatica di Pull Request. Questo documento aggiorna quindi perimetro, architettura e scelte tecnologiche del PoC, riflettendo lo stack realmente adottato e le decisioni di progettazione prese a valle della prima stesura.

2 Perimetro del PoC: cosa implemento e cosa no

Questa è la sezione più importante da condividere con il team, perché definisce le aspettative. Il PoC copre una fetta verticale per ciascuno dei tre agenti: una singola operazione rappresentativa, portata dall'inizio alla fine, sopra un'infrastruttura condivisa. Non copre gli agenti nella loro interezza né i casi d'uso di contorno.

2.1 Cosa implemento

- Un grafo di agente eseguibile (basato su LangGraph, in Python), con i suoi nodi, la gestione del timeout e la gestione strutturata degli errori.
- Un'astrazione del provider LLM (**LLMProvider**) con implementazione attiva verso un **API cloud gestita** (ad esempio Qwen) raggiunta via HTTPS con chiave di servizio, e un'implementazione per AWS Bedrock scritta ma non attivabile in questa fase (credenziali assenti).

- L'integrazione con **GitHub in sola lettura**: l'agente legge codice e issue del repository selezionato attraverso una *facade* esposta dal backend, che l'agente utilizza come insieme di *tool* LangGraph (§ 6).
- Un filtro sui linguaggi supportati (TypeScript, JavaScript, Python) e un limite dimensionale dell'ambito, applicati alla costruzione del contesto d'analisi.
- Una fetta per ciascun agente: documentazione inline per Docs, scansione vulnerabilità per Security, changelog tecnico per Changelog.
- Un contratto di output comune (l'oggetto **Report**), persistito su MongoDB e reso via API REST (§ 5).
- Un backend REST (NestJS su Node.js/TypeScript) che espone gli endpoint per avviare un agente, recuperarne lo stato e leggerne il report, che custodisce il token GitHub cifrato e che espone agli agenti la facade di lettura.
- Un'interfaccia grafica (React, con Vite, TanStack Router, pnpm) che consente di scegliere repository, ambito e operazione, avviare l'esecuzione e visualizzare il report.
- Un ambiente containerizzato (Docker Compose) che fa girare frontend, backend, servizio agenti, database e servizi di appoggio in locale.
- L'isolamento dei prompt in file esterni e i test con provider LLM simulato (per copertura e ripetibilità).

2.2 Cosa non implemento (in questa fase)

Questo elenco descrive il perimetro *del PoC*. Buona parte di queste esclusioni sono state colmate nelle fasi successive, documentate nella Parte II (Frontend MVP) e nella Parte IV (Infrastruttura AWS): lo segnalo puntualmente qui sotto.

- La gestione di batch complessi (dipendenze tra operazioni) e il quadro operativo di scalabilità (pool di agenti, circuit breaker). Il PoC include già la forma minima dell'Orchestratore necessaria a instradare un'operazione verso il suo agente (§ 8, RV.1). **Aggiornamento MVP**: questa stessa orchestrazione minima (**AgentRegistry/TaskProcessor**) resta l'Orchestratore dell'MVP ed è completata dalla dashboard di monitoraggio delle Task richiesta da RF.43–RF.47, progettata per intero nella Parte II (Frontend MVP); l'Orchestratore stesso non è quindi tra le esclusioni dell'MVP. Restano fuori dal perimetro MVP solo le estensioni di scalabilità in produzione (batch come DAG, retry automatico, pool di agenti, circuit breaker, dashboard operativa distinta da quella utente), la cui progettazione è in § 12.
- La scrittura su GitHub: nessuna Pull Request viene aperta. L'agente si ferma alla proposta (un diff), coerentemente con il vincolo RS.3 che vieta agli agenti autonomi la scrittura sul repository. **Aggiornamento MVP**: nell'MVP la Pull Request viene aperta, ma non dall'agente — è il backend, ricevuta la proposta, a invocare le API di scrittura di GitHub (ADR-AWS-10, § 39; ADR-FE-1, § 24), preservando il vincolo RS.3 a livello di isolamento di rete degli agenti.
- L'infrastruttura AWS (hosting, Bedrock, SES, S3): sostituita da esecuzione locale containerizzata e da un'API LLM gestita (§ 3). Il modello **non** viene eseguito localmente. **Aggiornamento MVP**: l'infrastruttura AWS reale è progettata per intero nella Parte IV.

- Registrazione, login, configurazione credenziali complete, notifiche email, dashboard di monitoraggio: casi d'uso di contorno. Il PoC assume un token GitHub già configurato via variabile d'ambiente o schermata minimale. **Aggiornamento MVP:** autenticazione, ruoli, credenziali e dashboard estesa sono progettati per intero nella Parte II (le notifiche email restano fuori perimetro anche per l'MVP).
- Le operazioni degli agenti non scelte come fetta: README e documentazione API per Docs, changelog di business per Changelog, e la seconda funzione dell'agente Security (policy-as-code su `POLICY.md`), inclusa come variante opzionale ma non obbligatoria. **Aggiornamento MVP:** tutte queste operazioni rientrano nel perimetro dell'MVP (§ 14.2, Tabella 10).

Tabella 1: Sintesi del perimetro: cosa entra nel PoC e cosa è rimandato.

Elemento	Nel PoC	Rimandato
Grafo agente + gestione errori/timeout	✓	
Provider LLM via API gestita (adapter Bedrock predisposto)	✓	
Integrazione GitHub read-only (codice + issue, dal vivo)	✓	
Facade GitHub come tool LangGraph	✓	
Contratto Report condiviso, persistito su MongoDB	✓	
Orchestrazione minima (instradamento operazione → agente)	✓	
Docs: documentazione inline	✓	
Security: scansione vulnerabilità OWASP	✓	
Changelog: changelog tecnico da GitHub Issues	✓	
Frontend React (Vite, TanStack, pnpm)	✓	
Backend NestJS + persistenza MongoDB	✓	
Orchestratore: estensioni di scalabilità (batch DAG, pool, circuit breaker)		✓
Apertura Pull Request su GitHub		✓
Hosting / Bedrock su AWS		✓
Auth completa, email, dashboard		✓
Docs: README e API docs		✓
Security: policy-as-code su <code>POLICY.md</code>	(opzionale)	
Changelog: changelog di business		✓

3 Vincoli, scelte tecnologiche e sostituti di AWS

Il capitolato prevede AWS in tre punti: gli LLM tramite un servizio cloud (Bedrock), l'hosting e l'infrastruttura, e il database MongoDB. A questi si aggiungono le scelte relative al linguaggio del core degli agenti e all'intero stack di backend e frontend. Poiché in questa fase non sono disponibili le credenziali AWS, ogni servizio cloud è sostituito da un equivalente locale scelto in modo che la migrazione futura cambi la configurazione, non il codice applicativo. Di seguito le motivazioni.

Python. È il linguaggio scelto per il core degli agenti. Oltre ai vantaggi tecnici consolidati (ecosistema maturo per l'integrazione con modelli linguistici tramite LangGraph, LangChain

e SDK dei provider, tipizzazione graduale con i type hints, ampia disponibilità di librerie per parsing e testing) la scelta è guidata da un criterio pragmatico: è un linguaggio che il team già conosceva. Con i tempi ristretti del PoC, evitare la curva di apprendimento di un nuovo linguaggio sulla parte più rischiosa del sistema ha permesso di concentrare lo sforzo sulla validazione del rischio tecnico reale.

MongoDB. È il database scelto per la persistenza dei report. È un database documentale (NoSQL) particolarmente adatto qui perché l'oggetto **Report** è naturalmente un documento semi-strutturato, con blocchi di tipo variabile e campi opzionali: rappresentarlo come documento JSON evita il sovraccarico di normalizzazione di uno schema relazionale rigido, mantenendo la validazione della struttura a livello applicativo. È inoltre coerente con l'ecosistema Node.js/TypeScript del backend, per cui esistono driver e ODM (Mongoose) maturi e ben integrati con NestJS. Nel PoC gira come container locale; in produzione il target è **MongoDB Atlas** (non DocumentDB, valutato e scartato: si veda § 31 per la motivazione) raggiunto via AWS PrivateLink, che riusa lo stesso driver Mongoose senza alcuno spike di compatibilità.

Infrastruttura: Docker Compose per il PoC, AWS in produzione. Nel PoC l'infrastruttura è containerizzata: frontend, backend, servizio agenti, database e servizi di appoggio girano in container orchestrati localmente con Docker Compose. Scelta transitoria, motivata dalla necessità di iterare rapidamente senza i tempi e i costi di provisioning cloud. La migrazione ad AWS, allora indicata come obiettivo delle fasi successive, è oggi progettata per intero nella Parte IV (Infrastruttura AWS), la cui Tabella 21 sostituisce la corrispondenza uno-a-uno originariamente qui presente con la mappatura reale, verificata sui servizi effettivamente provisionati.

Provider LLM: API gestita, non esecuzione locale. Nella prima stesura l'agente era vincolato all'uso di Claude tramite l'API diretta di Anthropic; la seconda stesura aveva ripiegato su un modello open source servito localmente. Entrambe le impostazioni sono state riviste in fase di progettazione, per una ragione precisa: **un modello eseguito localmente non produrrebbe conoscenza utile**. Il PoC deve dirci se il prodotto è realizzabile con le tecnologie di capitolato; un modello ridotto su CPU darebbe latenze, costi e qualità che non somigliano a nulla di ciò che vedremo in produzione, e i prompt tarati su quel modello andrebbero riscritti quando si passa a Bedrock.

Il PoC utilizza quindi un'**API cloud gestita** (ad esempio Qwen, tramite l'endpoint DashScope di Alibaba Cloud Model Studio), raggiunta via HTTPS con chiave di servizio, dietro l'astrazione **LLMProvider**. Il criterio di scelta del fornitore non è arbitrario: si adotta la **stessa famiglia di modelli** che sarà poi servita da Bedrock. Bedrock, infatti, non è un modello ma un canale di distribuzione: serve modelli di terze parti sotto contratto AWS. Usando l'API diretta dello stesso fornitore durante il PoC, il comportamento osservato (prompt, tool-calling, consumo di token) è quello vero, e la migrazione a Bedrock si riduce al trasporto. L'astrazione **LLMProvider** resta il punto di forza: cambiare provider è un cambio di configurazione, non una riscrittura.

L'esempio di Qwen è concreto proprio perché supera questo criterio: i modelli Qwen3 sono già disponibili come modelli gestiti su Amazon Bedrock (Milano inclusa tra le regioni), e la famiglia Qwen3-Coder è orientata all'analisi di codice e al *function calling*, il meccanismo su cui poggia l'intera integrazione GitHub descritta in § 6. Durante il PoC il modello si raggiunge via DashScope in modalità compatibile con l'API OpenAI, il che permette all'adapter di riusare l'SDK OpenAI cambiando solo `base_url` e chiave; alla migrazione si attiva **BedrockProvider** puntando allo stesso modello, senza toccare prompt né grafi. La scelta del taglio specifico (ad esempio Qwen3-Coder nella variante da 30B o 480B) e la sua validazione sul golden set restano

da confermare: il requisito di accuratezza $\geq 85\%$ (RQ.5) si misura sul modello effettivamente adottato, non si assume.

Backend: TypeScript, Node.js, NestJS. Il backend è in TypeScript su runtime Node.js, con NestJS. Il framework impone una struttura modulare (moduli, controller, provider/service) che riduce il rischio di disorganizzazione quando più persone contribuiscono in parallelo; l'iniezione delle dipendenze nativa semplifica anche l'introduzione dei test. TypeScript condiviso tra backend e frontend permette di riusare le stesse definizioni di tipo per il contratto **Report**, riducendo il disallineamento tra le due parti dello stack.

Frontend. L'interfaccia grafica è in React. Alcuni strumenti dell'ecosistema sono stati indicati come fortemente consigliati dall'azienda committente, che offre supporto diretto in caso di difficoltà:

- **Vite:** bundler/dev-server scelto per i tempi di avvio e hot-reload ridotti. Per il **Proof of Concept (PoC)** è stato adottato nell'impostazione standard (*vanilla*) mediante il plugin ufficiale `@vitejs/plugin-react`. L'integrazione con i motori **Oxc** e **Rolldown**, indicati dall'azienda con garanzia di supporto, è prevista per i successivi sviluppi ma non impiegata nella versione attuale del PoC, riducendo così il rischio legato alla loro maturità e documentazione.
- **pnpm:** gestore di pacchetti scelto per la gestione efficiente delle dipendenze (store condiviso e deduplicato, installazioni rapide) e per coerenza con le convenzioni interne dell'azienda.
- **TanStack Router:** router per React scelto per la tipizzazione nativa di rotte e parametri, integrata con TypeScript. Anche questa è una scelta indicata dall'azienda.

4 Architettura del PoC

4.1 Il perimetro in un colpo d'occhio

Il PoC è una web app locale containerizzata: questa sottosezione descrive la topologia *di sviluppo*. La topologia *di produzione* (VPC, ECS su Fargate, CloudFront) è progettata per intero in § 30–§ 34; i due livelli condividono le stesse immagini Docker e lo stesso codice applicativo (Principio di Migrazione, § 38), quindi quanto segue resta valido come descrizione dei componenti, non della loro collocazione fisica in produzione. L'utente interagisce con l'interfaccia grafica (React), che chiama il backend (NestJS) tramite API REST e riceve gli aggiornamenti di avanzamento via WebSocket. Il backend custodisce il token GitHub cifrato, avvia l'agente richiesto, ne riceve il report, lo persiste su MongoDB e lo restituisce al frontend. L'agente, al proprio interno, usa il provider LLM (via API gestita) e legge dal repository **non** in autonomia, ma chiedendo al backend attraverso la facade di sola lettura. Questa inversione è il cuore della decisione ADR-1 (§ 10) ed è ciò che tiene il token confinato in un solo servizio.

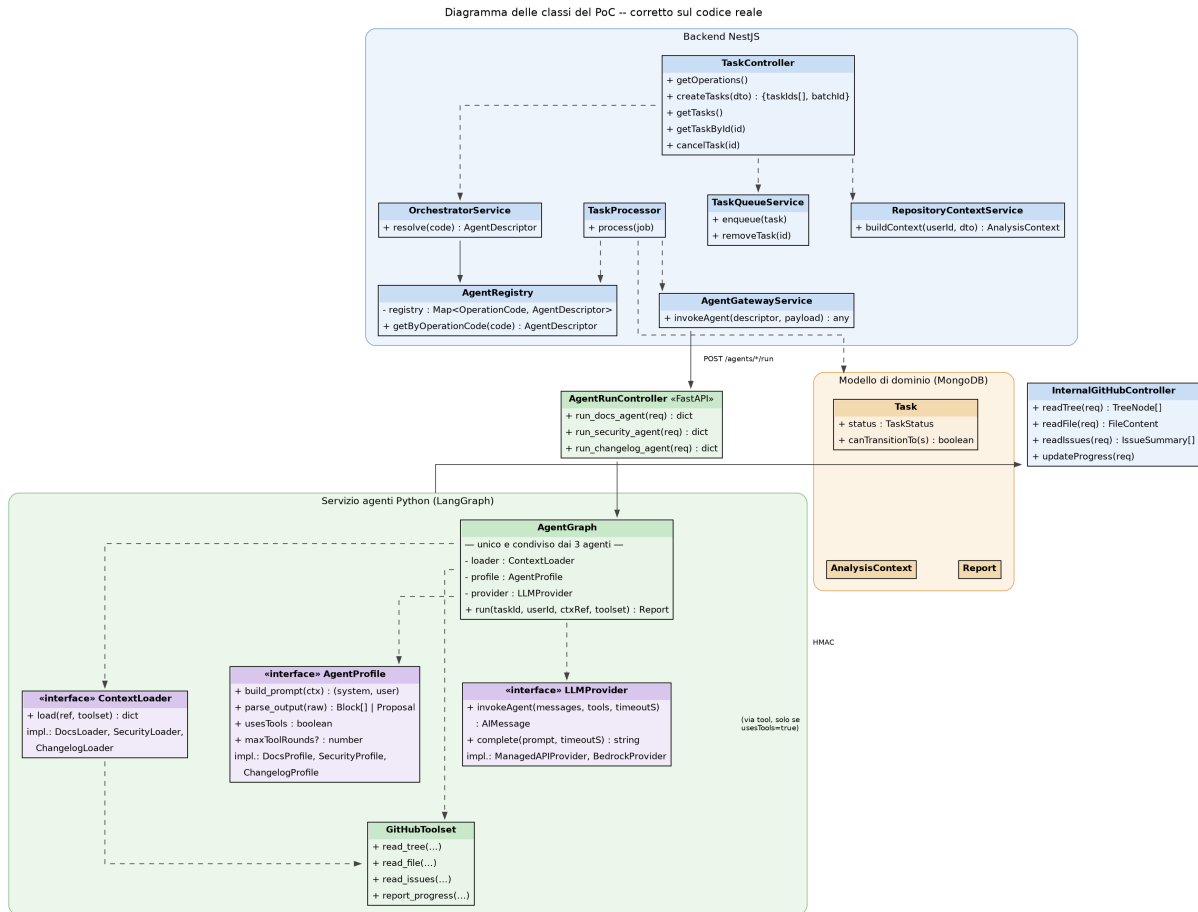


Figura 1: Diagramma delle classi del PoC.

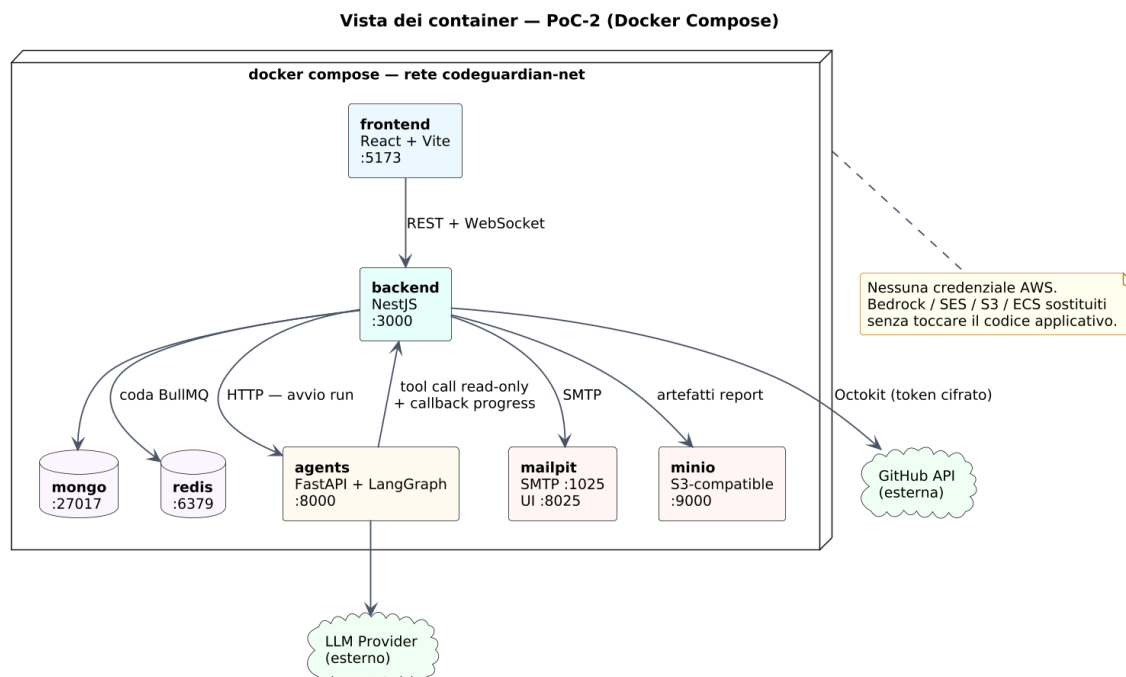


Figura 2: Architettura a container. Tre servizi portano codice nostro (frontend, backend, agents); i restanti sono infrastruttura. Le uniche connessioni verso l'esterno sono backend→GitHub e agents→provider LLM. La freccia agents→backend (traffico inverso) è la facade di lettura descritta in § 6.

4.2 Un solo scheletro, tre configurazioni

L'osservazione che rende economico coprire tutti e tre gli agenti resta valida: i tre agenti condividono la stessa struttura interna. Il grafo LangGraph è fisso, ed è più articolato di un semplice “carica contesto → costruisci prompt → chiama il modello”: oltre ai nodi di caricamento contesto, composizione del prompt, invocazione del modello, validazione/parsing e assemblaggio del report, il grafo include un nodo di **esecuzione dei tool** con un arco condizionale che, quando il modello richiede una chiamata, riporta l'esecuzione al nodo di invocazione del modello: è il loop che permette a un agente di leggere un file, ricevere il contenuto, e decidere se leggerne un altro prima di rispondere. Questo loop è predisposto nello scheletro condiviso ma viene effettivamente percorso solo dagli agenti che lo attivano. Ciò che varia da un agente all'altro resta comunque confinato in due porte (nel senso di *ports-and-adapters*). Lungo il grafo scorre un unico oggetto di stato, **AgentState**, che i nodi leggono e arricchiscono: parte da {**context_ref**, **profilo**}, si popola con il contesto caricato, il prompt, la cronologia dei messaggi scambiati col modello (comprese le eventuali chiamate a tool e i loro risultati), la risposta grezza, i blocchi o la proposta, e infine il report; un campo **error** opzionale devia verso lo stato fallito.

Le porte che variano sono due:

- **ContextLoader** — dato un riferimento, restituisce il contesto iniziale da dare al modello. Nel PoC-2 gli adattatori **non** leggono più da clone locale o fixture: leggono dal repository reale attraverso i tool della facade GitHub. Per Docs e Changelog questo passo carica *tutto* ciò che serve (rispettivamente: albero e contenuto dei file dell'ambito per Docs; le issue chiuse per Changelog), perché questi due agenti non hanno tool a disposizione durante l'esecuzione del grafo e devono ricevere l'intero materiale già pronto nel prompt. Per Security, invece, **ContextLoader** carica solo l'albero del repository (e l'eventuale **POLICY.md**): il contenuto dei singoli file non viene letto in questo passo, ma più avanti nel grafo, durante il ciclo di invocazione del modello, tramite i tool descritti sotto.

- **AgentProfile** — incapsula le cose specifiche di ogni agente: costruire il prompt (caricando il template esterno e iniettando il contesto: è qui che vive l'isolamento dei prompt) e rileggere la risposta (validarla e trasformarla nei blocchi del report secondo lo schema dell'agente). Il profilo dichiara inoltre se l'agente ha accesso ai tool di lettura durante l'esecuzione (`usesTools`) e, in caso affermativo, quante iterazioni del ciclo modello→tool→modello sono ammesse prima di considerare l'agente fallito (`maxToolRounds`). È questo flag, letto dallo scheletro condiviso, a decidere se il grafo percorre il nodo di esecuzione tool oppure lo salta: per Docs e Changelog è disattivato, per Security è attivo con un tetto di iterazioni.

C'è poi una terza porta, `LLMProvider`, condivisa e identica per tutti: cambia solo per configurazione, non per agente. È questa astrazione a rendere il passaggio a Bedrock un cambio di configurazione. Il metodo principale non è una singola chiamata testo-a-testo, ma un'invocazione che riceve anche l'elenco dei tool disponibili (eventualmente vuoto) e la cronologia dei messaggi, così da poter restituire sia una risposta finale sia una richiesta di chiamata a un tool.

Rispetto alla prima stesura, il core dell'agente non è più invocato da un driver a riga di comando, ma da un use case del backend NestJS, che inietta `context.ref` e `profilo`, avvia il grafo ed espone il `Report` risultante come risposta REST.

```
// Porte (interfacce) -- l'unico codice che varia per agente
interface ContextLoader { load(ref): LoadedContext } // via facade
    GitHub
interface AgentProfile { build_prompt(ctx): Prompt
                        parse_output(raw): Block[] | Proposal
                        usesTools: boolean // false per
                        Docs, Changelog
                        maxToolRounds?: number } // usato solo
                        se usesTools = true

interface LLMProvider { // condiviso
    invokeAgent(messages: Message[], tools: Tool[], timeoutS): AIMessage
    complete(prompt: string, timeoutS): string // scorciatoia
    quando tools = []
}

// Composizione degli adapter per agente (3 Loader, 7 Profile -- uno per
// OperationCode)
Docs/README -> GitHubCodeLoader + DocsReadmeProfile (
    usesTools: false)
Docs/Inline -> GitHubCodeLoader + DocsInlineProfile (
    usesTools: false)
Docs/API -> GitHubCodeLoader + DocsApiProfile (
    usesTools: false)
Security/OWASP -> GitHubTreeLoader + OwaspscanProfile (
    usesTools: true, maxToolRounds: 8)
Security/Policy -> GitHubTreeLoader + SecurityPolicyProfile (
    usesTools: true, maxToolRounds: 8)
                                (legge anche POLICY.md)
Changelog/Tech -> GitHubIssuesLoader + ChangelogTechnicalProfile (
    usesTools: false)
Changelog/Biz -> GitHubIssuesLoader + ChangelogBusinessProfile (
    usesTools: false)

% Backend NestJS -- sostituisce il driver CLI
POST /contexts { repoOwner, repoName, ref, scopeType, paths? } -> 201 {
    contextId }
GET /operations -> elenco OperationDescriptorDto disponibili per l'UI
POST /tasks { contextId, operations: OperationCode[] } -> 202 { taskIds:
```

```

    string[], batchId }
GET  /tasks?{query} -> TaskSummaryDto []
GET  /tasks/:id -> TaskDetailDto
POST /tasks/:id/cancel -> void
GET  /reports/:id -> Report persistito su MongoDB
GET  /reports/:id/export?format=md -> file scaricabile (Report ->
    Markdown)
WS    task.progress / task.updated -> avanzamento e stato finale (non
    polling)

```

Listing 1: Porte dell'agente e composizione degli adapter. `ContextLoader` e `AgentProfile` variano per agente; il ciclo di tool-calling è parte dello scheletro condiviso ma è attivo solo per i profili con `usesTools=true`.

Il frontend costruisce prima l'`AnalysisContext` tramite `POST /contexts` (repository, ambito, riferimento), quindi avvia una o più operazioni con `POST /tasks`, che risponde immediatamente con gli identificativi delle task create in stato `PENDING` e non con il report. L'avanzamento e la transizione di stato (`COMPLETED`, `FAILED`, `CANCELLED`) arrivano via WebSocket (`task.progress`, `task.updated`), non tramite interrogazione ripetuta di `GET /reports/:id`. Gli endpoint `/internal/*` restano riservati alla comunicazione agente→backend e non sono mai chiamati dal frontend.

4.3 Modello di dominio

Il diagramma in figura 3 mostra le entità persistite su MongoDB. Al centro c'è **Task**: ogni esecuzione richiesta dall'utente ne è un'istanza. Attorno ruotano **AnalysisContext** (su cosa si lavora: repository, riferimento risolto a SHA, ambito), **OperationCode** (cosa si fa: la chiave che l'orchestratore usa per instradare) e **Report** (cosa ne è uscito). L'**AccessLog** registra ogni lettura effettuata su GitHub per conto di una task – nello schema reale i campi sono `taskId`, `endpoint` e `resource` (non esistono un `userId` né un campo `operation` propri del log: l'endpoint richiamato già identifica il tipo di lettura) – ed è l'evidenza che rende verificabile il vincolo di sola lettura.

AnalysisContext è deliberatamente persistito e non passato inline: più task dello stesso avvio lo condividono senza duplicarlo, e il pin del riferimento a uno SHA rende i report riproducibili anche dopo nuovi commit sul repository. Il campo opzionale `sprintId` è usato solo dall'operazione Changelog, per filtrare le issue sulla milestone richiesta. **AnalysisContext** non espone alcun metodo di validazione dimensionale: il controllo sulla dimensione del contesto (`max_scope_chars`, fissato a **60.000 caratteri**, equivalenti a circa 15.000 token per i modelli Qwen3) è implementato lato agente Python, nel nodo *componi_prompt* del grafo LangGraph.

Il **Report** è un envelope unico e generico: qualunque sia l'agente che lo produce, la struttura è sempre `{..., body: Block[], proposal?: Proposal, error?: ReportError}`.

Infine, **ServiceCredential** custodisce il token GitHub cifrato: oltre a `ciphertext` e `authTag`, lo schema reale include `iv` (vettore di inizializzazione) e `salt`, entrambi necessari alla cifratura AES; il campo `provider` è una stringa libera (di fatto sempre "GITHUB" in questo PoC), non un'enumerazione chiusa. Gli sviluppatori implementano questi schemi con Mongoose.

5 Il contratto di output condiviso

Il nodo finale del grafo, *Assembla report*, produce sempre lo stesso oggetto **Report**, qualunque sia l'agente. È questo a rendere i tre output un contratto unico: sia il backend sia il frontend trattano sempre un **Report**, senza sapere quale agente l'ha prodotto. La struttura è un *envelope*: un'intestazione comune, un corpo fatto di blocchi ordinati, e un eventuale artefatto di proposta (il diff). Il **Report** viene persistito su MongoDB dal backend e reso via API REST al frontend, che lo renderizza nell'interfaccia grafica.

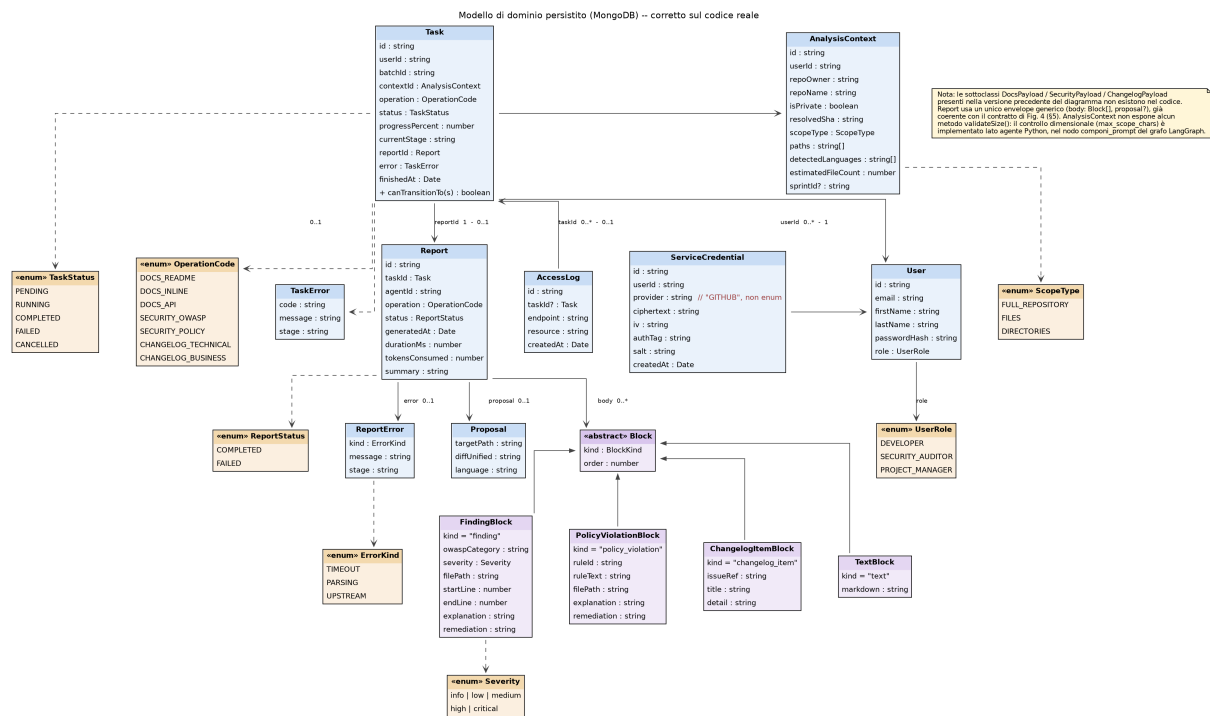


Figura 3: Modello di dominio persistito su MongoDB, corretto sul codice reale. Le sottoclassi Payload sono state rimosse (ridondanti con la figura 4); AccessLog e ServiceCredential riportano i campi effettivamente presenti negli schemi Mongooose.

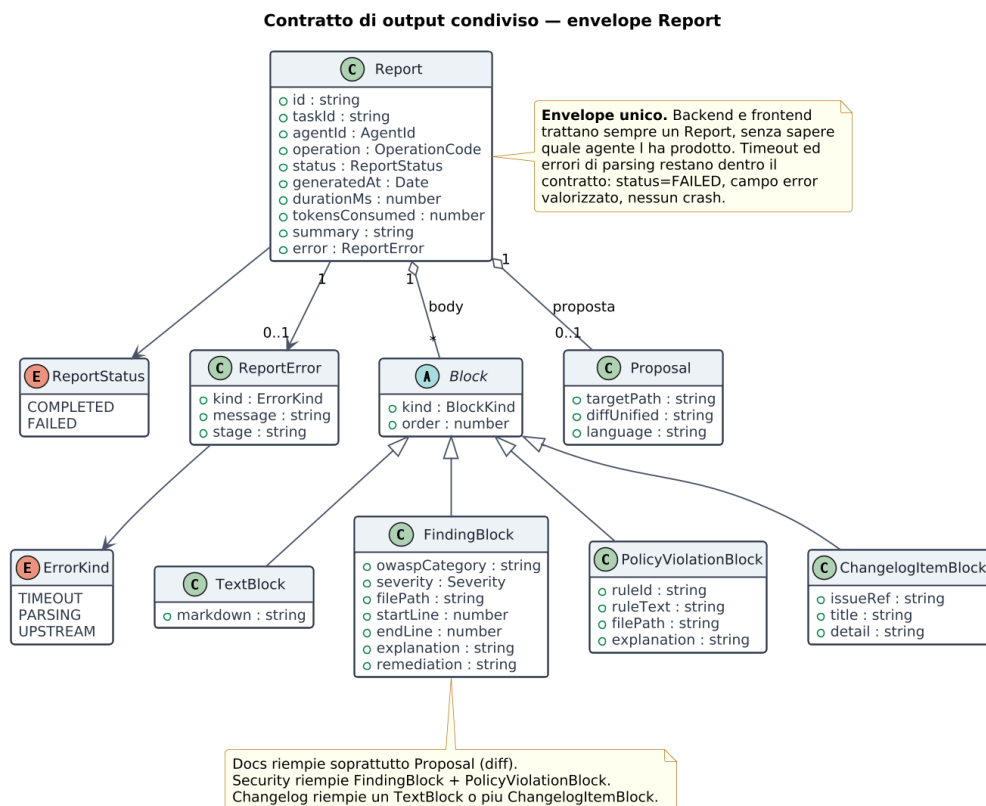


Figura 4: Il contratto **Report**: envelope comune, blocchi polimorfi nel corpo, proposta opzionale. Timeout ed errori di parsing restano dentro il contratto (**status=FAILED**, campo **error** valorizzato), senza provocare crash.

Ogni agente riempie lo stesso envelope in modo diverso: Security con `FindingBlock` ed eventuale `PolicyViolationBlock`; Docs soprattutto con una `Proposal` di diff; Changelog con un `TextBlock` o più `ChangelogItemBlock`. In tutti i casi, timeout ed errori di parsing non provocano crash: valorizzano `error` e portano `status` a `FAILED`, restando dentro il contratto; il frontend mostra questi stati con un componente di errore dedicato, senza gestione speciale lato backend.

6 Integrazione con GitHub

Questa è la sezione che gli sviluppatori devono leggere con più attenzione, perché è dove è più facile sbagliare in un modo che viola un vincolo di sicurezza. La regola da tenere ferma è una sola: **il token GitHub non lascia mai il backend NestJS, e gli agenti non chiamano mai GitHub direttamente.**

6.1 Come l'agente legge dal repository

L'agente non possiede il token e non conosce l'URL di GitHub. Quando ha bisogno di leggere qualcosa, invoca un *tool* LangGraph che, sotto il cofano, chiama un endpoint interno del backend. È il backend a decifrare il token, chiamare GitHub in sola lettura, registrare l'accesso e restituire il contenuto. La figura 5 mostra le classi coinvolte.

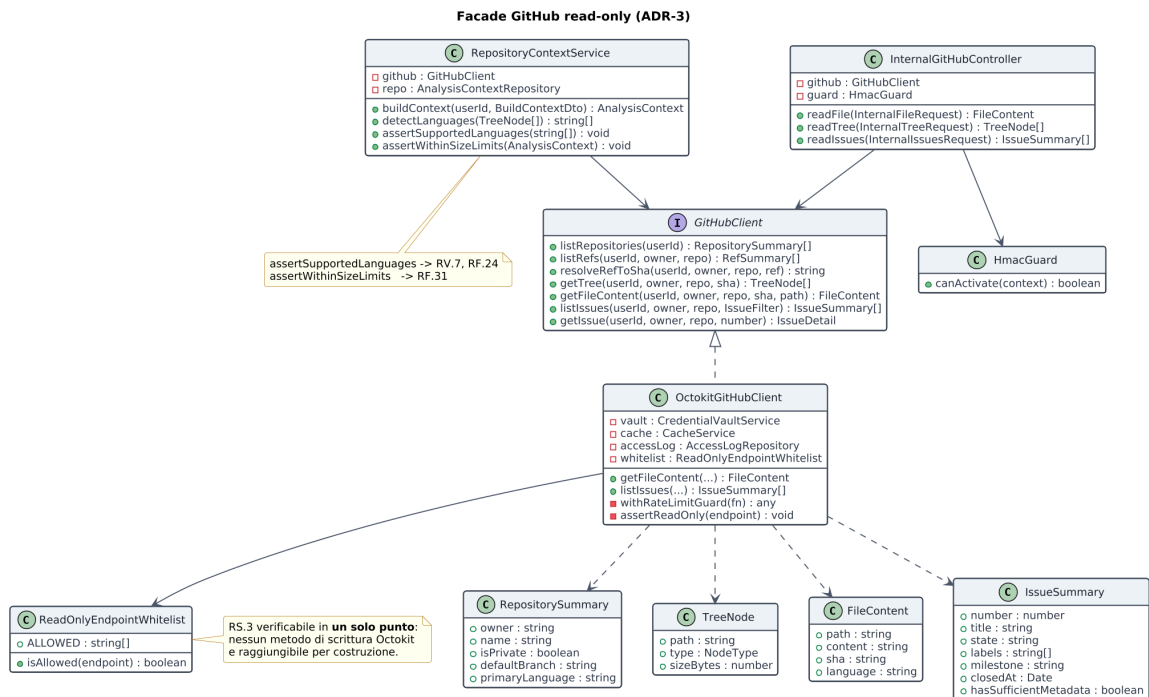


Figura 5: La facade GitHub. `ReadOnlyEndpointWhitelist` rende il vincolo di sola lettura verificabile in un solo punto: nessun metodo di scrittura è raggiungibile per costruzione. Gli endpoint `/internal/*` sono usati dagli agenti e protetti da HMAC.

I tool disponibili all'agente sono tre, tutti di lettura: `read_tree` (struttura del repository a un dato SHA), `read_file` (contenuto di un file) e `read_issues` (elenco e dettaglio delle issue, filtrabili). Mentre l'agente Security sfrutta un vero tool-calling autonomo e iterativo per esplorare i file, gli agenti Docs e Changelog seguono un flusso deterministico: un loader Python pre-carica interamente il contesto (file di codice o issue) prima dell'invocazione del modello, al quale viene esplicitamente vietato l'uso di tool esterni, ed è ciò che le stesure precedenti non dimostravano.

6.2 I quattro controlli su ogni chiamata

Ogni lettura verso GitHub attraversa quattro meccanismi, che gli sviluppatori implementano una sola volta in `OctokitGitHubClient`:

- **Whitelist** (`ReadOnlyEndpointWhitelist`): se un endpoint non è nell'elenco chiuso di operazioni di lettura, la chiamata non parte. Il vincolo RS.3 non dipende dall'attenzione di chi scrive il codice, ma è garantito per costruzione.
- **Cache Redis** con chiave `repo@sha:path`: poiché il contesto è pinnato a uno SHA, il contenuto di un file a quel commit è immutabile e la cache non può servire dati stantii. Serve a contenere il consumo di rate limit quando l'agente rilegge lo stesso file in iterazioni diverse.
- **Rate limit guard**: legge le intestazioni di quota di GitHub e, in prossimità dell'esaurimento, rallenta invece di fallire.
- **AccessLog**: ogni chiamata lascia una riga con task, endpoint e risorsa. È l'evidenza che rende RS.3 dimostrabile invece che dichiarata.

6.3 Autenticazione degli endpoint interni

Gli endpoint `/internal/github/*` e `/internal/tasks/:id/progress` non sono esposti al frontend e non compaiono nello Swagger pubblico. Sono raggiungibili solo dalla rete Docker e autenticati con **HMAC-SHA256** tramite un segreto condiviso tra backend e servizio agenti (`INTERNAL_SHARED_SECRET`), perché il chiamante non è un utente ma un servizio. Gli sviluppatori del servizio agenti firmano ogni richiesta; gli sviluppatori del backend verificano la firma in un guard dedicato (`InternalAuthGuard`).

Il messaggio firmato ha la struttura:

```
HMAC-SHA256(timestamp + ":" + method + ":" + path + ":" + bodyHash,  
INTERNAL_SHARED_SECRET)
```

dove `timestamp` è lo Unix timestamp in secondi (stringa), `method` è il verbo HTTP in maiuscolo (POST, GET), `path` è il path completo senza query string (es. `/internal/tasks/abc123/progress`) e `bodyHash` è lo SHA-256 hex del body JSON (stringa vuota "" per request senza body). La finestra anti-replay è di $\pm$ `HMAC_WINDOW_S` secondi (default 30). La request include gli header:

```
X-Internal-Timestamp: <timestamp> X-Internal-Signature: <hmac-hex>
```

6.4 Il caso dell'Agente Docs

L'Agente Docs merita una nota, perché è l'unico che *produce un file che potrebbe già esistere* nel repository, e quindi il vincolo di sola lettura sembra confliggere con la sua funzione. Non è così: l'agente legge dal repository ma non ci scrive.

Agente Docs -- dialogo con GitHub e destinazione dell'output (corretto sul codice reale)

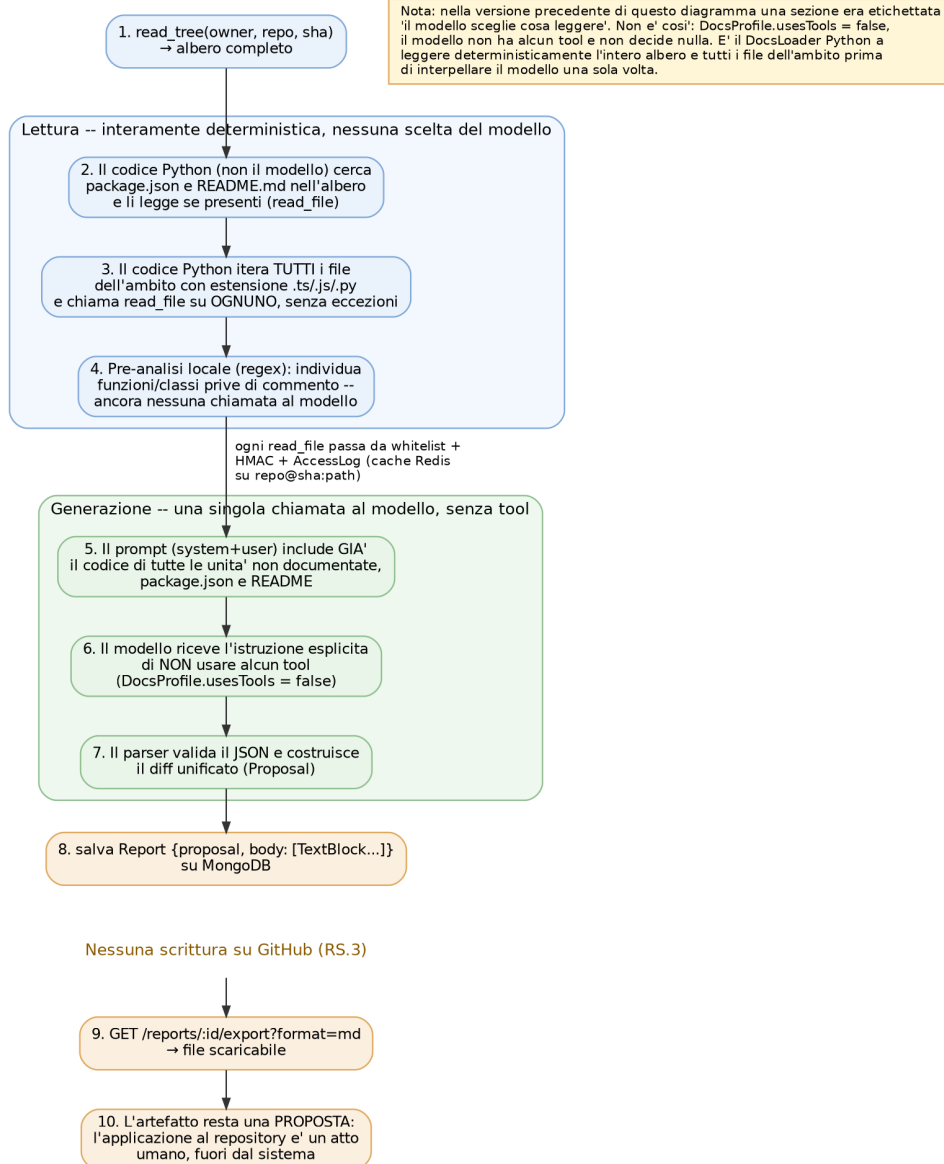


Figura 6: Agente Docs: legge in modo più esteso degli altri (deve prima capire che progetto è), ma in scrittura produce solo una proposta di diff salvata come **Report**. Nel PoC l'applicazione al repository è un atto umano (nessuna PR viene aperta). **Aggiornamento MVP**: il backend apre la Pull Request automaticamente ricevuta la **Proposal** (ADR-AWS-10, § 39); il *merge* resta un atto umano condotto su GitHub.

In lettura, l'Agente Docs ha un profilo diverso dagli altri: mentre Security legge pochi file mirati in profondità, Docs deve prima capire *che progetto è*, quindi parte con un'esplorazione a largo raggio (albero completo, `package.json`, `README` esistente). La presenza di un `README` trasforma il compito da generazione ad aggiornamento. In scrittura, l'output è un **Proposal** (diff verso il file di destinazione) salvato come **Report**: l'applicazione al repository resta un atto umano, coerente con RS.3.

7 Contratto delle API: comunicazione frontend–backend

Questa sezione è il riferimento operativo per chi implementa il backend (deve realizzare questi endpoint) e per il progettista frontend (deve sapere quali chiamare da ogni pagina). Elenca il

contratto in modo che le due parti possano procedere in parallelo senza ambiguità.

7.1 Due piani di comunicazione distinti

Il punto da cui partire, perché è la fonte di errore più comune: esistono **due piani di comunicazione separati**, e il frontend ne usa uno solo.

Il frontend è un'applicazione React che gira **nel browser dell'utente**, non dentro la rete Docker. Raggiunge quindi il backend attraverso la porta esposta sull'host (`localhost:3000`), non attraverso l'hostname interno del container. Gli agenti Python, invece, vivono *dentro* la rete Docker e raggiungono il backend all'hostname interno (`backend:3000`) su endpoint diversi, protetti da HMAC (§ 6). Il frontend non tocca mai gli endpoint interni, e gli agenti non toccano mai quelli pubblici: è questa separazione a garantire che il token GitHub non lasci il backend (ADR-1).

Tabella 2: Indirizzi base dei servizi in ambiente di sviluppo.

Servizio	Dal browser	Nella rete Docker	
Frontend (Vite)	<code>http://localhost:5173</code>	<code>http://frontend:5173</code>	
Backend (NestJS)	<code>http://localhost:3000</code>	<code>http://backend:3000</code>	
Agenti (FastAPI)	— (non esposto)	<code>http://agents:8000</code>	

Il frontend non ha l'indirizzo del backend cablato nel codice: lo legge dalla variabile `VITE_API_BASE_URL`, così in produzione cambia senza ricompilare. Il backend adotta un prefisso globale versionato (`app.setGlobalPrefix('api/v1')`), quindi la base REST effettiva è `http://localhost:3000/api/v1`. Sia il prefisso `/api/v1` sia la scelta del canale realtime (WebSocket, § 7.3) sono proposte del progettista: se il team decide diversamente, questa sezione è il punto in cui aggiornarle.

7.2 API REST pubbliche (browser → backend)

Tutti i path che seguono sono relativi alla base `/api/v1` e documentati in Swagger, raggiungibile durante lo sviluppo a `http://localhost:3000/api/docs`. Le chiamate autenticate portano il JWT nell'header `Authorization: Bearer`.

Tabella 3: Endpoint REST pubblici, raggruppati per pagina del frontend.

Metodo e path	Scopo	Requisiti
<i>Autenticazione</i>		
POST /auth/register	Registrazione di un nuovo utente (stub nel PoC, si veda nota)	RF.1–RF.6
POST /auth/login	Login, restituisce il JWT (stub nel PoC, si veda nota)	RF.7–RF.9
GET /auth/me	Profilo dell'utente autenticato	RF.9
GET /auth/health	Healthcheck del backend (uso interno, Docker Compose)	—
<i>Credenziali di servizio</i>		
POST /credentials	Salva il token GitHub (cifrato)	RF.10–RF.11, RS.2
GET /credentials	Elenco delle credenziali configurate	RF.12
POST /credentials/:id/validate	Verifica validità e scope	RF.12, RS.4
DELETE /credentials/:id	Revoca una credenziale	RF.12
<i>Selezione del repository e dell'ambito</i>		
GET /repositories	Elenco dei repository dell'utente	RF.21–RF.23
GET /repositories/:owner/:repo/refs	Branch e tag disponibili	RF.24
GET /repositories/:owner/:repo/tree?ref=	Struttura del repository	RF.24–RF.30
POST /contexts	Crea l'AnalysisContext, ritorna contextId	RF.25–RF.31
<i>Avvio ed esecuzione</i>		
GET /operations	Operazioni disponibili (per i pulsanti)	RF.32–RF.34
POST /tasks	Avvia una o più task in batch, ritorna 202 + {taskId[], batchId}	RF.35–RF.42
GET /tasks	Elenco delle task per la dashboard	RF.43–RF.45
GET /tasks/:id	Dettaglio di una singola task	RF.45
POST /tasks/:id/cancel	Annulla una task	RF.46
<i>Storico e report</i>		
GET /reports?operation=&from=&to=	Storico dei report, filtrabile	RF.49–RF.51
GET /reports/:id	Dettaglio di un re-	UC33

POST `/auth/register` e POST `/auth/login` sono implementati come **stub**: il primo restituisce un messaggio statico senza creare alcun utente, il secondo firma sempre lo stesso JWT per un utente fisso, senza verificare alcuna credenziale. È una scelta coerente con il perimetro dichiarato in § 3 (“il PoC assume un utente già noto”), ma va tenuta presente da chi consuma questo contratto: gli endpoint esistono e rispondono, ma non implementano ancora RF.1–RF.9.

7.3 Canale realtime (browser → backend)

L’avanzamento delle task arriva al frontend via WebSocket (Socket.IO), coerentemente con l’esecuzione asincrona su coda (ADR-3): la POST `/tasks` ritorna subito, e il frontend si iscrive al canale usando i `taskId` ricevuti. La connessione è a `ws://localhost:3000` e viene autenticata passando il JWT all’handshake. Il server emette quattro eventi verso il client nel PoC; il frontend non emette eventi applicativi, solo iscrizione. **Nota MVP:** la Parte VI (§ 48) aggiunge un quinto evento (`task.inputRequired`) e costituisce il contratto vincolante per l’implementazione.

Tabella 4: Eventi WebSocket emessi dal backend verso il frontend.

Evento	Payload	Uso nel frontend
<code>task.updated</code>	<code>{ taskId, status, reportId? }</code>	Aggiorna il badge di stato in dashboard
<code>task.progress</code>	<code>{ taskId, stage, percent }</code>	Muove la barra di avanzamento
<code>task.failed</code>	<code>{ taskId, error }</code>	Mostra un toast; non blocca le altre task
<code>batch.completed</code>	<code>{ batchId, completed, failed }</code>	Abilita i controlli successivi (RF.47)

7.4 Gli schemi dei DTO principali

Per tipizzare le chiamate lato frontend bastano gli schemi degli oggetti che attraversano il confine. Gli altri (elenchi, riassunti) sono proiezioni di questi. Il contratto **Report** completo è in § 5, figura 4.

```
// --- Body di POST /contexts ---
interface CreateContextDto {
  repoOwner: string
  repoName: string
  ref: string // branch, tag o SHA
  scopeType: 'FULL_REPOSITORY' | 'FILES' | 'DIRECTORIES'
  paths?: string[] // richiesto se scopeType != FULL_REPOSITORY
  sprintId?: string // opzionale: milestone usata dall'
    agente Changelog
}

// --- Risposta di POST /contexts ---
interface AnalysisContextDto {
  id: string
  repoOwner: string
  repoName: string
  isPrivate: boolean
  resolvedSha: string // ref risolto a commit, per
    riproducibilit 
  scopeType: string
  detectedLanguages: string[]
}
```

```

    estimatedFileCount: number
}

// --- Body di POST /tasks ---
interface CreateTaskBatchDto {
    contextId: string
    operations: OperationCode[]    // una o piu' operazioni, avviate
    insieme
}

// --- Elemento restituito da GET /tasks e GET /tasks/:id ---
interface TaskDto {
    id: string
    batchStepId: string | null    // null = task avviata singolarmente
    operation: OperationCode
    status: 'PENDING' | 'RUNNING' | 'COMPLETED' | 'FAILED' | 'CANCELLED'
    progressPercent: number
    currentStage: string | null
    reportId: string | null    // valorizzato quando status =
    COMPLETED
    error: { code: string, message: string, stage: string } | null
}

// --- Risposta di GET /reports/:id (envelope, vedi fig. contratto
// Report) ---
interface ReportDto {
    id: string
    taskId: string
    operation: OperationCode
    status: 'COMPLETED' | 'FAILED'
    generatedAt: string
    summary: string
    body: Block[]    // TextBlock | FindingBlock | ...
    proposal?: { targetPath: string, diffUnified: string, language: string
    }
    error?: { kind: string, message: string, stage: string }
}

type OperationCode =
    | 'DOCS_README' | 'DOCS_INLINE' | 'DOCS_API'
    | 'SECURITY_OWASP' | 'SECURITY_POLICY'
    | 'CHANGELOG_TECHNICAL' | 'CHANGELOG_BUSINESS'

```

Listing 2: Schemi dei DTO che il frontend consuma e produce. I tipi sono condivisi tra backend e frontend riusando le stesse definizioni TypeScript.

7.5 API interne (agenti → backend)

Elencate qui per completezza, ma **il frontend non le usa** e non compaiono in Swagger. Vivono sulla rete Docker a `http://backend:3000/internal/*`, autenticate via HMAC con `INTERNAL_SHARED_SECRET`. Sono la porta con cui gli agenti leggono da GitHub (§ 6): `POST /internal/github/tree`, `POST /internal/github/file`, `POST /internal/github/issues` (i tre tool di lettura) e `POST /internal/tasks/:id/progress` (il callback di avanzamento, che il backend inoltra sul WebSocket).

Evoluzione MVP. Il contratto qui sopra è quello validato nel PoC: autenticazione e credenziali sono stub, `CreateContextDto` usa i campi `repoOwner/repoName/ref` (superati in MVP) e

non esiste un endpoint per l'input asincrono dell'utente. La **Parte VI** (§ 48) costituisce il contratto vincolante per l'implementazione MVP: autenticazione reale (Argon2id + JWT), `CreateContextDto` con `repoUrl/branch/commitSha?`, endpoint `POST /tasks/:id/input`, quinto evento WebSocket `task.inputRequired`. Il prefisso `/api/v1` e la separazione dei piani pubblico/interno restano invariati.

8 Orchestrazione ed esecuzione

8.1 Instradamento deterministico

Anche nella sua forma minima, l'instradamento operazione→agente è **deterministico**, come impone il vincolo RV.1: non c'è alcun LLM nella logica di smistamento. Il componente responsabile (`AgentRegistry`) è una mappa statica popolata a bootstrap; dato un `OperationCode`, la destinazione è determinata senza inferenza. In sede di revisione è la prima classe da mostrare, perché il vincolo si verifica a occhio.

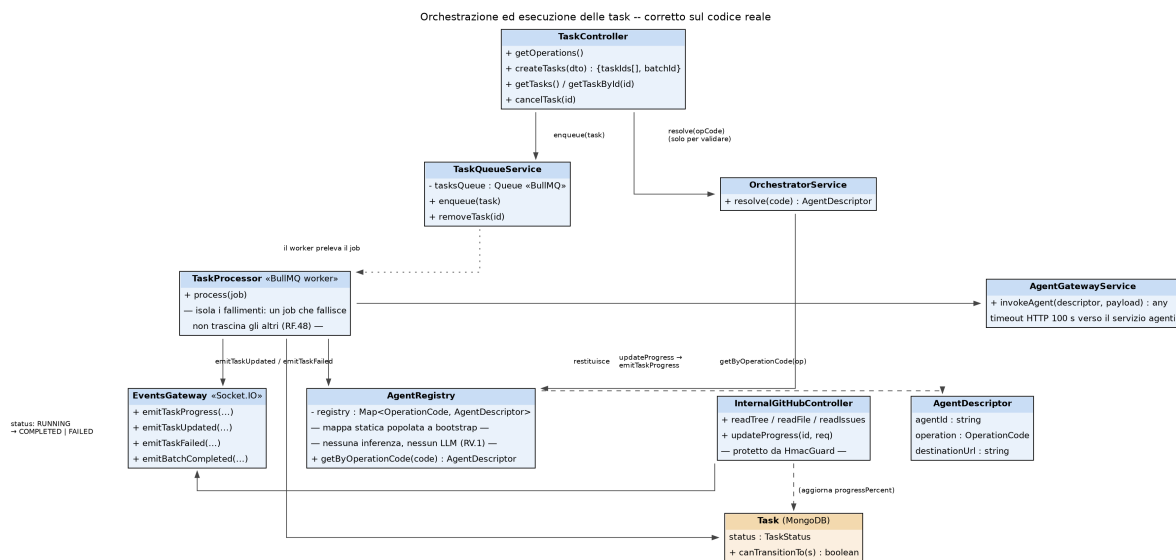


Figura 7: Orchestrazione ed esecuzione. `AgentRegistry` è il cuore di RV.1 (mappa statica, nessun LLM). Il `TaskProcessor` isola i fallimenti: un job che fallisce non trascina gli altri (RF.48).

8.2 Ciclo di vita di una Task

Una `Task` attraversa cinque stati (figura 8): le transizioni ammesse sono solo quelle disegnate, e `Task.canTransitionTo()` rifiuta le altre. Nessuno stato terminale ha uscite: un rilancio è una `Task` nuova che tiene il riferimento alla precedente, così lo storico dei report resta interpretabile.

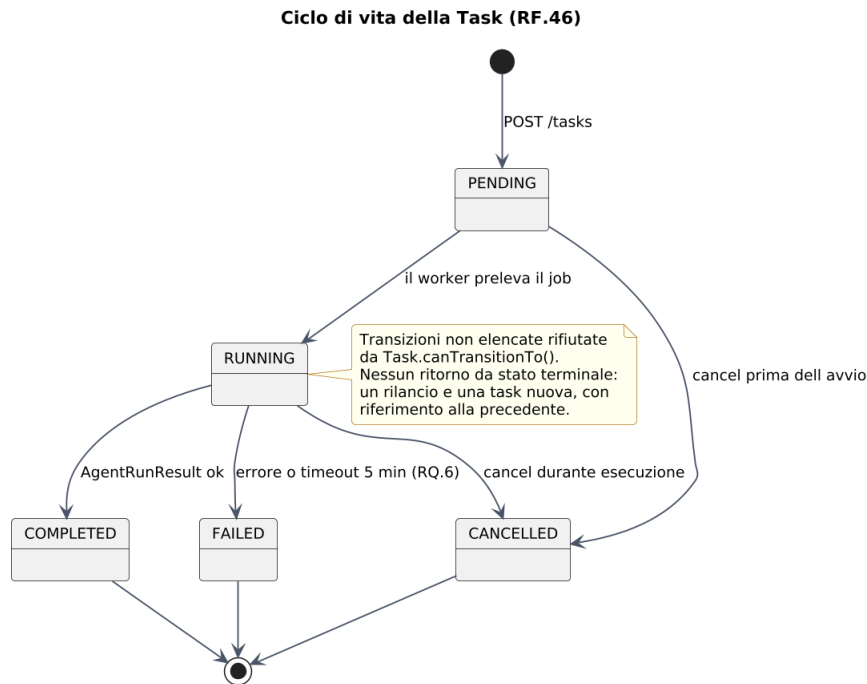


Figura 8: Ciclo di vita della Task (RF.46). Il superamento del timeout globale del task (RQ.6, 5 minuti nell’implementazione PoC) porta a **FAILED** restando dentro il contratto **Report**. I timeout per-agente configurabili via variabile d’ambiente (`AGENT_GATEWAY_TIMEOUT_DOCS_S` / `AGENT_GATEWAY_TIMEOUT_SECURITY_S`, cfr. § 10) agiscono a livello di gateway e hanno valori inferiori (90 s per Docs e Changelog, 200 s per Security). Il diagramma mostra inoltre lo stato **CANCELLED** (annullamento esplicito da parte dell’utente).

Nel PoC questi cinque stati bastano perché nessun agente richiede input a metà esecuzione. L’MVP introduce l’agente Changelog con un flusso realmente interattivo (Sprint ID, esclusione di task con metadati insufficienti, conferma del Changelog di business): la Parte II (§ 19.4, figura 16) estende questo stesso ciclo di vita con un campo `pendingInput`, una condizione sospensiva *interna* allo stato **RUNNING** e non un sesto stato: i cinque stati qui sopra restano quindi l’unica fonte di verità sulle transizioni, anche nell’MVP.

8.3 Il giro completo end-to-end

La figura 9 mostra lo scambio di messaggi per un’operazione concreta (scansione OWASP su un repository privato). Il blocco centrale è quello su cui fermarsi in revisione: l’agente chiede l’albero, *decide* di aprire un file, lo chiede, lo riceve. Nessuno di questi passi è programmato da noi: sono tool call scelte dal modello, e ognuna passa dal backend, che tiene il token e registra l’accesso. Un solo giro attraversa React, Vite, TanStack, NestJS, TypeScript, MongoDB, Redis, Python, LangGraph, Octokit, l’API GitHub, Docker, WebSocket e Swagger: è la dimostrazione di integrazione che le stesure precedenti non fornivano. Questo pattern di tool-calling guidato dal modello, qui illustrato sulla scansione OWASP, è specifico dell’agente Security: Docs e Changelog non lo attivano, perché il loro **ContextLoader** carica l’intero contesto necessario in un unico passo deterministico prima di interpellare il modello (§ 4.2).

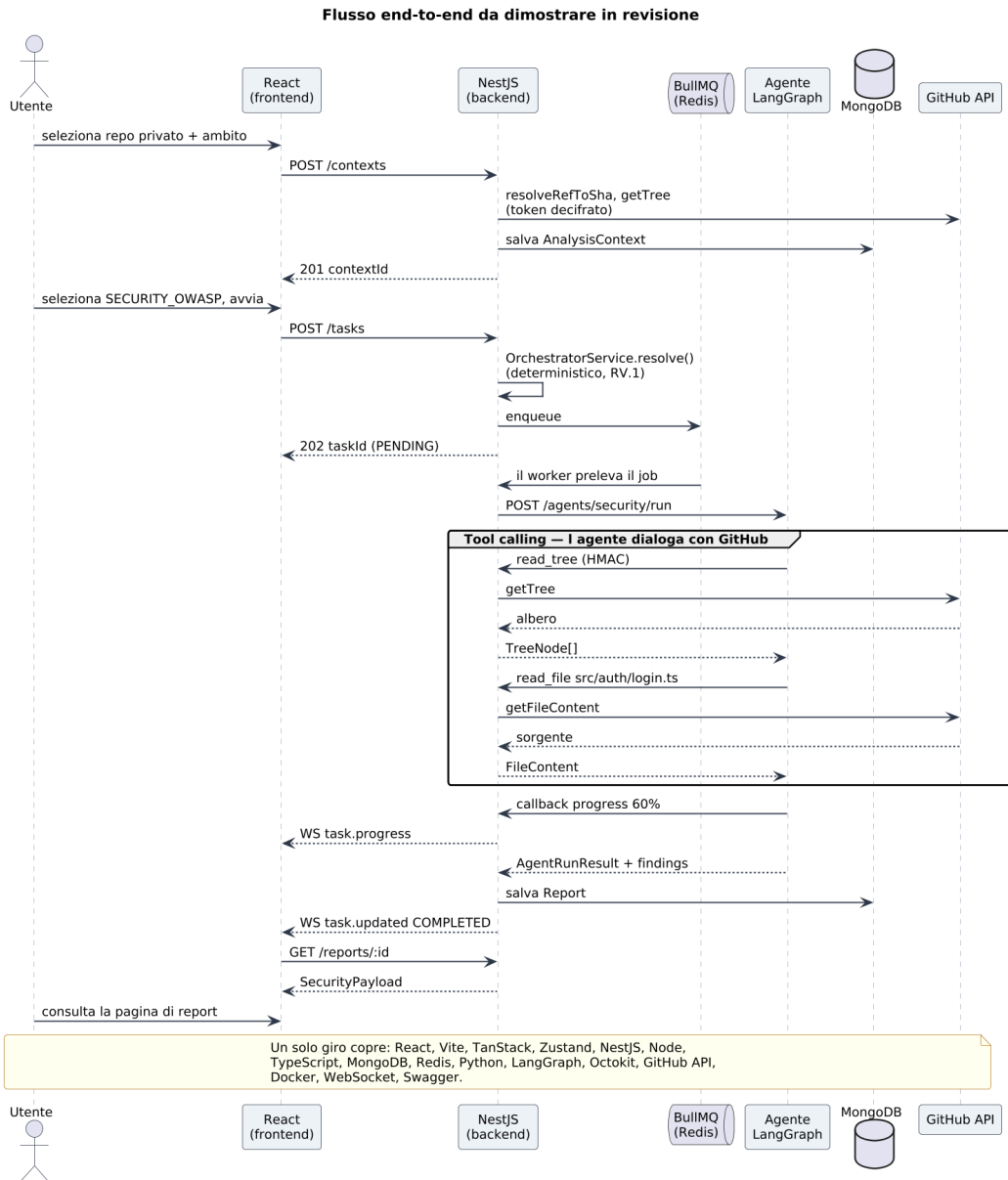


Figura 9: Sequenza end-to-end di una scansione. Il tool-calling dell’agente (blocco centrale) attraversa la facade GitHub, non GitHub direttamente.

9 Cosa implemento per ciascun agente

Poiché grafo, provider, envelope e gestione degli errori sono condivisi, “implementare un agente” significa scrivere sempre e solo tre cose: l’adattatore che carica il contesto (dai tool GitHub), il profilo con il suo template di prompt, e il parser che rilegge la risposta. L’esposizione via backend e la visualizzazione via frontend sono uguali per tutti e tre, scritte una sola volta contro il contratto **Report**. La figura 10 mostra la gerarchia.

Servizio Agenti Python — Architettura Ports & Adapters

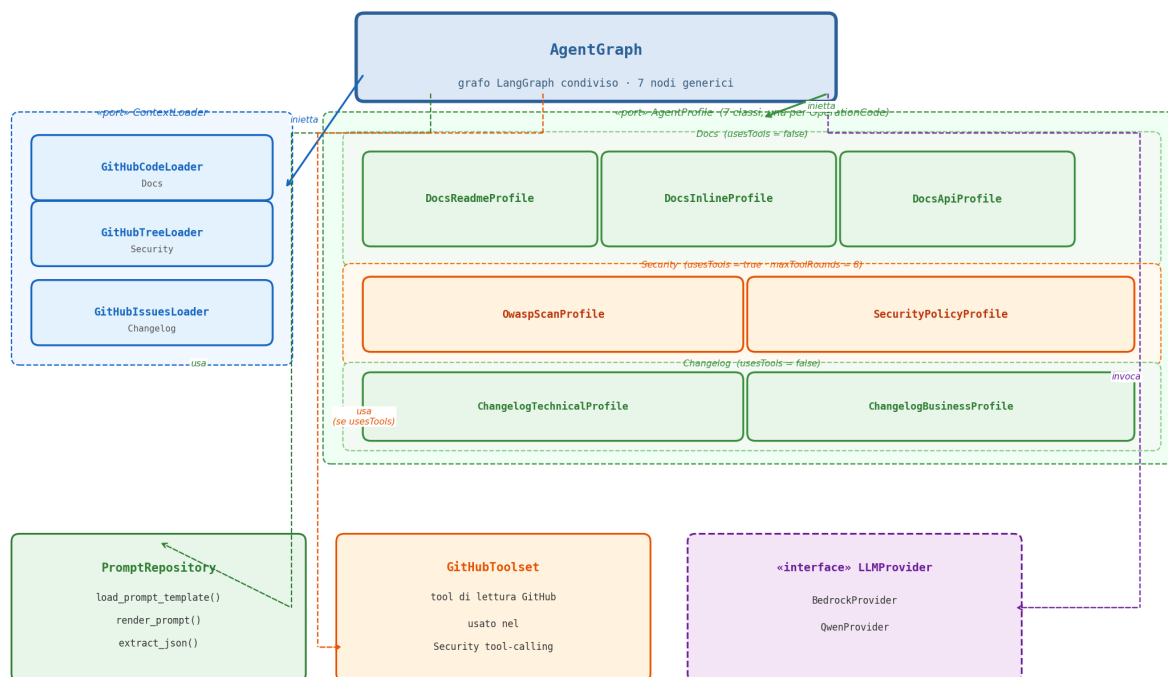


Figura 10: Il servizio agenti. Il grafo LangGraph è unico e condiviso da tutti gli agenti (classe `AgentGraph`, sette nodi generici, cfr. § 4.2): non esiste un `build_graph()` né un insieme di nodi distinto per sottoclasse. Ciò che varia per agente sono solo il `Loader` e il `Profile` iniettati (le porte descritte in § 4.2). Il `PromptRepository` centralizza le funzioni di gestione dei prompt (`load_prompt_template`, `render_prompt`, `extract_json`), rendendo misurabile l'isolamento dei prompt (RQ.4); `GitHubToolset` espone i tool di lettura.

9.1 Agente Docs — documentazione inline

L'adattatore di contesto legge il codice dal repository **tramite i tool della facade** (`read_tree`, `read_file`), limitandosi all'ambito selezionato e ai soli linguaggi supportati. Prima di chiamare il modello c'è una piccola preparazione: attraversare quei file e individuare le unità che meritano documentazione (funzioni, classi, metodi) che ne sono prive o il cui commento è disallineato. Per un PoC non serve un analizzatore sintattico completo: basta riconoscere le definizioni senza un blocco di documentazione sopra. Il profilo carica il proprio template e vi inietta il codice di quelle unità, chiedendo al modello il commento nella convenzione corretta (JSDoc per TS/JS, docstring per Python). Il parser verifica che ogni commento sia ben formato e lo riaggancia a file e riga; il report è una `Proposal` di diff, mostrata nel frontend con una vista diff dedicata. Dove il codice è troppo intricato, l'agente emette un avviso strutturato invece di forzare un commento sbagliato.

Implemento: lettura del codice dall'ambito via facade, rilevamento delle unità non documentate, generazione del commento conforme, diff proposto, avviso per complessità eccessiva, visualizzazione del diff nell'interfaccia.

Non implemento: README di progetto e documentazione API (le altre due operazioni di Docs), e l'apertura effettiva della Pull Request.

9.2 Agente Security — scansione vulnerabilità OWASP

L'adattatore di contesto usa gli stessi tool di lettura; nella variante policy-as-code legge in più il file `POLICY.md` alla radice. Qui non c'è pre-analisi: il codice dell'ambito, spezzato in porzioni che stanno nella finestra di contesto, va direttamente al modello. Il profilo carica il template di scansione, che istruisce il modello a cercare i pattern dell'OWASP Top 10 (SQL injection, cross-site scripting, segreti in chiaro, crittografia insicura, controllo accessi incompleto) e a restituire per ogni riscontro categoria, gravità, file e righe, spiegazione e rimedio. Il parser attende una risposta strutturata, la valida, normalizza la gravità sui livelli previsti e controlla i riferimenti alle righe; se la risposta è malformata scatta il ramo di parsing, se il modello è troppo lento quello di timeout, fissato per questo agente a 180 secondi (più ampio del default degli altri due, perché la scansione richiede più cicli di lettura file). Il report è una lista di `FindingBlock`, ciascuno con il suo rimedio, mostrata nel frontend come elenco filtrabile per gravità.

Scelta di `maxToolRounds` = 8. Una scansione OWASP realistica non richiede di aprire l'intero repository: il modello esplora l'albero, individua i file ad alto rischio (entrypoint, gestione autenticazione, validazione degli input, configurazioni) e vi accede selettivamente. In pratica questo equivale a **4 coppie `tool_call/tool_result`** — abbondantemente sufficienti per i pattern dell'OWASP Top 10 sui linguaggi supportati (TypeScript, JavaScript, Python). Il valore **8** bilancia tre vincoli in tensione:

- *Copertura*: 8 round consentono di ispezionare tra 4 e 6 file significativi con margine per visitarne uno; scendere sotto 4 rischierebbe di non coprire percorsi di esecuzione critici.
- *Budget di token*: `Qwen3-Coder-30B-A3B-Instruct` ha una finestra nativa di ~32 K token. Ogni coppia `tool_call/tool_result` produce in media 1.500–2.000 token di overhead (richiesta strutturata + risposta dell'agente); 8 round aggiungono quindi ~12–16 K token alla cronologia, compatibili con il `max_scope_chars` di 60.000 caratteri fissato in § 4.3 e con il margine riservato al system prompt e alla risposta finale.
- *Timeout*: 8 round completano ampiamente entro i 180 secondi del timeout interno, anche tenendo conto della latenza di rete verso la facade GitHub e dei tempi di inferenza del modello.

Il valore è esposto come variabile d'ambiente (`MAX_TOOL_ROUNDS=8`) e va trattato come punto di partenza: se la validazione sul golden set del Piano di Qualifica mostrasse che la copertura è insufficiente, basta alzare l'env var senza rilascio del codice.

Implemento: estrazione del codice via facade, scansione OWASP Top 10, output strutturato (categoria/gravità/posizione/rimedio), gestione di timeout e parsing, elenco dei finding nell'interfaccia. In opzione, la variante policy-as-code su `POLICY.md`, che riusa identico il grafo cambiando solo il template.

Non implemento: copertura esaustiva dell'intera OWASP Top 10 e applicazione automatica dei rimedi — l'agente segnala, è il Security Auditor a validare.

9.3 Agente Changelog — changelog tecnico

Rispetto alle stesure precedenti, l'adattatore di contesto **non** legge più una fixture JSON: interroga **GitHub Issues dal vivo** tramite il tool `read_issues`, recuperando le storie di uno Sprint (titolo, descrizione, criteri di accettazione, stato, etichette, milestone). Il lavoro prima del modello è interno: filtrare le issue effettivamente chiuse per lo Sprint indicato e poi passarle al controllo di qualità dei metadati, che mette da parte quelle con descrizione vuota o troppo povera. Si segue il percorso “procedi escludendo”: si registra quali task sono state scartate e si prosegue solo con quelle valide, così il report è onesto su cosa copre. Il profilo carica il template del changelog tecnico e chiede al modello un resoconto per sviluppatori in Markdown; siccome

l'output è prosa, la validazione è più lieve, e il testo viene arricchito con i collegamenti ai ticket. Il report è composto da un unico `TextBlock` in Markdown, generato dal modello, che contiene già tutte le voci incluse con i link ai ticket; gli eventuali `ChangelogItemBlock` che seguono non sono altre voci del changelog, ma l'elenco — per trasparenza — delle issue *escluse* dal cancello di qualità sui metadati. Il tutto è mostrato nel frontend come Markdown renderizzato, con la lista delle esclusioni in coda.

Implemento: lettura delle issue da GitHub dal vivo, filtro delle chiuse, cancello di qualità sui metadati (scarto delle povere), sintesi tecnica in Markdown, arricchimento con i link ai ticket, rendering Markdown nell'interfaccia.

Non implemento: il changelog di business (sarebbe un nodo in più che rigira il testo tecnico in un secondo prompt).

9.4 Il filo comune

In tutti e tre i casi l'unica cosa che scrivo davvero di nuovo, lato agente, è “da dove leggo” (quale tool della facade) e “cosa chiedo e come rileggo la risposta”. Orchestrazione dei nodi, chiamata al modello con timeout, forma del report e gestione degli errori arrivano dallo scheletro condiviso. A questo si aggiunge il filo comune lato stack applicativo: gli endpoint REST, la persistenza su MongoDB e i componenti dell'interfaccia che mostrano il **Report** sono scritti una sola volta e riutilizzati identici, perché tutti parlano lo stesso contratto.

Nota implementativa — 7 classi `AgentProfile`, una per `OperationCode`. Il listing di composizione degli adapter mostra tre righe (Docs, Security, Changelog), ma in implementazione il numero di classi `AgentProfile` è **sette** — una per ciascun valore di `OperationCode`:

```
DocsReadmeProfile · DocsInlineProfile · DocsApiProfile · OwaspScanProfile ·  
SecurityPolicyProfile · ChangelogTechnicalProfile · ChangelogBusinessProfile
```

La scelta di 7 classi distinte invece di 3 profili parametrizzati è motivata dalla struttura stessa dell'interfaccia `AgentProfile`: i metodi `build_prompt()` e `parse_output()` hanno implementazioni *genuinamente diverse* per ciascuna operazione — il prompt per generare un README differisce strutturalmente da quello per documentare API REST; il parser di `FindingBlock` OWASP differisce da quello di `PolicyViolationBlock`. Aggregare operazioni diverse in un solo profilo parametrizzato produrrebbe catene `if/elif` su `operation` dentro ogni metodo, violando il principio di singola responsabilità e rendendo i test per operazione più fragili. Con 7 classi invece:

- Il dispatch `OperationCode` → `AgentProfile` è una mappa statica in `AgentRegistry`, con zero logica condizionale nei profili stessi.
- Ogni profilo si testa in isolamento iniettando un provider LLM simulato, il che è essenziale per validare la copertura del golden set (PdQ).
- Aggiungere una nuova operazione significa aggiungere un file, non modificare classi esistenti (Principio Open/Closed).
- Il collegamento profilo → template di prompt nel `PromptRepository` è verificabile a colpo d'occhio: `docs/readme.txt` → `DocsReadmeProfile`, `security/owasp.txt` → `OwaspScanProfile`, ecc.

I tre “agenti” del listing restano l'unità concettuale corretta (Docs, Security, Changelog condividono rispettivamente `Loader` e comportamento `usesTools`); i 7 profili sono la granularità implementativa, invisibile al grafo `LangGraph` condiviso che li riceve già istanziati.

10 Decisioni architetturali (ADR)

Queste decisioni sono vincolanti per l'implementazione: sono i punti dove la strada più comoda viola un requisito. Gli sviluppatori le seguano anche quando una scorciatoia sembra allettante.

ADR-1 — Il token GitHub non lascia NestJS. Gli agenti devono dialogare con GitHub, ma RS.3 vieta loro la scrittura e RS.2 impone token cifrati. **Decisione:** facade read-only su NestJS, esposta agli agenti come tool LangGraph. Il token resta cifrato nel backend; l'agente decide cosa leggere e iterare, ma ogni chiamata passa dalla whitelist. **Da non fare:** passare il token al servizio Python perché "è più semplice". Sposterebbe il segreto oltre un secondo confine e renderebbe RS.3 da verificare in due codebase.

ADR-2 — L'LLM dietro una porta, fornitore della famiglia di Bedrock. Credenziali AWS assenti. **Decisione:** LLMProvider con implementazione attiva verso API gestita della stessa famiglia di modelli che useremo su Bedrock, e BedrockProvider scritto ma non attivo. **Da non fare:** eseguire un modello locale, che non direbbe nulla di utile sul prodotto e costringerebbe a ritardare i prompt.

ADR-3 — Coda per l'esecuzione, non chiamata sincrona. RQ.6 fissa a ≤ 5 minuti il tempo di risposta per l'esecuzione di un singolo agente, e i fallimenti vanno isolati (RF.48). Il timeout interno dell'agente è configurato a **90 secondi** per Docs e Changelog e a **180 secondi** per Security (che itera più cicli di lettura file via tool-calling): entrambi i valori restano ampiamente entro il limite di RQ.6; l'ottimizzazione di prompt e numero di letture nelle fasi successive resta utile per contenere costi e tempo percepito, non per rientrare in un vincolo già rispettato.

Il timeout HTTP del gateway (AgentGatewayService) è configurato **per operation code** per garantire che non si chiuda prima del timeout interno dell'agente: **90 s** per le operazioni Docs e Changelog, **200 s** per le operazioni Security (10 s di margine rispetto al timeout interno di 180 s). Il valore è letto da variabile d'ambiente (AGENT_GATEWAY_TIMEOUT_DOCS_S / AGENT_GATEWAY_TIMEOUT_SECURITY_S), così cambia la configurazione, non il codice. **Decisione:** POST /tasks accoda e ritorna subito; un worker esegue; gli aggiornamenti arrivano via WebSocket. **Da non fare:** eseguire l'agente dentro il ciclo della richiesta HTTP, che accoppierebbe l'interfaccia ai tempi del modello.

ADR-4 — Gli artefatti degli agenti sono proposte, non modifiche. **Decisione PoC:** l'output è un Report immutabile scaricabile, mai un commit. Il token richiesto ha scope di sola lettura. **Aggiornamento MVP:** l'apertura della Pull Request è delegata al *backend* NestJS (ADR-AWS-10, § 39), non all'agente — che rimane read-only. L'umano che esercita la supervisione richiesta da ADR-4 è l'utente che clicca *Applica* nella UI del report, avviando la chiamata verso il backend che apre la PR; il *merge* resta poi un atto distinto, condotto dall'utente direttamente su GitHub. Non esiste un endpoint /apply separato: il backend apre la PR immediatamente al completamento del task, e il link alla PR viene reso disponibile nella pagina del report (ADR-FE-1). Il token GitHub nell'MVP richiede scope di scrittura (**repo**), ma resta esclusivo del backend.

Queste quattro ADR sono le uniche vincolanti per il PoC e per l'orchestrazione minima (§ 8). Quattro decisioni ulteriori (ADR-5–ADR-8), relative al componente di *orchestratore di produzione* proposto per una fase successiva al MVP, sono presentate in § 12.5, subito dopo la descrizione del componente a cui si riferiscono (BatchStep, RetryPolicyEngine, CircuitBreaker Service, AgentPoolManager), invece che qui, per non costringere chi legge a memorizzare nomi di classi non ancora introdotte.

11 Come validiamo il PoC

Il PoC si aggancia direttamente alle metriche del Piano di Qualifica.

L'**accuratezza** (obiettivo $> 85\%$) si misura preparando un piccolo golden set di file con vulnerabilità note e confrontando l'output dell'agente Security. Con il provider via API gestita, questa metrica va verificata presto sul fornitore effettivamente scelto.

Il **tempo di risposta** si misura per singola esecuzione, dall'invocazione dell'endpoint REST alla disponibilità del report, e include l'overhead di persistenza su MongoDB e delle letture GitHub (mitigate dalla cache). *Nota di coerenza (verificata sull'Analisi dei Requisiti v1.0.0):* RQ.6 fissa il vincolo a ≤ 5 minuti (Obbligatorio, fonte UC27.2/PdQ MPD 12), come già riportato nella Tabella 38. I timeout attualmente configurati nel PoC (90 s per Docs e Changelog, 180 s per Security) restano quindi ampiamente *sotto* quella soglia vincolante, non sopra: il target interno più stringente che il team si è dato in fase di ottimizzazione va visto come un margine di sicurezza aggiuntivo da mantenere stretto per l'esperienza utente (§ 8, RQ.10), non come un vincolo di RQ.6 ancora da rispettare.

L'**isolamento dei prompt** è verificabile per costruzione, perché i prompt sono file esterni. Conviene aggiungere un test statico che fallisce se un modulo agente contiene literal di stringa oltre una soglia: rende la metrica MPD_14 una prova automatica.

La **copertura dei test** è raggiungibile perché sia LLMPProvider sia la facade GitHub sono dietro interfaccia: nei test si iniettano un provider simulato e un client GitHub finto con risposte registrate, così i test girano offline, deterministici e a costo zero.

Si aggiunge la verifica dell'**integrazione end-to-end**: un test che, partendo da una chiamata simulata dal frontend, verifica che il backend instradi correttamente l'agente, che l'agente legga via facade (con client GitHub finto), che il report sia persistito su MongoDB e restituito nel formato atteso.

Uso dei dati per l'addestramento (RS.5). Il vincolo RS.5 impone di garantire che il codice sorgente inviato al modello **non venga trattenuto né usato per addestrare modelli di terzi**. Questo punto merita attenzione perché, con la scelta di un'API gestita, la garanzia dipende dal fornitore e dal canale, e i due canali che il progetto usa in momenti diversi **non offrono la stessa copertura in modo automatico**.

Quando il modello sarà servito da **AWS Bedrock**, in produzione, la copertura è netta: AWS dichiara che i dati dei clienti non vengono mai usati per addestrare i modelli sottostanti, e questo vale anche per i modelli open-weight di terzi serviti tramite Bedrock, perché l'inferenza avviene nel perimetro AWS. In quella configurazione RS.5 è soddisfatto dai termini del servizio.

Durante il **PoC**, però, il modello non passa da Bedrock ma dall'**API diretta del fornitore** (nel nostro esempio, DashScope di Alibaba Cloud): la garanzia di Bedrock non si estende a questo canale, che è retto dai propri termini di servizio. La documentazione dei fornitori è esplicita nel rimandare le politiche sui dati al provider effettivamente usato. Ne consegue un adempimento preciso, non tecnico ma con scadenza esterna, da svolgere **prima** di inviare al modello il primo file di un repository privato del proponente:

1. verificare, nei termini del piano API scelto (per Qwen: il piano DashScope internazionale, regione di Singapore/Model Studio), la clausola specifica sulla ritenzione dei dati e sul loro eventuale uso per addestramento, controllando se esista un'opzione o un tier che escluda esplicitamente tale uso;
2. mettere a verbale l'esito della verifica, con data e versione dei termini consultati, perché è la forma in cui RS.5 è dimostrabile in sede di collaudo;
3. tracciare il requisito nel Piano di Qualifica come **controllo documentale**, non come test automatico: nessun test può dimostrare che un terzo non usi i dati, solo i termini contrattuali possono.

Se la verifica sui termini dell'API diretta non desse garanzie sufficienti, l'alternativa a costo maggiore è anticipare l'uso di un canale con garanzie enterprise sui dati (Bedrock stesso, se e quando le credenziali arrivano, o un intermediario equivalente), accettando però che questo tolga al PoC la sua indipendenza dalle credenziali AWS. È una decisione da prendere consapevolmente, non un dettaglio da rimandare.

12 Estensioni di scalabilità dell'orchestratore (roadmap oltre l'MVP)

Precisazione terminologica. L'*Orchestratore* richiesto da RV.1 — lo smistamento deterministico delle operazioni verso gli agenti, senza LLM nella logica di instradamento — **è interamente implementato ed è parte del perimetro valutato per l'MVP**, non di questa roadmap. Nel PoC, `AgentRegistry` risolve un `OperationCode` nel proprio agente con una mappa statica e `TaskProcessor` esegue le Task isolandone i fallimenti (RF.48); nell'MVP questa stessa coppia di componenti resta l'unico orchestratore in esecuzione ed è completata dalla dashboard di monitoraggio del Frontend MVP (RF.43–RF.47, Parte II), che ne rende visibile in tempo reale lo stato delle Task. È questo insieme — instradamento deterministico (§ 8) più dashboard (Parte II) — a soddisfare RV.1 e RF.40–RF.48, ed è questo insieme a essere oggetto di valutazione per l'MVP.

Quello che **questa sezione** progetta non è quindi “l'orchestratore”, ma un insieme di *estensioni di scalabilità e resilienza* che l'orchestratore minimo non regge in un prodotto a pieno carico: incatenare più operazioni con dipendenze, ritentare automaticamente un fallimento transitorio, distribuire il carico su più istanze del servizio agenti, dare a chi supervisiona un quadro d'insieme operativo dedicato (batch, code, pool, circuit breaker, distinto dalla dashboard utente della Parte II). Il disegno mantiene due vincoli ereditati dal PoC/MVP: l'instradamento resta deterministico (RV.1) e il contratto `Report/Task` non cambia forma, così il frontend esistente continua a funzionare senza riscrittura.

Nota di perimetro. Quanto segue è un disegno architetturale, non una specifica implementata: i suoi requisiti (§ 12.2) non sono ancora nel registro ufficiale e restano al di fuori del perimetro dell'MVP descritto nella Parte II (Frontend) e nella Parte IV (Infrastruttura AWS): nessuna delle due usa `BatchStep`, `AgentPoolManager` o i circuit breaker qui definiti — l'MVP resta sull'orchestrazione minima di § 8 (`AgentRegistry/TaskProcessor`, `Desired Count: 1` su ECS Fargate, nessun pool di istanze), che è a tutti gli effetti *l'Orchestratore dell'MVP*, non un suo sostituto provvisorio. Il componente qui descritto è collocato a chiusura della Parte I perché estende direttamente l'orchestrazione minima e le ADR-1–ADR-4 del PoC/MVP; va letto come roadmap per l'evoluzione oltre l'MVP, da formalizzare con il proponente prima di essere pianificato in una release.

12.1 Obiettivo e perimetro

L'orchestratore di produzione sostituisce la coppia `AgentRegistry/TaskProcessor` con un insieme di componenti cooperanti che rispondono a quattro esigenze lasciate fuori dal PoC:

- **Batch complessi:** un batch non è più un insieme di operazioni indipendenti avviate insieme, ma un grafo diretto aciclico (DAG) di passi con dipendenze — ad esempio, “esegui Security, e solo se non emergono finding critical esegui Docs sugli stessi file”.
- **Resilienza:** un fallimento transitorio (timeout di rete verso GitHub, errore 5xx del provider LLM) non deve richiedere un nuovo avvio manuale, ma un retry automatico e limitato.
- **Scaling:** il servizio agenti Python deve poter girare in più istanze, con il carico distribuito tra loro e senza che una singola istanza sovraccarica diventi un collo di bottiglia.

- **Osservabilità:** chi gestisce il sistema deve vedere, in tempo reale, quanti batch sono in corso, dove si sono fermati, quali agenti sono in salute e quali soglie di errore si stanno avvicinando.

Restano fermi, ereditati dal PoC: l'invarianza del contratto **Report** (§ 5), la separazione tra API pubbliche e interne (§ 7), il vincolo che il token GitHub non lasci il backend (ADR-1) e l'astrazione **LLMProvider** (ADR-2). L'orchestratore non introduce un nuovo linguaggio né un nuovo datastore: resta su NestJS, MongoDB, Redis e BullMQ, coerentemente col principio di minimizzare le variabili tecnologiche nuove (§ 3).

12.2 Requisiti

I requisiti che seguono **non sono ancora presenti nel registro dei requisiti** (RF.1–RF.105, RQ.1–RQ.11, RS.1–RS.5, RV.1–RV.21): descrivono capacità di produzione (batch come DAG, retry, circuit breaker, scaling, dashboard estesa) che il registro attuale non copre, perché il registro si ferma al perimetro del MVP e questa sezione guarda oltre. I codici proseguono la numerazione esistente per evitare collisioni (RF.106+, RQ.12+, RS.6) e vanno intesi come una **proposta** da sottoporre al proponente e formalizzare nel registro prima di essere trattati come requisiti vincolanti.

Tabella 5: Requisiti aggiuntivi dell’orchestratore di produzione.

Codice	Descrizione
RF.106	Un Batch è definito come DAG di passi (BatchStep), ciascuno con zero o più dipendenze verso altri passi dello stesso batch.
RF.107	Una Task fallita per errore classificato come transitorio (RS.6, sotto) viene ritentata automaticamente secondo una RetryPolicy configurabile per operazione.
RF.108	Il carico verso il servizio agenti è distribuito su più istanze registrate in un pool, senza affinità tra Task e istanza.
RF.109	Una dashboard mostra, in tempo reale, lo stato dei batch attivi, la profondità delle code, la salute del pool di agenti e lo stato dei circuit breaker.
RF.110	Ogni batch ha una priorità di coda configurabile; a parità di altre condizioni, i batch a priorità più alta sono schedulati per primi.
RF.111	Il fallimento di un passo del DAG non blocca i rami del batch che non ne dipendono; blocca (o salta, secondo la policy) solo i suoi discendenti.
RF.112	Le chiamate verso il provider LLM e verso l’API GitHub sono protette da un circuit breaker dedicato per bersaglio, che apre il circuito dopo una soglia di fallimenti consecutivi.
RQ.12	L’overhead di scheduling introdotto dall’orchestratore (dal momento in cui una Task è pronta al momento in cui viene inviata a un’istanza agente) resta sotto i 200 ms in condizioni nominali.
RQ.13	Il sistema sostiene almeno N Task concorrenti in esecuzione (valore di capacity planning da definire in fase di dimensionamento, non fissato in questo documento).
RQ.14	Gli eventi mostrati in dashboard riflettono lo stato reale con una latenza inferiore a 1 s dal loro verificarsi.
RS.6	Un errore è classificato <i>transitorio</i> (rete, timeout, 5xx, rate limit) o <i>permanente</i> (401/403, 404, errore di parsing ripetuto); solo il primo tipo attiva il retry automatico (RF.107).
RV.1	(<i>ereditato dal PoC, invariato</i>) L’instradamento operazione→agente resta deterministico: nessun componente dell’orchestratore usa un LLM per decidere cosa eseguire o in che ordine.

12.3 Architettura dei componenti

La figura 11 mostra i componenti nuovi (in blu) accanto a quelli del PoC che restano, riorganizzati (in verde). Il principio architetturale è lo stesso della facade GitHub (ADR-1): ogni responsabilità nuova è un servizio con un’interfaccia stretta, così da poter essere testato e sostituito in isolamento.

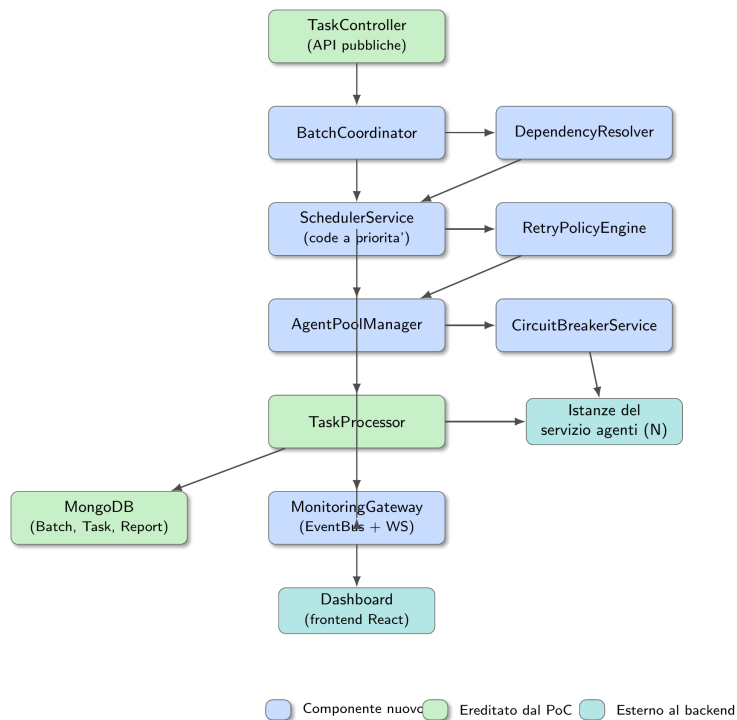


Figura 11: Architettura dei componenti dell'orchestratore di produzione. In blu i componenti nuovi; in verde quelli ereditati dal PoC (adattati); in azzurro gli elementi esterni al backend.

Il flusso   lineare: **BatchCoordinator** riceve la definizione del batch dal frontend e la traduce in **BatchStep** persistenti; **DependencyResolver** calcola, a ogni transizione di stato, quali passi sono *pronti* (tutte le dipendenze soddisfatte); **SchedulerService** li accoda rispettando le priorit ; **RetryPolicyEngine** decide, quando un passo fallisce, se e quando riprovarlo; **AgentPoolManager** sceglie un'istanza sana del servizio agenti e vi inoltra l'esecuzione tramite **TaskProcessor**, passando per **CircuitBreakerService** che pu  rifiutare la chiamata a priori se il bersaglio (LLM o GitHub)   in stato aperto. Ogni componente emette eventi verso **MonitoringGateway**, che li inoltra alla dashboard via WebSocket e ne persiste un riassunto per le metriche storiche.

12.4 Modello di dominio esteso

Il modello del PoC (figura 3) si estende con cinque entit  nuove, senza modificare **Report** n  la forma esterna di **Task** (si aggiungono solo campi opzionali retro-compatibili).

```

// Un batch e' ora un grafo, non piu' un insieme piatto di operazioni
interface Batch {
  id: string
  userId: string
  contextId: string
  status: 'DRAFT' | 'SCHEDULED' | 'RUNNING'
    | 'COMPLETED' | 'PARTIALLY_FAILED' | 'FAILED' | 'CANCELLED'
  priority: number // usata da SchedulerService, default 0
  createdAt: Date
  finishedAt?: Date
}

// Un nodo del DAG: una singola operazione dentro un batch
interface BatchStep {
  id: string

```

```

    batchId: string
    operation: OperationCode
    dependsOn: string[] // id di altri BatchStep dello stesso
                        batch
    status: 'PENDING' | 'READY' | 'RUNNING'
           | 'COMPLETED' | 'FAILED' | 'SKIPPED' | 'CANCELLED'
    taskId?: string // valorizzato quando lo step viene
                   eseguito
    onFailure: 'BLOCK_DESCENDANTS' | 'SKIP_DESCENDANTS' // RF.111
}

// Estensione non distruttiva del Task esistente (Sez. 4.3, modello di
// dominio)
interface Task {
    // ... campi invariati (id, userId, batchStepId, contextId, operation,
    // status, progressPercent, currentStage, reportId, error,
    // finishedAt)
    batchStepId?: string // nullo per le task fuori da un batch
                       DAG
    retryCount: number // default 0
    retryPolicyId?: string
}

interface RetryPolicy {
    id: string
    operation: OperationCode | '*' // '*' = policy di default
    maxAttempts: number // include il primo tentativo
    backoffStrategy: 'EXPONENTIAL' | 'FIXED'
    baseDelayMs: number
    maxDelayMs: number
    retryableErrorKinds: ('TIMEOUT' | 'UPSTREAM')[] // mai 'PARSING'
                    esaurito
}

interface AgentInstance {
    id: string
    baseUrl: string // es. http://agents-2:8000
    status: 'HEALTHY' | 'DEGRADED' | 'DOWN'
    activeTasks: number
    lastHeartbeatAt: Date
}

interface CircuitBreakerState {
    target: 'LLM_PROVIDER' | 'GITHUB_API'
    state: 'CLOSED' | 'OPEN' | 'HALF_OPEN'
    failureCount: number
    openedAt?: Date
}

```

Listing 3: Entita' nuove del modello di dominio dell'orchestratore.

Il campo `onFailure` su `BatchStep` rende esplicita, per ogni passo, la scelta tra due semantiche legittime quando un suo predecessore fallisce: `BLOCK_DESCENDANTS` lascia i discendenti in `PENDING` indefinitamente (il batch resta `PARTIALLY_FAILED` finché un umano non decide), `SKIP_DESCENDANTS` li marca `SKIPPED` e lascia proseguire il resto del DAG. Il default proposto è `SKIP_DESCENDANTS`, coerente con l'isolamento dei fallimenti già adottato per i batch piatti del PoC (RF.48): un batch parzialmente riuscito resta più utile di un batch bloccato.

12.5 Decisioni architetturali dell'orchestratore di produzione (ADR-5 – ADR-8)

Queste quattro decisioni completano le ADR-1–ADR-4 del PoC (§ 10) e riguardano esclusivamente il componente appena descritto; non sono vincolanti per l'MVP (§ 12, Nota di perimetro), ma lo diventano nel momento in cui l'orchestratore di produzione viene pianificato.

ADR-5 — Il DAG è persistito come lista di adiacenza in MongoDB, non delegato a un motore esterno. Un motore di workflow dedicato (Temporal, Camunda) risolverebbe elegantemente retry, timeout e stato distribuito, ma introdurrebbe una dipendenza tecnologica nuova non presente nel capitolato e un secondo sistema di persistenza dello stato da tenere sincronizzato con MongoDB. **Decisione:** `BatchStep` come collezione MongoDB con riferimenti `dependsOn`, risolta lato applicativo da `DependencyResolver` a ogni transizione di stato. **Da non fare:** introdurre un motore di workflow esterno finché la complessità dei DAG reali (profondità, fan-out) non lo giustifica; è un'evoluzione possibile, non un prerequisito.

ADR-6 — Il retry è responsabilità del backend NestJS, non del servizio agenti Python. Il servizio agenti già implementa una propria autocorrezione a grana fine per gli errori di parsing (fino a due tentativi dentro il grafo `LangGraph`); estendere quella logica al retry di un intero fallimento `TIMEOUT/UPSTREAM` frammenterebbe la fonte di verità sullo stato della Task tra due servizi. **Decisione:** il servizio agenti resta stateless rispetto ai retry; ogni tentativo è un'invocazione nuova e indipendente decisa da `RetryPolicyEngine` nel backend, che è anche l'unico scrittore dello stato Task. **Da non fare:** far ritentare al servizio agenti le proprie chiamate al provider LLM oltre l'autocorrezione di parsing già presente, per non duplicare la policy in due posti.

ADR-7 — Il circuit breaker è per bersaglio condiviso, non per utente o per Task. LLM e GitHub sono risorse a quota globale: se il circuito restasse per singolo utente, un utente rumoroso potrebbe esaurire la quota mentre il sistema continua a segnalare tutti gli altri circuiti come chiusi, ritentando invano. **Decisione:** uno stato di circuito per `LLM_PROVIDER` e uno per `GITHUB_API`, condivisi da tutte le Task di tutti gli utenti. **Da non fare:** dimensionare le soglie di apertura sul traffico di un singolo utente; vanno tarate sul throughput aggregato atteso.

ADR-8 — Nessuna affinità tra Task e istanza agente. Legare una Task all'istanza che l'ha iniziata (per riprendere un tool-calling interrotto, ad esempio) semplificherebbe alcuni casi di ripresa, ma impedirebbe lo scaling orizzontale reale: un'istanza sovraccarica resterebbe tale per le sue Task già assegnate anche se altre istanze sono libere. **Decisione:** ogni tentativo (primo avvio o retry) può andare a qualunque istanza `HEALTHY`; lo stato che deve sopravvivere a un cambio di istanza (contesto caricato, cronologia dei messaggi) resta interno all'esecuzione del grafo `LangGraph` di un singolo tentativo, mai persistito lato Python tra un tentativo e l'altro. **Da non fare:** introdurre sticky session tra frontend e istanza agente; la sessione utile è quella verso il backend (WebSocket autenticato), non verso il servizio agenti.

12.6 Ciclo di vita del Batch

La figura 12 mostra gli stati di `Batch`. A differenza della Task (che ha un ciclo di vita lineare, figura 8), il Batch ha due esiti intermedi possibili (`COMPLETED` solo se *tutti* i passi sono completati, `PARTIALLY_FAILED` se almeno uno è fallito ma altri sono comunque riusciti) oltre al fallimento totale.

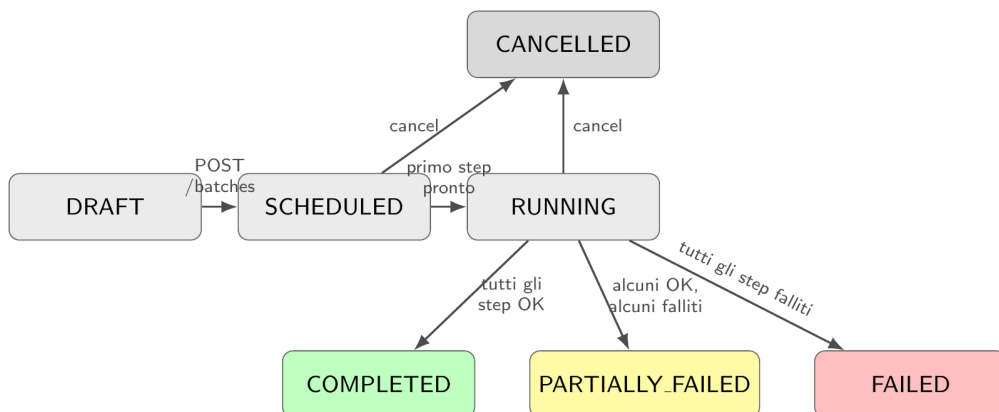


Figura 12: Ciclo di vita di un Batch (RF.106, RF.111). Gli stati terminali (verde/giallo/rosso/-grigio) non hanno uscite, come per Task (figura 8).

Un **BatchStep** attraversa uno stato aggiuntivo rispetto a **Task**: **READY**, che rappresenta l'istante in cui **DependencyResolver** ha verificato che tutte le sue dipendenze sono **COMPLETED** ma **SchedulerService** non lo ha ancora estratto dalla coda. La distinzione tra **READY** e **RUNNING** è ciò che rende misurabile RQ.12 (l'overhead di scheduling è il tempo trascorso in **READY**).

12.7 Retry e circuit breaker

RetryPolicyEngine interviene in un solo punto: quando **TaskProcessor** riceve un **Report** con **status=FAILED**. Non tutti i fallimenti sono uguali (RS.6): un errore **PARSING** persistente (il modello non produce mai un JSON valido) non migliora ripetendo la stessa chiamata, mentre un **TIMEOUT** o un errore **UPSTREAM** (5xx dal provider, connessione GitHub caduta) spesso sì. Per questo la policy limita il retry automatico ai soli **error.kind** elencati in **retryableErrorKinds** — di default **TIMEOUT** e **UPSTREAM**, mai **PARSING** (che nel servizio agenti ha già un suo meccanismo di autocorrezione a grana più fine, interno al grafo **LangGraph**, prima ancora di arrivare come **Report** fallito).

Il ritardo tra un tentativo e il successivo segue un backoff esponenziale con *jitter*, per evitare che più Task fallite nello stesso istante ripartano tutte insieme e ricreino il picco di carico che le aveva fatte fallire:

$$\text{delay}_n = \min(\text{maxDelayMs}, \text{baseDelayMs} \times 2^n) + \text{jitter}(0, \text{baseDelayMs})$$

dove n è il numero del tentativo (0-indicizzato). Ogni retry crea una *nuova* esecuzione della stessa Task (stesso **id**, incrementando **retryCount**), non una nuova Task: questo mantiene coerente la relazione uno-a-uno tra Task e la sua storia nel **BatchStep**, mentre il **Report** fallito dei tentativi precedenti resta comunque persistito per motivi di audit (l'**AccessLog** verso GitHub, § 6, resta corretto anche quando un tentativo fallito ha già letto dei file).

Il circuit breaker è per bersaglio (**LLM_PROVIDER** o **GITHUB_API**), non per singola Task: entrambi sono risorse condivise — una quota API, un rate limit GitHub — e un errore a cascata su una li riguarda indipendentemente da quale Task lo ha causato. Segue il pattern standard a tre stati: **CLOSED** (funzionamento normale) → **OPEN** dopo k fallimenti consecutivi entro una finestra temporale (rifiuta le nuove chiamate immediatamente, senza nemmeno tentarle, e le mette in coda per un nuovo giro di retry) → **HALF_OPEN** dopo un periodo di raffreddamento (lascia passare una singola chiamata di prova) → **CLOSED** se ha successo o di nuovo **OPEN** se fallisce. Aprire il circuito verso GitHub non blocca le Task che non hanno tool GitHub attivi (Docs e Changelog, se il loro **ContextLoader** ha già completato la lettura); aprire il circuito verso il provider LLM blocca invece qualunque Task, perché tutti e tre gli agenti dipendono da **LLMProvider**.

12.8 Scaling: il pool di agenti

Nel PoC il servizio agenti è una singola istanza raggiunta a un indirizzo fisso (`http://agents:8000`, § 4.2). In produzione questo non basta: se un'istanza è occupata su una scansione Security da 180s, le Task in coda aspettano anche se altre istanze sono libere. **AgentPoolManager** sostituisce l'indirizzo fisso con un registro dinamico di istanze, popolato dalla piattaforma di orchestrazione dei container (repliche Docker Compose in sviluppo, repliche ECS/Fargate in produzione, coerentemente con il target AWS già fissato in § 3) tramite service discovery, e applica tre meccanismi:

- **Health check:** ogni istanza espone `GET /health`; **AgentPoolManager** la interroga ogni 10s con timeout 3s. Tre fallimenti consecutivi portano lo stato a `DOWN` e la escludono dal bilanciamento finché non torna a rispondere.
- **Bilanciamento a connessioni attive:** tra le istanze `HEALTHY`, la Task pronta va a quella con `activeTasks` più basso (least-connections), non a rotazione semplice: gli agenti hanno tempi di esecuzione molto diversi tra loro (90s Docs/Changelog contro 180s Security), e una rotazione cieca finirebbe per accumulare lavoro sulle istanze più lente.
- **Nessuna affinità:** ogni Task può essere eseguita da qualunque istanza (ADR-8, sotto), perché lo stato che conta — Task, Report, AccessLog — vive nel backend, non nel processo Python. Questo è ciò che rende lo scaling orizzontale possibile senza cambiare il servizio agenti: un'istanza in più è solo un altro consumatore identico della stessa coda.

Quando tutte le istanze sono sature o `DOWN`, le Task pronte restano in coda su BullMQ (nessuna Task viene persa o eseguita in autonomia dal backend): è la stessa garanzia di back-pressure già presente nel PoC (ADR-3), estesa a più consumatori.

12.9 Dashboard di monitoraggio

La dashboard (RF.109) è una nuova pagina del frontend React esistente, non un servizio separato: riuserà la stessa connessione WebSocket già autenticata (§ 7.3), sottoscrivendo in più i canali emessi da **MonitoringGateway**. Mostra quattro riquadri:

1. **Batch attivi:** lista dei batch `RUNNING` con il loro DAG visualizzato come grafo (nodo verde = `COMPLETED`, blu = `RUNNING`, grigio = `PENDING/READY`, rosso = `FAILED`, tratteggiato = `SKIPPED`).
2. **Code:** profondità della coda per priorità, età della Task più vecchia in attesa (indicatore diretto di RQ.12 in degrado).
3. **Pool di agenti:** stato di ogni **AgentInstance** (salute, task attive, ultimo heartbeat).
4. **Circuit breaker:** stato corrente per `LLM.PROVIDER` e `GITHUB_API`, con il conteggio dei fallimenti nella finestra corrente.

Tabella 6: Nuovi eventi WebSocket emessi da **MonitoringGateway**, in aggiunta ai quattro già presenti nel PoC (tabella 4).

Evento WS	Payload
<code>batch.step.updated</code>	<code>{ batchId, stepId, status, taskId? }</code>
<code>batch.updated</code>	<code>{ batchId, status }</code>
<code>agent.health.changed</code>	<code>{ instanceId, status, activeTasks }</code>
<code>circuitbreaker.changed</code>	<code>{ target, state, failureCount }</code>
<code>queue.depth.updated</code>	<code>{ priority, depth, oldestWaitMs }</code>

12.10 Contratto API esteso

Gli endpoint REST pubblici del PoC (§ 7) restano validi; l’orchestratore ne aggiunge di nuovi, mantenendo lo stesso prefisso `/api/v1` e la stessa autenticazione JWT.

Tabella 7: Endpoint REST pubblici aggiunti dall’orchestratore.

Metodo e path	Scopo
POST <code>/batches</code>	Crea un Batch a partire da una definizione di DAG (<code>BatchStep[]</code> con le rispettive <code>dependsOn</code>); ritorna <code>202 + {batchId}</code> .
GET <code>/batches</code>	Elenco dei batch dell’utente, con stato aggregato.
GET <code>/batches/:id</code>	Dettaglio di un batch, incluso lo stato di ogni <code>BatchStep</code> .
POST <code>/batches/:id/cancel</code>	Annulla un batch (cancella gli step <code>PENDING/READY</code> , richiede annullamento cooperativo per quelli <code>RUNNING</code> , come già fatto per la singola Task).
GET <code>/metrics/summary</code>	Metriche aggregate: throughput, tasso di successo per operazione, tempo medio di esecuzione. Uso interno/dashboard.
GET <code>/agents/pool/status</code>	Stato corrente del pool di istanze agenti (sola lettura, per la dashboard).
GET <code>/circuit-breakers</code>	Stato corrente dei circuit breaker per bersaglio (sola lettura, per la dashboard).

Gli endpoint interni si estendono di uno solo: `POST /internal/agents/heartbeat`, che ogni istanza del servizio agenti chiama periodicamente per registrarsi nel pool — alternativa più semplice alla service discovery quando l’orchestrazione dei container non la offre nativamente (ad esempio in Docker Compose puro).

12.11 Migrazione dal PoC

La migrazione è pensata per essere incrementale e non richiedere una riscrittura: ogni passo aggiunge un componente sopra quelli esistenti, mantenendo il sistema funzionante a ogni tappa.

1. **DAG piatto retro-compatibile:** si introduce `BatchCoordinator` e `DependencyResolver`, ma ogni batch creato dal frontend attuale (un insieme di operazioni senza dipendenze, come oggi) genera un DAG con zero archi — ogni `BatchStep` è già `READY` dal primo istante. Il comportamento osservabile non cambia finché il frontend non inizia a dichiarare dipendenze esplicite.

2. **Retry sopra il TaskProcessor esistente:** si inserisce `RetryPolicyEngine` nel punto in cui `TaskProcessor` già gestisce un `Report` con `status=FAILED` (§ 8): invece di segnare la Task immediatamente come `FAILED`, si consulta la policy e si ri-accoda se applicabile. Nessuna modifica al servizio agenti.
3. **Pool dinamico:** si sostituisce l'URL statico verso il servizio agenti con `AgentPoolManager`, inizialmente con una sola istanza registrata (comportamento identico al PoC), poi con repliche multiple quando il capacity planning (RQ.13) lo richiede.
4. **Circuit breaker e dashboard:** ultimo livello, perché richiede che i primi tre siano stabili — non ha senso mostrare in dashboard uno stato di retry o di pool che non esiste ancora.

Nessuno di questi passi tocca il contratto `Report` né gli endpoint pubblici già consumati dal frontend (§ 7): l'orchestratore è un'evoluzione del *come* le Task vengono eseguite, non del *cosa* il frontend riceve.

12.12 Pattern architetturali aggiuntivi raccomandati

I pattern già adottati nel PoC e nell'orchestratore di produzione (facade in § 6, strategy/adapter su `LLMProvider` e sui `ContextLoader` per agente, coda con backpressure in ADR-3, circuit breaker per bersaglio in ADR-7, retry con backoff esponenziale e jitter in § 12.5, artefatto immutabile in ADR-4) coprono bene la resilienza dell'esecuzione. Restano alcuni punti ciechi, individuati rileggendo il documento nel suo complesso, che non richiedono nuovi servizi AWS né nuove tecnologie: sono estensioni dei componenti già disegnati. Vengono proposti qui come parte della stessa roadmap post-MVP di § 12, non come requisiti dell'MVP corrente.

Tabella 8: Pattern architetturali aggiuntivi proposti, con il problema che chiudono e dove si innestano nel disegno esistente.

Pattern	Problema che chiude	Dove si innesta
Idempotency Key	La Parte Frontend segnala come rischio accettato l'assenza di retry automatici client-side (Tabella 18): un doppio invio manuale dopo un timeout di rete può creare due batch identici. Un header <code>Idempotency-Key</code> generato dal client, con indice univoco lato <code>Batch/Task</code> per una finestra di deduplica, chiude il buco senza introdurre retry automatici.	<code>POST /tasks</code> (§ 7) e <code>POST /batches</code> (Tabella 7).
Outbox Pattern	Lo stato <code>Task</code> è scritto su MongoDB e l'evento è emesso via <code>Websocket/BullMQ</code> come due operazioni distinte: se il processo cade tra le due, l'evento si perde e il polling di riallineamento (§ 7.3) resta l'unica rete di sicurezza. Un outbox transazionale (l'evento scritto nella stessa transazione Mongo dello stato, drenato da un publisher separato) elimina la classe di bug "evento perso silenziosamente".	<code>TaskProcessor</code> e <code>MonitoringGateway</code> (§ 12, Architettura dei componenti).

(continua)

Tabella 8 (segue dalla pagina precedente)

Pattern	Problema che chiude	Dove si innesta
Bulkhead	AgentPoolManager bilancia a connessioni attive su un pool omogeneo, ma Security (timeout 180s) e Docs/Changelog (90s) hanno profili di esecuzione molto diversi: una raffica di scansioni OWASP può occupare tutte le istanze anche con bilanciamento a least-connections. Isolare code/capacità per famiglia di operazione evita che un agente ne affami un altro.	Scaling: il pool di agenti (§ 12).
Dead Letter Queue	Una Task che esaurisce maxAttempts in RetryPolicyEngine oggi diventa semplicemente FAILED : nessun posto dedicato la rende ispezionabile in massa dal team operativo, che deve altrimenti filtrare lo storico report a mano.	Retry e circuit breaker (§ 12.5); si aggancia a RF.109 (dashboard).
Optimistic Concurrency Control	Con Desired Count: 1 il rischio è nullo, ma nel momento in cui anche il backend NestJS (non solo il pool agenti) scala orizzontalmente, due istanze potrebbero contendersi la stessa transizione di stato su Task/BatchStep . Un campo di versione (o un compare-and-swap sullo status atteso) previene la corsa critica prima che serva.	Modello di dominio esteso (§ 12): campi Task e BatchStep .
Feature Flag	La Tabella 18 prevede già un fallback “stub” per l’input interattivo del Changelog in caso di taglio budget (RF.98–RF.102): oggi sarebbe un ramo di codice da ricompilare e ridistribuire. Un booleano letto da Parameter Store rende il fallback un toggle operativo invece di un nuovo deploy — utile proprio nello scenario di emergenza per cui è previsto.	Segreti e Configurazione (§ 35), variabile aggiuntiva accanto a quelle di Tabella 36.
Secret Rotation	JWT_SECRET , CREDENTIAL_MASTER_KEY e INTERNAL_SHARED_SECRET sono creati una volta in Secrets Manager (§ 27) ma nessuna politica ne prevede la rotazione periodica: un gap tipico in sede di audit di sicurezza.	Segreti e Chiavi (§ 27) e Segreti e Configurazione (§ 35).

Nessuno di questi sette punti richiede una nuova ADR vincolante per l’MVP: sono annotati qui come debito tecnico consapevole, da valutare insieme al proponente quando si pianifica l’evoluzione oltre il perimetro descritto nella Parte II e nella Parte IV.

13 Sintesi e prossimi passi

Il PoC dimostra molto più di “un agente funziona”: prova che i tre agenti condividono backbone, provider e contratto di output, differendo solo in due punti localizzati, e prova che l’intero stack applicativo (backend NestJS/MongoDB, frontend React, servizio agenti Python) si integra con quel core e con GitHub in modo end-to-end. È un’affermazione architetturale più forte di quella delle stesure precedenti, e copre la soglia di integrazione richiesta dal proponente. Resta un PoC: una operazione per agente, orchestrazione minima, gli stessi rami d’errore per tutti, nessuna scrittura verso l’esterno e infrastruttura locale in attesa della migrazione ad AWS.

I passi immediatamente successivi, una volta condiviso questo perimetro, sono: la stesura dei template di prompt (Docs, OWASP, Changelog) che i profili caricheranno; la scelta e la validazione del fornitore LLM sul golden set; l’implementazione della facade GitHub con la sua whitelist e i tre tool; e la preparazione dei componenti di interfaccia condivisi per la visualizzazione del **Report**. Il passo successivo, quando l’orchestrazione minima (§ 8) non basterà più, è l’evoluzione verso l’orchestratore di produzione progettato in § 12: batch come DAG, retry automatico, scaling del pool di agenti e dashboard di monitoraggio, migrabile in modo incrementale senza toccare il contratto **Report** né gli endpoint già consumati dal frontend.

Questi passi sono stati effettivamente percorsi. Le due Parti che seguono documentano il risultato: la Parte II–III progetta il frontend dell’MVP (autenticazione, ruoli, credenziali, storico, esportazione PDF) riusando lo stack e i contratti validati qui; la Parte IV–V progetta l’infrastruttura AWS di produzione (VPC, ECS su Fargate, MongoDB Atlas, Bedrock) che sostituisce Docker Compose senza toccare il codice applicativo. Insieme, le tre parti raccontano l’intero percorso dal PoC all’MVP.

Parte II

Frontend MVP — Specifica Operativa (UX/UI)

Questa parte contiene esclusivamente le specifiche implementative, la mappa delle pagine, i flussi operativi e le regole di visibilità del frontend. I principi guida, le motivazioni delle scelte e le decisioni architetturali (ADR-FE) sono documentati nella Parte III.

14 Perimetro del documento

14.1 Obiettivo

Il presente documento definisce la progettazione UX/UI del frontend di **Code Guardian** per la release MVP, a partire dai requisiti descritti nell’Analisi dei Requisiti (AdR). L’architettura e lo stack tecnologico validati nel Proof of Concept (Parte I) vengono riutilizzati come base implementativa; le funzionalità non coperte in quella fase (autenticazione, storico, esportazione PDF) vengono qui progettate ex novo. Tutte le scelte operative rispettano i vincoli definiti nella Parte IV (Progettazione Infrastruttura AWS).

14.2 Cosa progetto

- L’intero flusso di autenticazione e la gestione del ruolo operativo dell’utente (**Sviluppatore**, **Security Auditor**, **Project Manager**), definito in fase di registrazione.

- La configurazione delle credenziali dei servizi esterni (GitHub e Task Management unificati).
- La selezione del contesto operativo (repository, riferimento di base, ambito di analisi).
- La configurazione, l'avvio e il monitoraggio delle operazioni degli agenti tramite canale realtime.
- La visualizzazione del report strutturato, estesa con il collegamento alla Pull Request generata dal backend e l'esportazione in formato PDF.
- Lo storico dei report pregressi.
- Le regole di visibilità e abilitazione delle interfacce in base al ruolo dell'utente autenticato.

14.3 Cosa non progetto (fuori perimetro MVP)

I seguenti requisiti sono esclusi dal perimetro della presente progettazione:

- **RF.18** — Selezione di una Pull Request come riferimento di base (UC9.2, UC9.5, UC14). Il tipo di riferimento supportato è esclusivamente *Branch* (con hash di Commit opzionale).
- **RF.38, RF.75–RF.78** — Impostazioni di notifica e flussi di invio email (UC21.2, UC30–UC33).
- **RF.79–RF.81** — Gestione e personalizzazione del template del README (UC34, UC34.1–UC34.3). L'Agente Docs opera esclusivamente con il template di default.

14.4 Mappatura Componenti PoC → Frontend MVP

La Tabella 9 mappa le aree funzionali validate nel Proof of Concept rispetto alle azioni implementative previste per il frontend dell'MVP.

Tabella 9: Mappatura dei componenti del PoC riutilizzati, estesi o introdotti ex novo per il frontend dell'MVP.

Area validata nel PoC	Riuso	Azione per l'MVP
Stack (React, Vite, TanStack Router, pnpm, Zustand)	Totale	Nessuna modifica: stack confermato e utilizzato come base di partenza.
sessionStore	Esteso	Conversione da store di sola presenza del token a store di sessione autenticata (aggiunta utente, JWT, ruolo attivo e stato credenziali).
selectionStore	Totale	Nessuna modifica strutturale: rimozione dell'opzione Pull Request dal tipo di riferimento.
tasksStore + Client WebSocket	Esteso	Integrazione della gestione degli stati interattivi (UC41.1, UC43) per le richieste di input utente. Questo flusso costituisce un'estensione reale rispetto al PoC e richiede l'introduzione di un nuovo evento WS e di un nuovo endpoint REST.
Configurazione Token (GitHub)	Esteso	Evoluzione in schermata generica di "Servizi connessi", per supportare estensioni future.
Pagina /select	Totale	Riuso dell'albero di selezione contestuale; rimosso il ramo Pull Request.
Pagina /run	Esteso	Aggiunta logica di disabilitazione e filtraggio delle operazioni in base al ruolo attivo.
Pagina /tasks (Dashboard)	Esteso	Supporto alla renderizzazione condizionale per gli stati interattivi (inserimento Sprint ID, conferma metadati).
Rendering del Report	Esteso	Mantenuto il dispatcher a blocchi polimorfi. Il componente Proposal viene esteso promuovendo a elemento primario il link alla Pull Request.
Esportazione report	Esteso	Sostituzione dei formati .md/.json con l'invocazione dell'endpoint backend per il download del file .pdf.
Autenticazione e Storico	Nuovo	Componenti e viste progettate ex novo, in quanto non necessarie nella validazione tecnologica del PoC.

15 Architettura dell'informazione

15.1 Ruoli e permessi

Il sistema prevede tre specializzazioni di Utente Generico: lo **Sviluppatore**, il **Security Auditor / Team Lead** e il **Project Manager**. Il ruolo viene selezionato in fase di registrazione e rimane fisso per l'intera durata dell'account, coerentemente con i requisiti di base del sistema.

Ogni ruolo eredita le funzionalità trasversali (selezione del repository, monitoraggio, visual-

izzazione ed esportazione dei report) e aggiunge l'accesso privilegiato alle operazioni del proprio dominio, come delineato nella Tabella 10.

Tabella 10: Mappatura delle operazioni per ruolo operativo.

Operazione	Attore abilitato	Codice Op.
Generazione/aggiornamento README	Sviluppatore	DOCS_README
Documentazione inline (JSDoc)	Sviluppatore	DOCS_INLINE
Documentazione API	Sviluppatore	DOCS_API
Scansione vulnerabilità OWASP Top 10	Security Auditor / Team Lead	SECURITY_OWASP
Verifica conformità Policy-as-code	Security Auditor / Team Lead	SECURITY_POLICY
Changelog tecnico	Project Manager / Sviluppatore	CHANGELOG_TECHNICAL
Changelog di business	Project Manager	CHANGELOG_BUSINESS

Regola di visibilità (Filtraggio): La lista delle operazioni disponibili sulla pagina di avvio mostra *esclusivamente* le opzioni abilitate per il ruolo attivo dell'utente. Le operazioni non permesse vengono omesse dall'interfaccia (nessuna visualizzazione in stato disabilitato). Questo filtraggio è applicato nativamente dall'endpoint backend in base al ruolo dichiarato nel token di sessione.

15.2 Mappa delle pagine e delle rotte

L'accesso alle pagine è regolato dallo stato di autenticazione e dallo stato della credenziale GitHub. La Figura 13 illustra la mappa di navigazione principale e i reindirizzamenti (redirect) condizionali applicati dalle guardie di rotta (dettagliate in § 19.6).

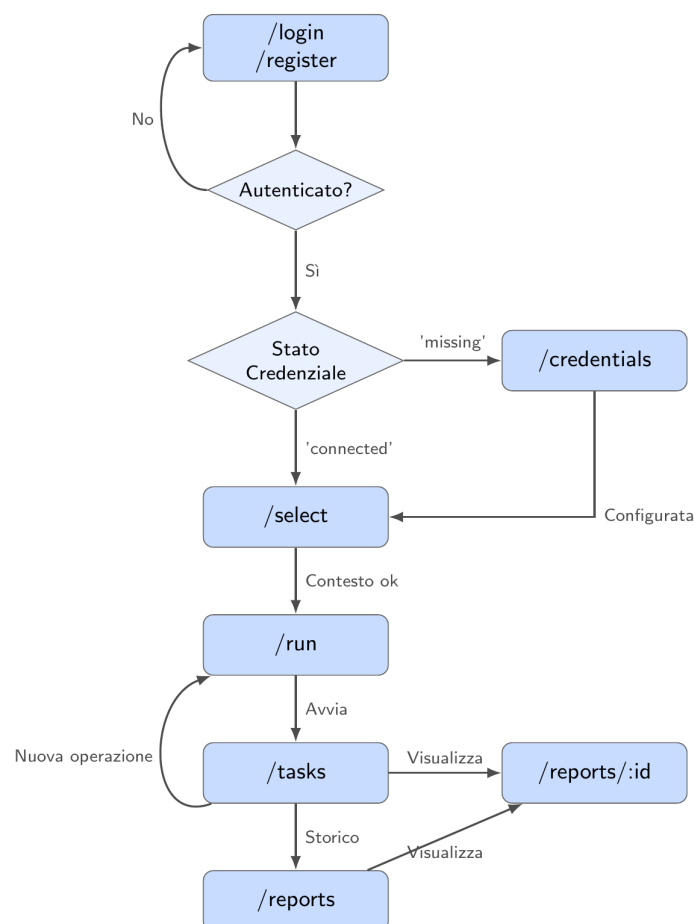


Figura 13: Mappa di navigazione principale e reindirizzamenti condizionali del frontend.

La Tabella 11 definisce le rotte dell'applicazione, i ruoli ammessi e gli endpoint REST consumati da ciascuna di esse.

Tabella 11: Mappa delle rotte del frontend e relative dipendenze API.

Pagina	Rotta	Ruoli	Endpoint consumati
Registrazione	/register	Non autenticato	POST /auth/register
Login	/login	Non autenticato	POST /auth/login
Servizi connessi	/credentials	Tutti	POST /credentials, .../validate, GET /credentials
Selezione contesto	/select	Tutti	GET /repositories, .../refs, .../tree, POST /contexts
Avvio operazioni	/run	Tutti (filtrate)	GET /operations, POST /tasks
Dashboard	/tasks	Tutti	GET /tasks, POST /tasks/:id/cancel, POST /tasks/:id/input, WebSocket
Storico report	/reports	Tutti	GET /reports
Dettaglio report	/reports/:id	Tutti	GET /reports/:id, GET /reports/:id/export?format=pdf

15.3 Credenziali e servizi esterni

La schermata `/credentials` è implementata come una lista scalabile di `ServiceCredential`. Nell'MVP è presente un unico servizio esterno configurabile: **GitHub**. Questo singolo token assolve a entrambe le necessità descritte nei requisiti (accesso al codice e accesso al sistema di Task Management/Issue).

Invalidazione a runtime: Se durante l'esecuzione di un'operazione il backend rileva che il token GitHub non è più valido (es. scadenza o revoca), l'operazione in corso fallisce con codice `CREDENTIAL_INVALID`. Il frontend intercetta questo stato tramite l'evento `WebSocket`, aggiorna lo stato globale dell'utente a `'invalid'` e mostra un banner persistente che invita a riconfigurare la credenziale su `/credentials`, bloccando nel frattempo l'avvio di nuove analisi.

16 Flussi utente per ruolo

16.1 Notazione

I flussi sono descritti come sequenze ordinate di schermate, per mantenere la tracciabilità rispetto agli scenari principali dell'Analisi dei Requisiti. I punti di diramazione per errore (validazioni fallite, timeout, superamento soglie) non sono ripetuti qui, poiché seguono tutti il pattern di stato interrotto descritto in § 21.

16.2 Flusso trasversale: accesso e configurazione iniziale

Comune a tutti e tre i ruoli, precede qualunque operazione degli agenti.

1. L'utente non autenticato arriva su `/login`. Se non possiede un account, passa a `/register`, compila l'anagrafica, le credenziali e seleziona il proprio ruolo operativo iniziale.
2. Al primo login, l'utente privo di credenziali configurate viene reindirizzato a `/credentials`: nessuna operazione degli agenti è raggiungibile senza almeno un servizio connesso e validato.
3. L'utente inserisce il token GitHub; il sistema esegue la validazione sintattica locale e la chiamata di test prima di persistere la credenziale.
4. Completata la configurazione, l'utente accede a `/select` per definire il contesto operativo: URL del repository, branch e, opzionalmente, hash del commit e ambito di analisi.
5. Il contesto salvato (`contextId`) è condiviso da tutte le operazioni successive avviate nella stessa sessione di lavoro; l'utente procede a `/run`.

Le credenziali e il contesto restano validi tra una sessione e l'altra: un utente già configurato, ai login successivi, salta direttamente da `/login` a `/run` o a `/select` se desidera cambiare repository.

16.3 Selezione ed avvio multiplo delle operazioni (trasversale)

Questo flusso è indipendente dal ruolo attivo, il quale determina unicamente le operazioni visibili sulla schermata.

1. Su `/run` il sistema recupera ed elenca le operazioni disponibili per il ruolo attivo dell'utente.
2. L'utente seleziona/deseleziona una o più operazioni (selezione multipla supportata).
3. L'utente conferma l'avvio. Il sistema verifica lato client la presenza di almeno un'operazione selezionata e dei prerequisiti di contesto raccolti in § 3.2.

4. Il sistema invia `POST /tasks` con l'elenco delle operazioni e reindirizza l'utente a `/tasks`, dove ciascuna operazione avviata compare come elemento indipendente della dashboard, partendo dallo stato di *In attesa* (PENDING).
5. Ogni operazione procede in isolamento, delegando al backend la gestione della concorrenza (le operazioni per lo stesso agente rimarranno *In attesa* finché l'orchestrazione non le sbloccherà). Il fallimento di un'analisi non blocca o invalida le altre.

16.4 Flusso Sviluppatore — Agente Docs

Copre le operazioni di Generazione README, Documentazione inline e Documentazione API. Condividono lo stesso flusso a livello di interfaccia.

1. Lo Sviluppatore segue il flusso comune (§ 3.2, § 3.3), selezionando una o più operazioni del dominio Docs.
2. Su `/tasks`, l'operazione avanza attraverso gli stati *In attesa* → *In elaborazione* → *Completato*, senza ulteriori input richiesti.
3. Al completamento, lo Sviluppatore apre il report. Il corpo del report mostra come elemento primario il collegamento alla Pull Request aperta automaticamente dal backend; per la documentazione inline, include inoltre gli avvisi di complessità eccessiva come elementi a gravità informativa.
4. Lo Sviluppatore valuta la proposta direttamente su GitHub: l'interfaccia non prevede un passo di approvazione interno.

16.5 Flusso Security Auditor / Team Lead — Agente Security

Copre la Scansione vulnerabilità OWASP e la Verifica conformità Policy-as-code.

1. Il Security Auditor / Team Lead segue il flusso comune, selezionando le operazioni di sicurezza.
2. Per la verifica policy-as-code, nessun input aggiuntivo è richiesto (la presenza del file `POLICY.md` è validata automaticamente dal backend).
3. Al completamento, il report mostra l'elenco dei riscontri come lista strutturata filtrabile per gravità: per ciascun elemento vengono esposti categoria, gravità, riferimento al file/riga e la *remediation* proposta tramite snippet di codice o testo descrittivo.

16.6 Flusso Project Manager — Agente Changelog

È il flusso con la maggiore interattività, in quanto prevede punti di sospensione a metà esecuzione per l'inserimento e la validazione di input da parte dell'utente. La Figura 14 illustra lo scambio di eventi e schermate.

Al termine dell'elaborazione, il Project Manager visualizza il changelog tecnico formattato in Markdown con i collegamenti ipertestuali ai ticket di origine.

16.6.1 Dipendenza di esecuzione tra Changelog tecnico e di business

Coerentemente con la preconditione di UC41 (“L'utente ha selezionato l'operazione ‘Generazione changelog tecnico’ o ‘Generazione changelog di business’”), la selezione di `CHANGELOG_BUSINESS` in `/run` è sufficiente e autonoma: non richiede la co-selezione di `CHANGELOG_TECHNICAL`. L'Orchestratore avvia un'unica Task con `operation: 'CHANGELOG_BUSINESS'` che attraversa due fasi interne allo stesso stato `RUNNING`:

1. Generazione del Changelog tecnico (UC41), il cui esito viene reso disponibile all'utente come report intermedio.
2. Sospensione dell'esecuzione: la Task valorizza `pendingInput` con `kind: 'BUSINESS_CONFIRMATION'`, forzando in Dashboard l'espansione dell'area interattiva con i comandi "Accetta e genera Business" / "Annulla" (UC45/UC46) — lo stesso pattern grafico già usato per lo Sprint ID.
3. Alla conferma, la medesima Task riprende l'elaborazione e genera il Changelog di business (UC45), transitando infine a `COMPLETED`.



Figura 14: Diagramma di sequenza delle interazioni intermedie per l'Agente Changelog.

In Dashboard, dall'avvio alla conclusione, l'operazione compare quindi come un **singolo elemento**, non come due Task distinte. Se l'utente desidera anche un report tecnico standalone (senza business), seleziona separatamente `CHANGELOG_TECHNICAL`: in tal caso viene

creata una seconda Task indipendente, che non ha alcuna relazione con l'eventuale Task `CHANGELOG.BUSINESS` avviata in parallelo (nessuna Task condivide risultati con l'altra; ciascuna esegue la propria generazione tecnica in isolamento).

Nota Implementativa. La scelta di mantenere disaccoppiati i comandi di selezione in UI e delegare l'attesa all'Orchestratore backend evita di inserire rigida logica di business all'interno del client. Il frontend si limita a leggere lo stato `pendingInput` inoltrato tramite WebSocket e a renderizzare le modali di conseguenza, ignorando del tutto le motivazioni sequenziali che lo hanno innescato.

16.7 Flusso trasversale: consultazione dei report pregressi

1. Da qualunque punto dell'applicazione, l'utente autenticato raggiunge `/reports`, che elenca i report salvati con identificativo, titolo, data e ora di generazione.
2. La selezione di un elemento apre `/reports/:id`. La pagina di dettaglio del report è un unico componente condiviso, raggiungibile sia dalla dashboard (a operazione appena completata) sia dallo storico.
3. Da entrambi i punti di ingresso, l'utente può richiedere l'esportazione in PDF, generato lato backend e reso disponibile in download.

16.8 Flusso trasversale: rilevamento di una credenziale non più valida

Questo evento può interrompere qualunque flusso in qualsiasi momento.

1. Durante l'esecuzione di un'operazione, il backend incontra un errore 401/403 nel leggere dal servizio esterno.
2. Il backend marca la Task come **FAILED** con codice `CREDENTIAL_INVALID`; l'evento WebSocket trasporta questa informazione.
3. Il frontend aggiorna la Task interessata come un normale fallimento e imposta lo stato della credenziale globale a `'invalid'` (§ 2.3).
4. L'intestazione applicativa mostra un banner persistente su tutte le pagine finché lo stato resta `'invalid'`, offrendo un collegamento diretto a `/credentials`.
5. Ripristinata la connessione, il banner scompare e le guardie di rotta (§ 19.6) tornano a consentire l'accesso pieno alle operazioni.

17 Wireframe descrittivi delle schermate

17.1 Notazione

Ogni schermata è descritta per regioni di layout (intestazione, corpo, aree laterali o modali) e per elementi ordinati dall'alto verso il basso. I riferimenti tra parentesi (UC/RF) individuano il requisito di origine; i riferimenti preceduti da **Componente:** rimandano al catalogo del Design System (§ 5.3), in cui l'elemento è definito e stilizzato una singola volta.

17.2 Registrazione — `/register`

Corpo. Modulo centrato a colonna singola:

- Campo testo **Nome** (UC1.1, RF.2).

- Campo testo **Cognome** (UC1.1, RF.2).
- Campo testo **Indirizzo email** (UC1.3, RF.3), con validazione inline al cambio del focus.
- Campo **Password** (UC1.4, RF.3).
- **Selettore di ruolo** (UC1.5, RF.4) a scelta singola (Sviluppatore, Security Auditor / Team Lead, Project Manager) con riga di descrizione sintetica per opzione.
- Pulsante di conferma primario “Crea account”.
- Collegamento secondario verso `/login` per l’utente già registrato.

Stati.

- *Email già registrata* (UC2/RF.5): messaggio non bloccante sopra il modulo, con link diretto a `/login`.
- *Dati non validi* (UC3/RF.6): **Componente:** Campo di modulo con validazione inline. Il pulsante primario resta disabilitato.

17.3 Login — `/login`

Corpo. Modulo centrato:

- Campo testo **Indirizzo email** (UC4.1, RF.8).
- Campo **Password** (UC4.2, RF.8).
- Pulsante di conferma primario “Accedi”.
- Collegamento secondario verso `/register`.

Stati.

- *Credenziali errate* (UC5/RF.9): messaggio d’errore generico posizionato sopra il modulo, senza indicare quale campo specifico è errato.
- *Successo*: redirect condizionale a `/credentials` (se token assente) o `/select` (se token presente e valido).

17.4 Servizi connessi — `/credentials`

Corpo. Lista generica di provider (nell’MVP, un solo elemento).

- Elemento “GitHub” con sottotitolo esplicativo. Come argomentato in § 2.3, questo singolo campo collassa e assolve contestualmente all’inserimento del token per l’accesso al codice (RF.11) e per l’accesso al sistema di Task Management (RF.12).
- *Se non configurato*: campo di input per il token e pulsante “Connetti”.
- *Se configurato*: etichetta di stato “Connesso” con timestamp, pulsanti “Verifica di nuovo” e “Rimuovi”.
- Area informativa inferiore sulla predisposizione per futuri provider.

Stati.

- *Formato non valido* (UC8/RF.14): errore inline sul campo prima dell’invio.
- *Validazione fallita* (UC7/RF.13): messaggio d’errore del servizio. Pulsante “Continua” disabilitato.
- *Credenziale invalidata a runtime* (§ 2.3): messaggio dedicato “Il token risulta scaduto o revocato”, stesso comportamento dello stato di validazione fallita.

17.5 Selezione del contesto — /select

Corpo, prima area (Repository e riferimento).

- Campo testo **URL del repository Git** (UC9.1, RF.16).
- Campo testo **Nome del branch** (UC9.3, RF.17), con autocompletamento attivo post-validazione URL (UC11/RF.20).
- Campo testo opzionale **Hash del commit** (UC9.4, RF.17).

Corpo, seconda area (Ambito di analisi, abilitata post-validazione).

- **Componente:** **Albero file** per la selezione a tre stati (file/directory). Selezionare una directory marca ricorsivamente i figli (UC16.2, RF.28).
- Comando rapido “Seleziona intero repository” (UC16.1, RF.26).
- Contatore in tempo reale (file e righe stimate) con soglia visiva (UC19/RF.31).
- Pulsante di conferma primario “Conferma contesto”.

Stati.

- *Repository o riferimento non trovati/permessi mancanti:* errore inline sul campo.
- *Linguaggio non supportato:* avviso non bloccante sopra l'albero (UC15/RF.24).
- *Ambito vuoto/inaccessibile:* messaggio d'errore bloccante prima dell'invio (UC17, UC18).

17.6 Avvio operazioni — /run

Corpo.

- Lista delle operazioni disponibili, già filtrata in base al ruolo utente (§ 2.1). Ogni operazione della Tabella 10 — incluso **CHANGELOG_BUSINESS** — compare come voce indipendente e paritetica, coerentemente con UC20.1/UC21.1.
- Per ogni operazione: Nome, descrizione sintetica, casella di selezione.
- **CHANGELOG_BUSINESS** è selezionabile e avviabile autonomamente, senza necessità di selezionare anche **CHANGELOG_TECHNICAL**: la generazione tecnica è inclusa come primo passo interno della stessa Task (§ 3.6.1).
- Pulsante di conferma primario “Avvia”.

Stati.

- *Nessuna operazione selezionata:* impedimento all'attivazione del pulsante (UC21.4/RF.41).

17.7 Dashboard — /tasks

Corpo. Lista delle operazioni del batch corrente (UC22).

- Nome dell'operazione.
- **Componente:** **Indicatore di stato** (*In attesa, In elaborazione, Completato, Fallito, Annullato*) e barra di avanzamento (RF.46).
- Comando “Annulla” (attivo in *attesa/elaborazione*).

- Comando “Visualizza report” (attivo in *Completato* o *Fallito*).

Aree interattive (Agente Changelog). Espansioni *inline* per le task in stato di richiesta input:

- *Richiesta Sprint ID* (UC41.1): campo testo, comandi “Conferma/Annulla” (UC42).
- *Metadati insufficienti* (UC43): elenco identificativi scartabili, comandi “Procedi escludendo/Annulla operazione” (UC44).
- *Conferma Changelog di business* (`pendingInput.kind === 'BUSINESS_CONFIRMATION'`): anteprima del changelog tecnico appena generato, comandi “Accetta e genera Business” (UC45) / “Annulla” (UC46).

L’elemento della Dashboard associato a un’operazione `CHANGELOG_BUSINESS` resta un unico elemento per l’intera durata del flusso (generazione tecnica, attesa conferma, generazione business): non vengono mai mostrate due righe distinte per la stessa selezione.

Stati.

- *Errore di orchestrazione*: **Componente**: **Stato d’errore** limitato al singolo elemento fallito, garantendo l’isolamento visivo (UC23/RF.48).

17.8 Dettaglio report — /reports/:id

Intestazione. Riga di metadati: repository, riferimento, ambito analizzato, tempo di esecuzione, timestamp (UC26.1).

Corpo (Dispatcher a blocchi). Ordine e tipo definiti dalla risposta API:

- **TextBlock**: Markdown renderizzato (changelog).
- **FindingBlock / PolicyViolationBlock**: Liste strutturate filtrabili per gravità (Agente Security) con **Componente**: **Snippet di codice** per le remediation.
- **Proposal**: **Componente**: **Collegamento Pull Request** (elemento primario) e anteprima diff collassabile (Agente Docs).
- **Modulo di validazione**: Comandi di accettazione/annullamento (changelog di business).

Comandi di pagina.

- Pulsante “Esporta PDF” (UC28/RF.73) con indicatore di caricamento.

Stati.

- *Status FAILED*: Corpo sostituito dal **Componente**: **Stato d’errore** (tipo, fase, messaggio).
- *Esportazione fallita*: Errore transitorio a pagina (UC29).

17.9 Storico report — /reports

Corpo.

- Lista di elementi report (UC24): identificativo, titolo descrittivo, timestamp.
- Selezione elemento → navigazione verso /reports/:id (UC25).

17.10 Intestazione applicativa

Elemento persistente su tutte le rotte autenticate.

- Nome dell'utente autenticato.
- **Componente:** `Indicatore di stato` utilizzato come badge di sola lettura per il ruolo assegnato.
- **Banner di credenziale invalidata** (§ 2.3): fascia d'avviso non bloccante per la navigazione in lettura, con collegamento a `/credentials`.
- Comando "Esci".

Stati.

- *Fallimento di rete (Logout)*: Errore contestuale al tentativo di uscita.

18 Design System e componenti condivisi

18.1 Stack di implementazione

Il design system dell'MVP è implementato utilizzando **Tailwind CSS** come base per le classi di utilità (*utility-first*) e **shadcn/ui** come libreria di primitive accessibili per i componenti interattivi complessi (modali, selettori, form). Questa scelta tecnologica garantisce nativamente il rispetto del requisito **RV.5**, assicurando la piena compatibilità senza anomalie grafiche sulle versioni più recenti dei principali browser desktop (Google Chrome, Mozilla Firefox, Microsoft Edge v145 o superiore).

18.2 Token visivi

I token sono definiti come variabili di tema globali, in modo da poter essere declinati visivamente senza modificare il codice dei singoli componenti che li consumano.

Tabella 12: Categorie di token visivi condivisi e ambito di applicazione.

Categoria di token	Uso previsto
Colori semantici di stato	Un colore per ciascuno dei cinque stati operativi (<i>Attesa, Elaborazione, Completato, Fallito, Annullato</i>). Riutilizzati identicamente nell'Indicatore di stato e in ogni badge o messaggio che comunica un esito.
Colori semantici di gravità	Un colore per ciascun livello di gravità delle segnalazioni dell'Agente Security (<i>Critica, Alta, Media, Bassa, Informativa</i>). Il livello informativo è riutilizzato per gli avvisi di complessità dell'Agente Docs.
Colore di superficie neutra	Sfondo di pagina, delle card e dei moduli; un'unica scala di grigi utilizzata per l'intera applicazione.
Tipografia	Una famiglia font per il testo dell'interfaccia (etichette, corpo, moduli) e una famiglia monospaziata dedicata esclusivamente agli Snippet di codice e ai riferimenti a percorsi/righe di file.
Spaziatura	Scala unica su base 4px, applicata uniformemente per garantire coerenza tra densità informative molto diverse (es. form di login vs lista report Security).
Raggio e ombreggiatura	Valori unici per card, finestre modali e badge, per mantenere la stessa grammatica visiva tra le diverse aree dell'applicativo.

18.3 Catalogo dei componenti condivisi

I seguenti componenti sono definiti una sola volta e riutilizzati in tutte le schermate descritte in § 4.

- **Indicatore di stato:** Etichetta testuale con colore semantico associato, usata nella Dashboard e per lo stato dei servizi connessi. Include opzionalmente una barra di avanzamento.
- **Badge di gravità:** Etichetta compatta con colore semantico, usata per i riscontri di sicurezza e gli avvisi di complessità del codice.
- **Albero file:** Struttura ad albero navigabile con caselle di selezione a tre stati (non selezionato, selezionato, parzialmente selezionato), usata nella selezione dell'ambito di analisi.
- **Blocco di report:** Componente *dispatcher* che, dato un elemento del payload di risposta, renderizza il sottocomponente corretto in base al tipo (testo, lista, snippet, modulo).
- **Collegamento Pull Request:** Pulsante o link in evidenza verso la Pull Request aperta dal backend, con anteprima del *diff* disponibile come dettaglio secondario collapsabile.
- **Snippet di codice:** Contenitore monospaziato con evidenziazione sintattica minima, usato per il codice correttivo proposto e per le anteprime diff.
- **Modulo di validazione:** Coppia di comandi (primario/secondario) con testo esplicativo, usato per innescare i changelog di business e per le aree interattive della Dashboard.
- **Stato d'errore:** Componente a pagina intera o a riquadro che riporta il tipo di errore, la fase in cui si è verificato e il messaggio descrittivo (§ 21).
- **Campo di modulo con validazione inline:** Campo di input con stato neutro/errore e messaggio contestuale posizionato al di sotto del campo stesso.

18.4 Stati interattivi comuni

Ogni componente che avvia una richiesta di rete (REST) espone sempre tre stati distinti, per garantire un feedback visivo immediato:

1. **In corso:** Il controllo che ha originato la richiesta mostra un indicatore di caricamento e viene disabilitato per prevenire invii duplicati.
2. **Esito positivo:** Transizione visiva immediata verso lo stato successivo previsto (es. redirect, aggiornamento lista, chiusura modale). L'utente non viene mai lasciato sulla schermata originaria con un semplice messaggio di conferma senza indicazioni.
3. **Esito negativo:** Attivazione del componente **Stato d'errore** o del campo in stato di errore (§ 21), senza perdita dei dati già inseriti dall'utente nella form.

18.5 Accessibilità (A11y)

L'utilizzo di `shadcn/ui` fornisce nativamente la navigazione da tastiera, la corretta attribuzione dei ruoli ARIA e la gestione visibile del focus. Come regola aggiuntiva di progettazione, i colori semantici di stato e gravità (§ 5.2) devono essere sempre accompagnati da un'etichetta testuale esplicita, e non essere mai affidati alla sola percezione cromatica.

19 Stato applicativo

19.1 Principio generale

L'applicazione adotta una politica rigorosa per la gestione dello stato globale: entra in uno *store* Zustand esclusivamente ciò che deve sopravvivere a un cambio di rotta (pagina) o che deve essere letto contemporaneamente da più componenti distanti nell'albero. Tutto il resto rimane relegato allo stato locale del singolo componente o della singola pagina.

La Figura 15 illustra l'architettura dei tre store globali dell'applicativo e le loro direttrici di comunicazione.

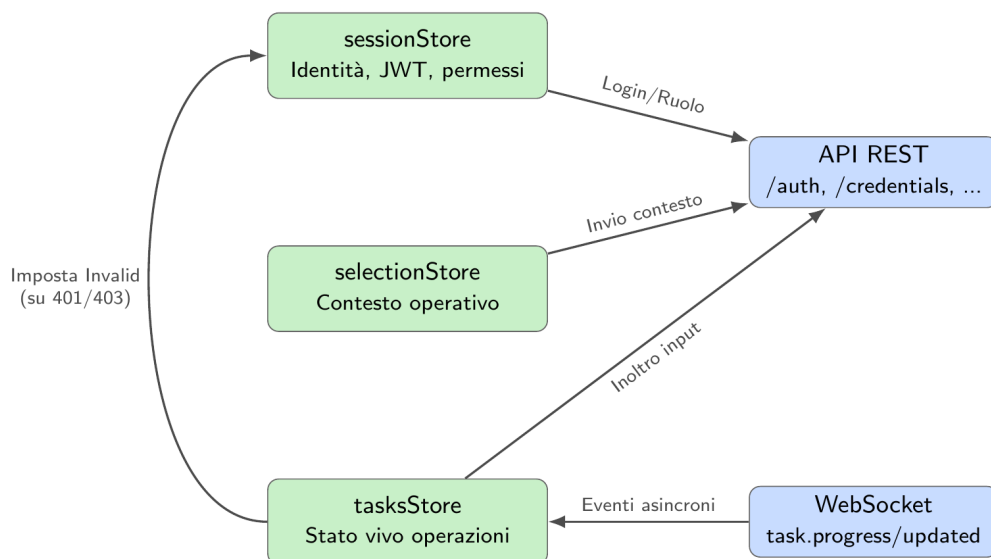


Figura 15: Architettura degli store applicativi, comunicazione di rete ed effetti collaterali.

Attenzione. Il JSON Web Token non deve **mai** essere salvato all'interno del `localStorage` o del `sessionStorage` del browser. Poiché l'MVP non prevede protezioni WAF avanzate contro attacchi XSS, conservare il token esclusivamente nella memoria volatile dello store Zustand garantisce che uno script malevolo non possa estrarlo in modo persistente.

19.2 sessionStore — Identità e credenziali

Mantiene la sessione utente e lo stato autorizzativo. Il JSON Web Token (JWT) è mantenuto esclusivamente in memoria (nello store) e mai persistito nel `localStorage`. Una ricarica completa della pagina richiede un nuovo login.

```
interface SessionState {
  user: { id: string; firstName: string; role: UserRole } | null
  token: string | null
  credentialsStatus: 'unknown' | 'missing' | 'connected' | 'invalid'
  isAuthenticated: () => boolean

  login(user, token): void
  logout(): void
  setCredentialsStatus(status): void
  markCredentialsInvalid(): void
}

type UserRole = 'DEVELOPER' | 'SECURITY_AUDITOR' | 'PROJECT_MANAGER'
```

Listing 4: Interfaccia del sessionStore.

- **Campo role:** Unica fonte di verità per le regole di visibilità delle operazioni (§ 2.1). Popolato al login e immutabile per la durata della sessione.
- **Campo credentialsStatus:** Evita di interrogare GET `/credentials` su ogni rotta protetta. Il valore `'invalid'` differisce da `'missing'`: indica che una credenziale configurata è stata revocata o è scaduta durante l'utilizzo.

19.3 selectionStore — Contesto operativo

Contiene il `contextId` e i parametri dell'ambito di analisi in uso nella sessione corrente. Viene popolato dalla pagina `/select` e letto in fase di avvio dalle richieste della pagina `/run`.

19.4 tasksStore — Stato vivo delle operazioni

Contiene lo stato in tempo reale delle operazioni correnti, aggiornato asincronamente dagli eventi WebSocket inviati dal backend. La Figura 16 descrive la macchina a stati di una Task gestita da questo store.

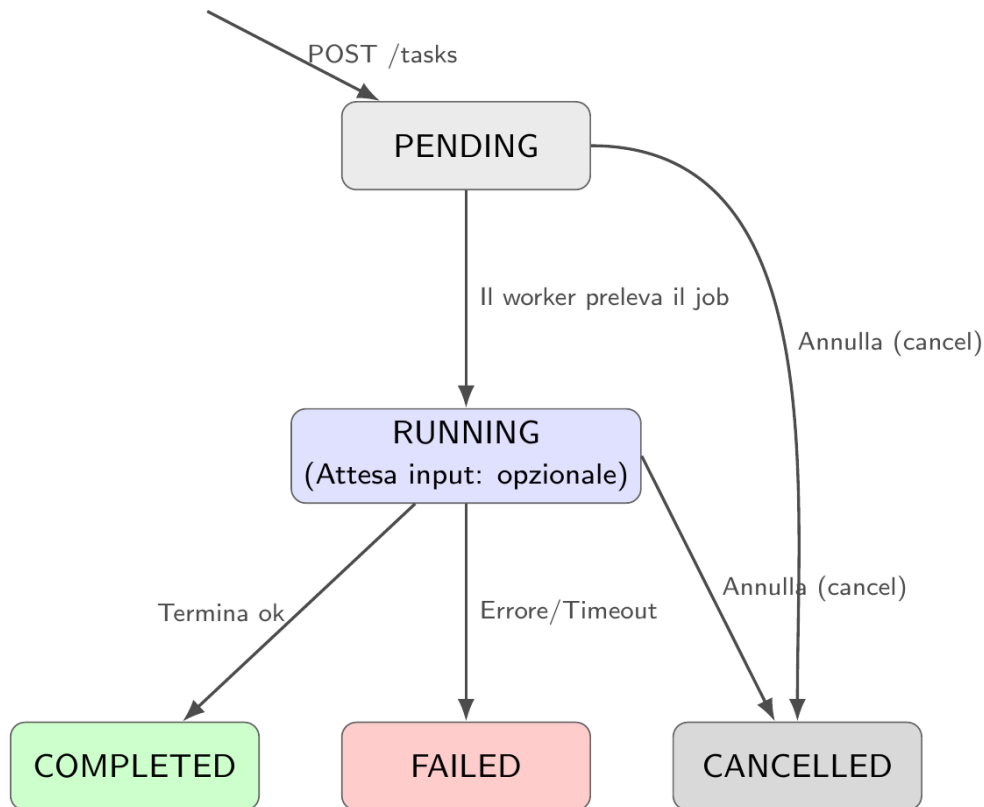


Figura 16: Ciclo di vita di una Task. La richiesta di input (`pendingInput`) non costituisce un nuovo stato, ma una condizione sospensiva interna allo stato `RUNNING`.

Il campo `pendingInput`, quando valorizzato, è il segnale che innesca l'espansione dell'area interattiva sulla Dashboard (§ 4). L'annullamento della Task, anche durante la fase di input, avviene invocando l'azione unificata `cancel(taskId)`.

```

interface TasksState {
  tasks: Record<string, TaskEntry>
  upsertFromEvent(event: TaskEvent): void // event: task.progress |
    task.updated
  cancel(taskId): void // chiamata a POST /tasks/:id
    /cancel
  resolvePendingInput(taskId, response): void // risoluzione input
    utente
}

interface TaskEntry {
  id: string
  operation: OperationCode
  status: 'PENDING' | 'RUNNING' | 'COMPLETED' | 'FAILED' | 'CANCELLED'
  progressPercent: number
  currentStage: string | null
  reportId: string | null
  error: { code: string; message: string; stage: string } | null

  pendingInput: PendingInput | null
}

type PendingInput =
  | { kind: 'SPRINT_ID' }

```

```
| { kind: 'INCOMPLETE_TASKS'; taskIds: string[] }
| { kind: 'BUSINESS_CONFIRMATION'; technicalReportId: string }
```

Listing 5: Interfaccia del tasksStore con gestione input interattivi.

Effetto collaterale su sessionStore: Alla ricezione dell'evento `task.failed` con `error.code === 'CREDENTIAL_INVALID'`, l'handler del `tasksStore` invoca l'azione `markCredentialsInvalid()` del `sessionStore`. È l'unico punto architetturale in cui uno store modifica lo stato di un altro, in risposta a un evento di rete che invalida le credenziali a livello globale.

19.5 Dati intenzionalmente non globali

Per il principio di separazione dichiarato, le seguenti entità rimangono confinate allo stato locale di React (`useState` o `useQuery`):

- La lista dei servizi connessi (`GET /credentials`), letta e mostrata unicamente dalla pagina `/credentials`.
- L'albero dei file del repository e l'elenco di *refs*, necessari solo in `/select`.
- La cache dello storico dei report (`GET /reports`), mantenuta per la sola pagina `/reports`.
- Il contenuto del singolo report aperto (`GET /reports/:id`).

19.6 Guardie di rotta

Le restrizioni di navigazione client-side si basano unicamente sulla lettura sincrona del `sessionStore`. Non vengono effettuate chiamate di rete al momento del cambio rotta.

- **Accesso bloccato per utenti autenticati:** `/login` e `/register` (redirect a `/run`).
- **Accesso bloccato per utenti non autenticati:** Tutte le altre rotte (redirect a `/login`).
- **Accesso in lettura a Task e Report:** Libero per tutti gli utenti autenticati, indipendentemente dallo stato delle credenziali GitHub (inclusi `'missing'` o `'invalid'`).
- **Avvio nuove operazioni bloccato:** Se `credentialsStatus` è `'missing'` o `'invalid'`, l'accesso a `/select` e la conferma di avvio su `/run` effettuano un redirect immediato a `/credentials`.

20 Contratto API consumato

20.1 Principi generali del contratto

Il contratto API si appoggia sul design architetturale già validato nella fase di Proof of Concept (Parte I). Il frontend aderisce ai seguenti vincoli strutturali:

- **Prefisso globale:** Tutte le rotte consumate dal frontend sono esposte sotto il prefisso `/api/v1`. Le chiamate utilizzano esclusivamente percorsi relativi, delegando l'instradamento all'infrastruttura sottostante (dettagliato in § 26).
- **Separazione dei piani:** Esiste una netta separazione tra il piano pubblico (browser → backend) e il piano interno (agenti → backend). Il frontend ha visibilità e accesso unicamente al piano pubblico.

- **Autenticazione realtime:** La connessione al canale WebSocket viene autenticata trasmettendo il JSON Web Token (JWT) durante la fase di *handshake*.

Attenzione — Vincoli di Rete Ereditati dall’Infrastruttura AWS:

- **Nessun setup CORS:** Il frontend e il backend sono serviti dalla stessa origine (CloudFront). Non configurare intercettori per preflight OPTIONS, né iniettare header CORS custom nei client HTTP (come Axios o Fetch API).
- **Path Relativi (Vite):** È categoricamente vietato inserire URL assoluti (es. `https://api...`) all’interno della variabile d’ambiente `VITE_API_BASE_URL`. Usare sempre percorsi relativi (es. `/api/v1`). Hardcodare un dominio causerà il fallimento silente delle pipeline CI/CD tra i vari ambienti.

20.2 Endpoint REST pubblici

La Tabella 13 elenca gli endpoint del piano pubblico consumati dal frontend, inclusivi delle estensioni implementate per l’MVP (autenticazione, gestione credenziali, storico, esportazione PDF ed elaborazione input asincroni).

L’endpoint di esportazione report utilizza esclusivamente il formato PDF. La generazione avviene lato backend e il frontend consuma il file direttamente in streaming (nel corpo della risposta HTTP).

Tabella 13: Endpoint REST pubblici consumati dal frontend, raggruppati per area funzionale.

Metodo e Path	Scopo	Requisiti
<i>Autenticazione</i>		
POST /auth/register	Registrazione, include il ruolo operativo	RF.1–6
POST /auth/login	Login, restituisce il JWT	RF.7–9
GET /auth/me	Profilo e ruolo dell'utente autenticato	RF.7
<i>Servizi Connessi</i>		
GET /credentials	Elenco dei provider configurati e relativo stato	RF.10
POST /credentials	Salva una credenziale per un provider	RF.10–11
POST /credentials/:id/validate	Verifica la validità e i permessi del token	RF.13, RS.4
DELETE /credentials/:id	Rimuove una credenziale	UC6
<i>Contesto Operativo</i>		
GET /repositories	Elenco dei repository accessibili dall'utente	RF.15
GET /repositories/:owner/:repo/refs	Branch disponibili	RF.16–17
GET /repositories/:owner/:repo/tree	Struttura del repository (richiede ?ref=)	RF.25–28
POST /contexts	Crea l'AnalysisContext	RF.15–31
<i>Avvio ed Esecuzione</i>		
GET /operations	Operazioni disponibili (già filtrate per ruolo)	RF.32–34
POST /tasks	Avvia una o più task	RF.36–42
GET /tasks	Elenco delle task per la Dashboard	RF.43–45
GET /tasks/:id	Dettaglio di una singola task	RF.45–46
POST /tasks/:id/cancel	Annula una task, anche durante l'attesa di input	RF.46, UC42, UC44
POST /tasks/:id/input	Risolve una richiesta di input utente in sospenso (Sprint ID o esclusione task)	UC41.1, UC43, RF.98
<i>Storico e Report</i>		
GET /reports	Storico dei report	RF.49–52
GET /reports/:id	Dettaglio di un report	RF.53–64
GET /reports/:id/export?format=pdf	Esporta il report in PDF (streaming binario)	RF.73–74

20.3 Canale realtime (WebSocket)

Il canale WebSocket è utilizzato in modalità unidirezionale (backend → frontend) per il flusso applicativo. Il frontend emette unicamente l'evento di sottoscrizione, mentre le risposte ap-

plicative fornite dall'utente non transitano per il socket, bensì tramite l'endpoint REST POST `/tasks/:id/input`.

In caso di caduta della connessione di rete, alla riapertura del socket il frontend esegue un polling di riallineamento tramite GET `/tasks` per recuperare l'eventuale stato `pendingInput` o gli avanzamenti persi.

Attenzione. I meccanismi di riconnessione automatica integrati nel client Socket.IO non recuperano i messaggi persi durante il periodo di disconnessione. È responsabilità del frontend, all'evento di `reconnect`, invocare l'endpoint REST GET `/tasks` per allineare forzatamente lo stato del `tasksStore`, recuperando eventuali variazioni di percentuale o richieste di input sfuggite al socket.

Tabella 14: Eventi WebSocket emessi dal backend verso il frontend.

Evento	Payload	Uso nel frontend
<code>task.updated</code>	<code>{ taskId, status, reportId? }</code>	Aggiorna l'indicatore di stato nella Dashboard.
<code>task.progress</code>	<code>{ taskId, stage, percent }</code>	Avanza la barra di progresso.
<code>task.failed</code>	<code>{ taskId, error }</code>	Attiva lo stato di errore isolato sulla singola task (§ 21). Il codice <code>'CREDENTIAL_INVALID'</code> ha effetti globali.
<code>batch.completed</code>	<code>{ batchId, completed, failed }</code>	Sblocca i controlli successivi (es. abilitazione link Report).
<code>task.inputRequired</code>	Il payload coincide con il tipo <code>PendingInput</code> (§ 20): <code>{ kind: 'SPRINT_ID' }</code> oppure <code>{ kind: 'WORK_ITEM_QUALITY', insufficientItems: [...] }</code> oppure <code>{ kind: 'TECHNICAL_CHANGELOG_CONFIRMATION', technicalChangelogPreview: string }</code> .	Valorizza <code>pendingInput</code> nel <code>tasksStore</code> (§ 19.4), forzando l'espansione dell'area interattiva con il pannello di input specifico per il kind ricevuto.

20.4 Schemi dei DTO principali

Gli schemi (Data Transfer Object) implementati nel frontend riflettono esattamente il contratto TypeScript fornito dall'API. L'enumerazione `OperationCode` ricalca le operazioni definite in § 2.1.

```
// --- Body di POST /auth/register
interface RegisterDto {
  firstName: string
  lastName: string
  email: string
  password: string
  role: 'DEVELOPER' | 'SECURITY_AUDITOR' | 'PROJECT_MANAGER'
}

// --- Risposta di POST /auth/login
// (il profilo completo si ottiene con GET /auth/me)
interface AuthTokenDto {
```

```

    accessToken: string
}

// --- Risposta di GET /auth/me
interface UserProfileDto {
    id: string
    firstName: string
    lastName: string
    email: string
    role: 'DEVELOPER' | 'SECURITY_AUDITOR' | 'PROJECT_MANAGER'
}

// --- Credenziali di Servizio ---
interface ServiceCredentialDto {
    id: string
    provider: 'GITHUB'
    status: 'CONNECTED' | 'INVALID'
    lastValidatedAt: string | null
}

// --- Struttura di una Task e degli Input Asincroni ---
interface TaskDto {
    id: string
    batchStepId: string | null // null = task avviata singolarmente
    operation: OperationCode
    status: 'PENDING' | 'RUNNING' | 'COMPLETED' | 'FAILED' | 'CANCELLED'
    progressPercent: number
    currentStage: string | null
    reportId: string | null
    error: { code: string; message: string; stage: string } | null
    // sempre presente (null se nessun input richiesto)
    pendingInput: PendingInput
}

// Allineato con il contratto Backend MVP (Parte V, sez. be-contratto-api)
// FONTE DI VERITA' per i kind name: sezione Backend MVP, sez. 6.3
type PendingInput =
    | { kind: 'SPRINT_ID' }
    | { kind: 'INCOMPLETE_TASKS'; taskIds: string[] }
    | { kind: 'BUSINESS_CONFIRMATION'; technicalReportId: string }
    | null

// --- Body di POST /tasks/:id/input ---
type SubmitInputDto =
    | { kind: 'SPRINT_ID'; sprintId: string }
    | { kind: 'INCOMPLETE_TASKS'; action: 'PROCEED' | 'CANCEL' }
    | { kind: 'BUSINESS_CONFIRMATION'; action: 'PROCEED' | 'CANCEL' }

// --- Elemento restituito da GET /reports ---
interface ReportSummaryDto {
    id: string
    operation: OperationCode
    status: 'COMPLETED' | 'FAILED'
    generatedAt: string
    title: string
}

```

20.5 API interne

Le API interne (`/internal/*`) sono adibite alla sola comunicazione machine-to-machine tra gli agenti intelligenti e il backend applicativo. Sono autenticate tramite firme HMAC. Il frontend non le consuma, non vi ha accesso di rete e non è tenuto a integrarne le interfacce nel proprio perimetro di progetto.

21 Gestione degli errori

21.1 Due famiglie di errore

Il frontend gestisce due categorie distinte di errori, trattate con due pattern di interfaccia differenti:

- **Errori di validazione e configurazione:** Sincroni. Emergono durante la compilazione di un modulo o la conferma di una selezione. Vengono risolti immediatamente dall'utente rimanendo sulla stessa schermata.
- **Errori di esecuzione di un'operazione:** Asincroni. Emergono a distanza di tempo dall'avvio di un'operazione, solitamente notificati tramite canale realtime o al caricamento della Dashboard.

21.2 Pattern per gli errori di validazione e configurazione

La regola generale prevede che il messaggio di errore compaia *contestualmente* (al di sotto del campo o dell'area interessata), avvalendosi del **Componente: Campo di modulo con validazione inline**.

Eccezioni e regole di comportamento:

- **Login (Credenziali errate):** Il messaggio è generico e posizionato sopra il modulo, senza indicare quale campo specifico (email o password) sia errato.
- **Persistenza dati:** I dati già inseriti dall'utente restano sempre visibili e non vengono mai azzerati a seguito di un errore di validazione.
- **Blocco rete:** Nessuna chiamata di rete aggiuntiva verso le API o verso servizi esterni (es. validazione token GitHub) viene effettuata finché l'errore locale non è stato corretto.

21.3 Pattern per gli errori di esecuzione di un'operazione

Gli errori che si verificano durante l'esecuzione dell'agente o dell'orchestrazione vengono propagati all'interfaccia seguendo questo flusso:

1. Il backend porta lo stato della task a **FAILED** e lo notifica via WebSocket.
2. Nella Dashboard, esclusivamente l'elemento interessato passa allo stato "Fallito", garantendo l'isolamento (le altre operazioni continuano ad aggiornarsi senza interruzione).
3. L'elemento fallito espone il comando "Visualizza report". L'apertura della vista dettaglio sostituisce i blocchi di contenuto con il **Componente: Stato d'errore**.

Il componente **Stato d'errore** è parametrizzato per mostrare il messaggio specifico, la fase di interruzione e un'eventuale azione secondaria, come mappato nella Tabella 15.

Tabella 15: Mappatura degli errori di esecuzione sul componente Stato d'errore.

Causa / Trigger	Messaggio in interfaccia	Azione secondaria
Superamento soglia massima di utilizzo LLM	Limite di utilizzo del modello AI raggiunto; riprovare successivamente.	Nessuna (blocco temporizzato)
Timeout del modello LLM (oltre soglia configurata)	Il modello AI ha impiegato troppo tempo a rispondere.	“Riprova” (riavvia sola task)
Output del modello LLM non parsabile	L'output generato dall'AI non è nel formato atteso.	“Riprova”
Superamento dei limiti dimensionali a runtime	Il contesto analizzato eccede la capacità di lettura del modello.	Rimanda a <code>/select</code> per ridurre l'ambito
Risorsa di contesto mancante (es. <code>POLICY.md</code> , Sprint ID)	Risorsa richiesta non trovata: [nome risorsa].	Nessuna
Risorsa di contesto malformata	La risorsa richiesta non è interpretabile: [dettaglio problema].	Nessuna
Fallimento creazione Pull Request (permessi insufficienti)	Impossibile aprire la Pull Request; verificare i permessi del token.	Rimanda a <code>/credentials</code>
Token GitHub scaduto o revocato a runtime	Il token configurato non è più valido; verificalo su “Servizi connessi”.	Rimanda a <code>/credentials</code>

21.3.1 Errore di orchestrazione o routing

I fallimenti dovuti a disservizi infrastrutturali o di rete che non derivano esplicitamente dalla logica interna degli agenti o dalle validazioni (es. caduta connettività verso GitHub) vengono gestiti dal frontend come errori generici. Il messaggio mostrato è: *“Impossibile completare l'operazione per un errore interno di sistema”*. L'unica azione secondaria proposta è il comando generale “Riprova”.

21.4 Errore di esportazione (Report)

Il fallimento dell'esportazione PDF lato backend diverge dal pattern di esecuzione: l'errore viene mostrato come messaggio transitorio (toast/alert) nella pagina di dettaglio del report. Non altera lo stato del report (che rimane **COMPLETED**) e non sostituisce i blocchi di contenuto renderizzati a schermo, permettendo all'utente di continuare la consultazione.

22 Tracciamento Requisito → Decisione Frontend

Questa sezione mappa i requisiti funzionali e qualitativi (definiti nell'Analisi dei Requisiti) sulle scelte operative e implementative applicate nel frontend. Al fine di mantenere la tabella focalizzata sulle scelte architetturali, non sono elencati i requisiti funzionali di base (relativi alla mera presenza di campi di testo o pulsanti) o i requisiti fuori dal perimetro di competenza del client.

22.1 Decisioni Operative e Architetture

Tabella 16: Mappatura Requisiti - Decisioni operative del Frontend.

Requisito	Descrizione Sintetica	Implementazione Frontend	Rif.
RQ.10	Tempo di risposta percepito $\leq$ 2 secondi	Adozione di <code>sessionStore</code> in memoria per guardie di rotta sincrone (zero chiamate bloccanti al cambio pagina) ed esposizione nativa degli stati di caricamento per ogni componente interattivo.	§ 5.4, § 19.6
RQ.11	Apprendibilità sistema $\leq$ 30 minuti	Filtraggio visivo a monte: la pagina di avvio operazioni nasconde totalmente le opzioni disabilitate per il ruolo attivo, abbassando il carico cognitivo.	§ 2.1
RF.4	Scelta del ruolo operativo	Componente: Selettore di ruolo integrato esclusivamente nella fase di registrazione, definendo il ruolo per l'intera vita dell'account.	§ 4
RF.14, RS.4	Validazione sintattica e logica token	Componente di validazione <i>in-line</i> sul client per la sintassi, seguito da validazione esplicita via API per garantire autorizzazioni remote sufficienti prima del salvataggio.	§ 8.2
RF.36, RF.37	Selezione e avvio multiplo	Adozione di una lista con caselle di controllo indipendenti in <code>/run</code> e invio batch tramite array nel payload della <code>POST /tasks</code> .	§ 3.3
RF.96, RF.103–RF.105 UC41, UC45, UC46	Dipendenza Changelog tecnico → business	<code>CHANGELOG_BUSINESS</code> è selezionabile in <code>/run</code> come operazione indipendente e autonoma (UC21.1, UC41), senza richiedere la co-selezione di <code>CHANGELOG_TECHNICAL</code> . L'Orchestratore genera il changelog tecnico come primo passo interno della stessa Task e la sospende (<code>pendingInput.kind === 'BUSINESS_CONFIRMATION'</code>) finché l'utente non conferma esplicitamente dal report intermedio (UC26.2.5), riusando il medesimo meccanismo già adottato per lo Sprint ID.	§ 3.6.1, § 19.4

(continua)

Tabella 16 (segue dalla pagina precedente)

Requisito	Descrizione Sintetica	Implementazione Frontend	Rif.
RF.46	Avanzamento stato esecuzione	Sottoscrizione WebSocket agli eventi <code>task.progress</code> e <code>task.updated</code> , aggiornamento reattivo nella Dashboard tramite <code>tasksStore</code> .	§ 19.4
RF.48	Isolamento dei fallimenti in esecuzione	L'errore di una singola Task attiva il Componente: Stato d'errore limitato al solo elemento in Dashboard, le altre Task proseguono l'aggiornamento grafico.	§ 8.3
RF.98–RF.101	Richiesta input umano a metà esecuzione	Gestione dello stato <code>pendingInput</code> interno allo stato <code>RUNNING</code> ; l'UI espande un'area modale <i>inline</i> direttamente sull'elemento della lista. In caso di taglio budget, l'interfaccia non ostacola un approccio automatizzato stub.	§ 3.6, § 12
RF.73	Esportazione report PDF	Consumo del file tramite streaming binario su rete same-origin senza redirect esterni; indicatore di caricamento globale per la durata del download.	§ 7
RF.63, RS.3	Divieto scrittura autonoma e accesso PR	Coerentemente al vincolo RS.3, il frontend non espone comandi di approvazione interna o "Push" verso Git: sostituisce la <i>diff view</i> primaria con un Collegamento Pull Request delegando l'azione umana alla piattaforma GitHub.	§ 5.3
RS.2	Token e credenziali protetti	Il JWT di sessione non viene mai scritto nel <code>localStorage</code> per mitigare l'estrazione via attacchi XSS; conservazione in sola memoria volatile.	§ 6.2

22.2 Requisiti mappati a livello di Flussi e Wireframe

Per evitare estrema ridondanza documentale, i requisiti puramente funzionali relativi alla presenza di specifici campi di input, pulsanti, visualizzazione formattata dei report o logiche standard di routing (es. RF.1–RF.3, RF.15–RF.35, RF.49–RF.64) non sono elencati singolarmente in questa tabella. La loro soddisfazione è garantita strutturalmente e visivamente all'interno dei Flussi Utente (§ 3) e dei Wireframe descrittivi (§ 4), in cui ogni elemento grafico o schermata cita esplicitamente il requisito funzionale (RF) e il caso d'uso (UC) di origine.

22.3 Requisiti Fuori Perimetro Frontend

I seguenti requisiti non compaiono nel tracciamento poiché la loro soddisfazione dipende interamente dall'infrastruttura AWS, dal backend Node.js o dal core degli agenti Python, e non richiedono alcuna implementazione tecnica specifica nel codice sorgente del Frontend:

- **RV.1, RV.10–RV.15, RV.21:** Vincoli tecnologici e architetturali di backend, database, agenti e infrastruttura AWS.

- **RF.82–RF.97, RF.104:** Logica di business per l'individuazione di vulnerabilità, documentazione del codice, scraping dei ticket e traduzione da parte dei modelli LLM.
- **RS.1, RS.5:** Sicurezza crittografica degli hash delle password nel database e isolamento e data-residency sul modello LLM.
- **RQ.1–RQ.8:** Copertura test di codice sorgente backend/agenti, complessità ciclomatica, efficienza e accuratezza del modello AI e metriche Flesch Reading Ease.

Parte III

Frontend MVP — Motivazioni e Decisioni Architettureali

Questa parte raccoglie le argomentazioni, i principi guida e le decisioni architetturali (ADR-FE) che hanno portato alla definizione delle interfacce, degli stati e dei flussi descritti nella Parte II.

23 Principi Guida e Vincoli di Progetto

Le scelte architetturali e di design del frontend per questo MVP sono state guidate da un set di vincoli applicati trasversalmente all'intero documento. Invece di ribadire queste giustificazioni in ogni singola schermata o flusso, vengono dichiarate qui come fondamento delle decisioni successive.

- **Vincolo Tempo/Risorse (Pragmatismo e Riutilizzo):** Il team di sviluppo è composto da sei sviluppatori non a regime pieno, con una finestra di tempo di meno di due settimane per l'implementazione dell'MVP. Questo vincolo ha imposto il riutilizzo sistematico di ogni componente, scelta di libreria e pattern architettonico già validato nella progettazione del Proof of Concept (stack React, Vite, TanStack Router, Zustand), estendendo e riprogettando solo le parti scritte originariamente per validare l'integrazione tecnologica ma non allineate ai requisiti.
- **Feedback immediato e reattività (RQ.10):** Il tempo di risposta percepito dell'interfaccia non deve superare i 2 secondi tra l'azione dell'utente e il feedback visivo. Poiché diverse operazioni dipendono da chiamate asincrone o tempi di latenza di rete (es. attesa dell'avvio degli agenti o esportazione PDF in streaming dal backend), il frontend deve sempre fornire indicatori di stato (es. *In corso*, *Esito positivo/negativo*) senza mai bloccare l'interfaccia in attesa della risoluzione.
- **Coerenza sopra varietà (RQ.11):** Un nuovo utente deve poter apprendere il sistema in 30 minuti o meno. Si predilige un numero ridotto di componenti condivisi e riusati ovunque (ad esempio, un unico componente per gli stati d'errore o per i moduli di validazione inline) rispetto a varianti specifiche create per singola schermata. Questo riduce drasticamente sia il carico cognitivo per l'utente, sia i tempi di sviluppo.
- **Interfaccia Desktop-First:** Code Guardian è uno strumento concepito per essere utilizzato da Sviluppatori, Security Auditor e Project Manager da postazione fissa durante il normale flusso di lavoro. L'Analisi dei Requisiti non pone alcun vincolo di supporto mobile; pertanto, il layout è progettato esclusivamente per viewport desktop, evitando di disperdere il budget di tempo nello sviluppo di versioni *responsive* non richieste per l'MVP.

Per offrire una visione immediata di come questi principi si traducano in scelte pratiche, la Tabella 17 mappa i vincoli principali sulle conseguenti direttive di progettazione del frontend.

Tabella 17: Sintesi dei vincoli di progetto e relativo impatto sulla progettazione Frontend.

Vincolo / Principio	Impatto Architettuale e UX
Tempi di sviluppo ristretti (Team ridotto)	Adozione di librerie pronte per le primitive di stile (Tailwind CSS, shadcn/ui) anziché creare un design system da zero. Adozione totale del contratto API REST e WebSocket già stabilito dal PoC.
Natura desktop dello strumento	Layout progettato a pagina intera, ottimizzando la densità informativa per le liste strutturate (es. report Agente Security) e l'albero dei file, senza logiche di impaginazione mobile.
Reattività (Soglia 2 secondi, RQ.10)	Nessuna rotta protetta effettua chiamate di rete bloccanti per verificare i permessi durante la navigazione: lo stato di autenticazione e ruolo è letto in modo sincrono da un <code>sessionStore</code> in memoria.
Semplicità di apprendimento (RQ.11)	Filtraggio a monte delle azioni: l'utente vede esclusivamente le operazioni che il suo ruolo operativo lo abilita a eseguire, eliminando la frustrazione visiva di elementi permanentemente disabilitati.

24 Decisioni Architettureali (ADR-FE) e Motivazioni

Di seguito vengono espanse e contestualizzate le decisioni architettureali cardine del frontend (ADR-FE), raggruppate per dominio di competenza. L'inventariazione di queste decisioni chiarisce il razionale dietro le specifiche descritte nella Parte II.

24.1 Identità, Sessione e Ruoli

- **ADR-FE-2 — Filtraggio delle operazioni disponibili per ruolo:** La schermata di avvio mostra esclusivamente le operazioni permesse al ruolo attivo dell'utente, omettendo del tutto quelle disabilitate. Un elenco pre-filtrato riduce il carico cognitivo, azzera il rischio di interazioni non valide e soddisfa il requisito di apprendibilità (RQ.11). La sicurezza resta comunque garantita dal backend, che verifica il token a ogni richiesta.
- **ADR-FE-5 — Persistenza del JWT in sola memoria:** Il token di sessione viene conservato esclusivamente nello stato dell'applicazione (`sessionStore`) e mai nel `localStorage` o in cookie non-`httpOnly`. Questo minimizza la superficie di attacco XSS. Si accetta il compromesso UX che una ricarica completa della pagina (F5) comporti la perdita della sessione e richieda un nuovo login. Questa scelta è complementare, ma indipendente, dal rischio accettato lato infrastruttura sul trasporto in chiaro tra CloudFront e ALB (§ 40).
- **ADR-FE-6 — Changelog di business come Task singola con checkpoint interno:** Coerentemente con la preconditione di UC41 (selezione di “changelog tecnico”

o “changelog di business”), il frontend tratta `CHANGELOG_BUSINESS` come un’unica Task che incorpora al proprio interno la generazione tecnica come fase preliminare, riusando il medesimo meccanismo di sospensione già adottato per l’inserimento dello Sprint ID (`pendingInput`, § 19.4). Questa scelta evita di introdurre un secondo concetto di “Task bloccante” nel `tasksStore` e mantiene un solo elemento in Dashboard per l’intero ciclo di vita dell’operazione, in linea con il principio di coerenza sopra varietà (§ 10).

24.2 Integrazioni e Credenziali

- **ADR-FE-3 — Credenziale Task Management unificata con GitHub (copre RF.11 e RF.12):** L’AdR modella concettualmente due credenziali distinte — accesso al codice (RF.11) e accesso al Sistema di Task Management (RF.12). Tuttavia, poiché nell’MVP il Sistema di Task Management è GitHub Issues, le due credenziali convergono sullo stesso token: si è deciso di non richiedere all’utente di inserirlo due volte. Questo scostamento da RF.12 è intenzionale: GitHub Issues è l’equivalente operativo del “Sistema di Task Management” richiesto dall’AdR per l’MVP, e un unico token con scope `repo` copre entrambe le funzioni. La schermata dei Servizi Connessi è stata progettata come lista generica (con un solo elemento “GitHub” per ora) affinché l’aggiunta futura di un provider diverso (es. Jira) richieda solo un nuovo elemento nella lista, senza alcuna riprogettazione della pagina o del database.

24.3 Stack, Design System e Report

- **ADR-FE-4 — Adozione di Tailwind CSS e shadcn/ui:** In assenza di un design system preesistente, la necessità di sviluppare l’intera interfaccia in meno di due settimane ha reso impraticabile la scrittura di stili CSS personalizzati da zero. Tailwind CSS garantisce coerenza e velocità tramite classi di utilità, mentre shadcn/ui fornisce primitive accessibili (focus, ruoli ARIA, navigazione da tastiera) già pronte. La scelta è puramente di tempo-su-mercato (Time-to-Market) e non vincola la futura identità visiva del prodotto, essendo i token incapsulati in variabili di tema.
- **ADR-FE-1 — Il blocco Proposal privilegia il link alla Pull Request:** Nel PoC (Parte I), la proposta dell’Agente Docs era visualizzata solo come un blocco diff isolato. Coerentemente con le decisioni di Progettazione AWS (e nello specifico con **ADR-AWS-10**, § 39, che delega al backend l’apertura automatica della PR per rispettare il vincolo RS.3), il frontend dell’MVP promuove il link alla Pull Request come elemento primario dell’interfaccia. L’anteprima del diff rimane disponibile come elemento secondario collapsabile. Questo ribadisce che l’applicazione effettiva della modifica al repository resta un atto umano condotto esternamente al sistema, su GitHub.

25 Compromessi e Rischi Accettati

Il rispetto del vincolo temporale per lo sviluppo dell’MVP ha reso necessario bilanciare la completezza della *User Experience* (UX) con il pragmatismo del rilascio rapido. La Tabella 18 consolida i compromessi accettati a livello di frontend.

Tabella 18: Matrice dei Compromessi e Rischi Accettati per il Frontend MVP.

Risorsa / Scelta	Compromesso UX/Architettuale	Rischio Accettato
Design System Desktop-First	Nessuna media query o layout alternativo è stato implementato per gli schermi degli smartphone. L'interfaccia assume una larghezza minima tipica dei monitor da postazione di lavoro.	L'applicazione risulta difficilmente navigabile da dispositivi mobili (sovrapposizioni, tabelle tagliate). Rischio accettato in base ai casi d'uso tipici definiti nell'AdR.
Autenticazione (ADR-FE-5) Volatile	Conservazione del token in memoria (variabile di stato) per proteggerlo da vettori di attacco XSS comuni sul <code>localStorage</code> .	Una ricarica accidentale della scheda del browser chiude la sessione attiva, forzando l'utente a ripetere il login (peggioramento della <i>Quality of Life</i>).
Assenza di Retry Automatici Client-Side	Se una chiamata REST (es. configurazione del contesto, avvio task) fallisce per cause di rete, il frontend mostra un errore e richiede all'utente un'azione manuale (es. click su "Riprova"). L'unica eccezione automatizzata è la riconnessione del WebSocket.	Potenziata frustrazione dell'utente in caso di microdisconnessioni, poiché deve reinviare attivamente il modulo. Evita tuttavia la complessità logica di gestire <i>idempotency keys</i> lato frontend in questa fase.
Validazioni Sincrone non esaustive	Le validazioni sui moduli (es. robustezza password, sintassi URL Git) forniscono un <i>feedback</i> rapido ma non duplicano la logica di business o i controlli di sicurezza completi del backend.	Un utente malintenzionato può bypassare la UI e inviare payload non validi all'API. Accettato perché il frontend non è considerato, per definizione, un confine di sicurezza (nessuna protezione WAF attiva, § 26.6).
Interattività Changelog Agente	L'MVP implementa la richiesta di input a metà esecuzione tramite eventi WS (<code>task.inputRequired</code>) e <code>POST /tasks/:id/input</code> . Essendo un rischio architetturale alto non validato dal PoC, in caso di esaurimento budget si adotterà il fallback al comportamento del PoC (decisione autonoma dell'agente e stub), sacrificando temporaneamente i requisiti RF.98–RF.102.	Parziale scostamento dall'AdR se il fallback si rende necessario, con l'agente che processerà tutti i task indiscriminatamente senza chiedere l'intervento umano per l'inserimento dello Sprint ID o l'esclusione.

26 Vincoli Ereditati dall'Infrastruttura AWS

Le sezioni della Specifica Operativa (Parte II) assumono come valide alcune proprietà fondamentali del sistema di esecuzione (ad esempio l'assenza di policy CORS o l'uso di URL relativi). Questa sezione rende esplicito il legame tra tali assunzioni e le decisioni prese nella Parte IV (Progettazione Infrastruttura AWS). Raccogliere questi vincoli in un unico punto garantisce che un eventuale cambiamento futuro dell'infrastruttura comporti una revisione mirata di questa sola sezione, evitando disallineamenti silenti nel resto della progettazione.

26.1 Origine unica e assenza di policy CORS

L'infrastruttura AWS pone il frontend (ospitato su S3) e il backend (gestito tramite Application Load Balancer) dietro un'unica distribuzione CloudFront condivisa. Il comportamento di routing instrada le richieste ai percorsi API (`/api/*` e `/socket.io/*`) verso il backend, e tutto il resto verso l'artefatto statico del frontend. Poiché frontend e backend condividono lo stesso dominio agli occhi del browser, nessuna richiesta effettuata dall'interfaccia è considerata *cross-origin*. Di conseguenza, il frontend non richiede alcuna configurazione, gestione o negoziazione di policy CORS, eliminando alla radice un'intera classe di potenziali anomalie di rete.

26.2 Routing lato client e Custom Error Response

L'applicazione frontend gestisce il routing interamente lato client (tramite TanStack Router). Una richiesta diretta del browser a un percorso specifico (es. `/reports/abc123`) raggiungerebbe S3, che non possedendo tale file restituirebbe un errore 403/404. L'infrastruttura risolve questo comportamento configurando su CloudFront una *Custom Error Response* che intercetta questi errori e restituisce il file `index.html` con stato HTTP 200. Per lo sviluppo frontend, questo vincolo impone che ogni rotta protetta o pubblica sia in grado di auto-risolversi e di leggere lo stato di autenticazione in modo sincrono direttamente al caricamento (come garantito dalle guardie di rotta lette dal `sessionStore`), permettendo il corretto funzionamento dell'accesso diretto o della ricarica della pagina tramite browser.

26.3 Configurazione a build-time per risorse statiche

A differenza del backend, che riceve la configurazione a *runtime* tramite variabili d'ambiente iniettate nel container, il frontend è distribuito come artefatto statico puro. Variabili come l'indirizzo base delle API (`VITE_API_BASE_URL`) vengono risolte in fase di compilazione (*build-time*) e integrate irreversibilmente nel pacchetto JavaScript. Per supportare deploy multi-ambiente senza dover ricompilare artefatti diversi, l'infrastruttura impone l'uso di percorsi relativi (es. `/api/v1`). Il frontend non deve quindi mai concatenare domini assoluti nelle chiamate ai servizi interni.

26.4 Requisiti per connessioni WebSocket

Il canale realtime bidirezionale necessario per monitorare l'avanzamento delle Task è supportato nativamente dall'infrastruttura tramite specifiche policy CloudFront (che inoltrano gli header `Upgrade` e `Connection`) e configurazioni dell'Application Load Balancer (che implementa *stickiness* e *idle timeout* estesi). Questo solleva il frontend dal dover implementare logiche complesse di *failover* o *long-polling* di fallback: una semplice logica di riconnessione automatica del *socket* è sufficiente a garantire la resilienza, in quanto l'infrastruttura stessa mantiene la connessione aperta durante l'esecuzione di agenti a lunga durata.

26.5 Streaming dei file ed Esportazione PDF

Il bucket S3 designato per ospitare gli artefatti applicativi (come i report PDF generati) è strettamente privato e accessibile unicamente dai container del backend tramite VPC Endpoint. Questo vincolo di sicurezza impedisce categoricamente al frontend di tentare l'accesso diretto ai file tramite URL pubbliche o di generare *presigned URL*. Il comando di esportazione del PDF nel frontend deve consumare un endpoint REST del backend che scarica l'oggetto da S3 e lo trasmette in *streaming* nel corpo della risposta HTTP same-origin. L'indicatore di caricamento sulla UI deve coprire l'intera durata di questo transito.

26.6 Gestione degli errori di rete e Sicurezza Trasversale

L'infrastruttura AWS accetta determinati rischi a fronte di un forte contenimento dei costi e dei tempi di rilascio dell'MVP. Questi si riflettono direttamente sulla gestione errori e sulla sicurezza del frontend:

- **SPOF del NAT Gateway:** L'uso di un singolo NAT Gateway per far dialogare il backend con GitHub introduce un punto singolo di fallimento di rete. Se questa connettività si interrompe, l'errore che l'utente visualizza nel frontend ricade nella famiglia degli errori generici di sistema, e non deve innescare falsi avvisi di "Credenziale GitHub non valida".
- **Trasporto in chiaro interno:** Il perimetro MVP esclude l'adozione di un dominio custom, costringendo l'Application Load Balancer a operare senza certificato TLS pubblico (ricevendo traffico da CloudFront in chiaro). Questo vincolo infrastrutturale giustifica pienamente la decisione architetturale del frontend (ADR-FE-5) di mantenere il JSON Web Token esclusivamente in memoria, evitando ulteriori esposizioni sul client.
- **Assenza di Web Application Firewall (WAF):** In mancanza di un WAF perimetrale, le validazioni condotte dal frontend (formati, lunghezze, correttezza sintattica) sono classificate puramente come ausili per la *User Experience* (UX). Non costituiscono una barriera di sicurezza, e non si deve assumere che il backend sia esentato dal ri-validare strettamente ogni singolo payload ricevuto.

Tabella 19: Riepilogo dei vincoli tecnici imposti al frontend dalle decisioni infrastrutturali.

Vincolo per il frontend	Origine Architetturale	Impatto sullo Sviluppo
Nessuna gestione delle policy CORS	Distribuzione CloudFront condivisa per routing traffico statico (S3) e dinamico (ALB).	Tutte le chiamate API e WebSocket risultano same-origin. Nessun preflight o gestione header CORS necessaria lato client.
Risoluzione rotta garantita ad ogni punto di ingresso	Custom Error Response configurata su CloudFront (403/404 da S3 reindirizzati a <code>index.html</code>).	Le guardie di rotta devono poter validare lo stato (<code>isAuthenticated</code> , <code>credentialsStatus</code>) immediatamente al caricamento diretto di un URL profondo.
Path API relativo	Artefatti su S3 immutabili a <i>runtime</i> ; <code>VITE_API_BASE_URL</code> compilata a <i>build-time</i> .	Il client usa esclusivamente prefissi relativi (es. <code>/api/v1</code>) per non legarsi a un dominio statico, agevolando le pipeline CI/CD.
Autenticazione WebSocket all'handshake	CloudFront Origin Request Policy (Forward All) e ALB con <i>stickiness</i> abilitata.	La libreria Socket.IO può autenticarsi al momento dell'apertura inviando header custom senza dipendere da riconessioni complesse o long-polling.
Download dei PDF interamente mediato	Bucket S3 degli artefatti privato e inaccessibile da Internet pubblico (solo via VPC Endpoint).	I PDF non vengono mai referenziati o aperti con URL esterne. Il frontend attende il completamento dello streaming binario dal backend.
Validazioni client-side a puro scopo UX	Assenza di Web Application Firewall nel perimetro MVP.	Il frontend evita chiamate di rete inutili bloccando input errati (es. email malformate), ma la validazione formale e la sicurezza restano in capo al backend.

Parte IV

Infrastruttura AWS — Manuale Operativo

Questa parte contiene esclusivamente i parametri di configurazione, le tabelle di mappatura e le istruzioni operative per il rilascio. Tutte le argomentazioni, le motivazioni delle scelte e le assunzioni di rischio sono documentate nella Parte V.

27 Guida Operativa di Rilascio (Checklist)

Nota Implementativa. Questa sequenza descrive le **dipendenze logiche** tra risorse, non l'ordine di scrittura del codice CDK. CDK/CloudFormation crea

in parallelo tutto ciò che non ha dipendenze esplicite; se una risorsa referencia l'oggetto/ARN di un'altra, la dipendenza è inferita automaticamente, altrimenti va dichiarata con `.node.addDependency()`.

Questa sequenza riflette le dipendenze tecniche tra le risorse. Le fasi indipendenti (es. richiesta Bedrock e Rete) possono procedere in parallelo.

27.1 Attività da avviare il Giorno 1

Queste richieste hanno tempi di approvazione AWS non controllabili dal team:

- **Amazon Bedrock:** Richiedere l'accesso ai modelli nella regione `eu-south-1`. *Verificare in anticipo l'assenza di Service Control Policy (SCP) organizzative bloccanti.*

27.2 Rete e Sicurezza

1. Creazione VPC (10.0.0.0/16) su due AZ, con `enableDnsSupport` e `enableDnsHostnames` attivi.
2. Creazione 4 subnet (2 pubbliche, 2 private), 2 route table e Internet Gateway.
3. Creazione NAT Gateway singolo in `public-a`.

Attenzione. Il traffico da `private-b` verso il NAT Gateway in `public-a` attraversa un confine di Availability Zone: AWS addebita i costi di **trasferimento dati cross-AZ** in aggiunta al costo orario/per-GB del NAT stesso. Non è solo un rischio di disponibilità (Parte V) ma una voce di costo da includere nella stima con AWS Pricing Calculator (§ 10.2).

4. Creazione VPC Endpoints con Private DNS abilitato.

Nota Implementativa. Poiché i task ECS girano in subnet private senza IP pubblico, il pull delle immagini da ECR avviene esclusivamente tramite i VPC Endpoint `api/dkr` configurati in questo stesso punto: non è necessaria alcuna route verso Internet per il deploy dei container.

5. Creazione Security Group rispettando l'ordine di dipendenza: prima `sg-alb`, `sg-atlas`, `sg-redis`, `sg-vpce`; successivamente `sg-backend` e `sg-agents`.

Attenzione. Le regole tra `sg-backend` e `sg-agents` sono **reciproche** (backend → agents in egress, agents → backend in ingress, e viceversa). Definirle inline nel costruttore del `SecurityGroup` genera un errore di **dipendenza circolare** in CloudFormation. Istanziare i due SG vuoti e aggiungere le regole *dopo* con `addIngressRule()` / `addEgressRule()`:

```
// 1. Istanziare i SG vuoti
const sgBackend = new ec2.SecurityGroup(this, 'SgBackend', { vpc });
const sgAgents  = new ec2.SecurityGroup(this, 'SgAgents',  { vpc });

// 2. Aggiungere le regole reciproche DOPO la creazione
sgBackend.addEgressRule(sgAgents, ec2.Port.tcp(8000), 'To agents');
sgAgents.addIngressRule(sgBackend, ec2.Port.tcp(8000), 'From backend');

sgAgents.addEgressRule(sgBackend, ec2.Port.tcp(3000), 'To backend');
sgBackend.addIngressRule(sgAgents, ec2.Port.tcp(3000), 'From agents');
```

Listing 7: Pattern corretto per evitare dipendenza circolare tra SG

27.3 Segreti e Chiavi

1. Creazione chiave KMS condivisa (Parte V: Compromessi e Rischi).

Attenzione. La cifratura a riposo di **ElastiCache** deve essere specificata *al momento della creazione*: non è possibile abilitarla in un secondo momento su un nodo esistente. Se dimenticata, l'unica soluzione è ricreare la risorsa da zero. Verificare la *key policy* della CMK: deve concedere esplicitamente l'uso ai Task Execution Role di **backend** e **agents**, altrimenti anche con permessi IAM corretti le chiamate falliscono con `KMS.AccessDeniedException`.

2. Creazione voci in Secrets Manager e Parameter Store con valori effettivi (vietati i placeholder in ambiente condiviso).

Attenzione. Se un segreto in Secrets Manager è salvato come oggetto JSON, l'ARN referenziato nella Task Definition deve specificare la chiave da estrarre con la sintassi `<secret-arn>:<jsonKey>::`, altrimenti ECS inietta l'intero blob JSON come valore della variabile d'ambiente:

```
containerDefinitions: [{
  secrets: [{
    name: 'MONGO_URI',
    valueFrom: '${secret.secretArn}:MONGO_URI::', // nota il suffisso
  }],
}]
```

Listing 8: Estrazione di una singola chiave da un secret JSON

3. Creazione ruoli IAM separati: Task Execution Role e Task Role per **backend** e **agents**.

Nota Implementativa. Chi arriva da Docker Compose spesso confonde i due ruoli IAM di ECS. La tabella seguente chiarisce la differenza:

Ruolo	Usato da	Permessi tipici
Task Execution Role	L'agente ECS, <i>prima</i> che il container parta	Pull immagine da ECR, lettura segreti da iniettare come env var, scrittura log su CloudWatch
Task Role	Il codice applicativo, <i>dentro</i> il container in esecuzione	Chiamate API AWS runtime: bedrock:InvokeModel , accesso S3, ecc.

Attenzione. Assegnare i permessi Bedrock/S3 al ruolo sbagliato (es. Execution Role invece di Task Role) non impedisce l'avvio del container: la chiamata AWS fallisce a runtime con **AccessDenied**, un errore silenzioso e difficile da diagnosticare se non si sa dove guardare.

27.4 Dati e Caching

1. Provisioning cluster **MongoDB Atlas** (tier M10, regione AWS eu-south-1) tramite console/API Atlas o Atlas Terraform/CDK provider.
2. Configurazione **AWS PrivateLink** verso Atlas: creazione del Private Endpoint dal progetto Atlas e dell'Interface VPC Endpoint corrispondente lato AWS (endpoint service fornito da Atlas) su **private-a/private-b**.

3. Creazione cluster ElastiCache Redis (nodo singolo), subnet group privato e cifratura KMS.
4. *Azione Dev:* Recuperare da Atlas la *connection string del Private Endpoint*, restringere la *Network Access List* al solo Private Endpoint e verificare la connettività dal task **backend**. Nessuno spike di compatibilità richiesto: Atlas è MongoDB nativo.

Nota Implementativa. A differenza di DocumentDB, Atlas è MongoDB nativo: non esistono divergenze di compatibilità sul driver Mongoose (operatori di aggregazione, transazioni multi-documento, tipi di indice e upsert su array nidificati si comportano come in un MongoDB self-hosted). L'attività di setup si sposta quindi dalla validazione applicativa alla **configurazione della connettività PrivateLink** e della *Network Access List* lato Atlas.

27.5 Immagini e Registro (ECR)

1. Creazione repository ECR `codeguardian/backend` e `codeguardian/agents`.
2. Attivazione scansione vulnerabilità e Lifecycle Policy.
3. Creazione ruolo IAM CI/CD per push immagini.
4. Primo push manuale per test del flusso.

27.6 Calcolo e Orchestrazione (ECS)

1. Creazione cluster ECS (Capacity provider: `FARGATE`).
2. Creazione Task Definition (`backend`, `agents`) con iniezione segreti via ARN.
3. Creazione namespace Cloud Map (`codeguardian.local`).
4. Creazione servizi ECS (Desired count: 1, Rete: `awsvpc`, Service Discovery abilitata).

Attenzione. Se il container `backend` impiega alcuni secondi ad avviarsi (connessione a MongoDB Atlas/Redis, caricamento moduli NestJS), l'ALB può marcarlo `unhealthy` e forzarne il riavvio prima ancora che sia pronto, generando un **restart-loop**. Impostare esplicitamente `healthCheckGracePeriod` sul servizio ECS:

```
const service = new ecs.FargateService(this, 'BackendService', {
  cluster,
  taskDefinition,
  desiredCount: 1,
  healthCheckGracePeriod: cdk.Duration.seconds(60),
});
```

Listing 9: Grace period per evitare restart-loop in avvio

5. Creazione Application Load Balancer (ALB), target group (ip su porta 3000) e listener (HTTP 80).

27.7 Frontend e Distribuzione

1. Creazione bucket S3 statico (privato, Origin Access Control).

Attenzione. “Origin Access Control” da solo **non basta**: va aggiunta esplicitamente una *bucket policy* che conceda `s3:GetObject` al principal `cloudfront.amazonaws.com`, con una condizione su `AWS:SourceArn` che punta alla distribuzione. Dimenticarlo è la causa più comune di 403 Forbidden su tutto il sito dopo il deploy.

```
{
  "Effect": "Allow",
  "Principal": { "Service": "cloudfront.amazonaws.com" },
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::<bucket-name>/*",
  "Condition": {
    "StringEquals": {
      "AWS:SourceArn": "arn:aws:cloudfront::<account-id>:distribution/<dist-id>"
    }
  }
}
```

Listing 10: Bucket policy minima per accesso via OAC

2. Creazione distribuzione CloudFront condivisa (Behavior `/*` verso S3, Behavior `/api/*` e `/socket.io/*` verso ALB).

Nota Implementativa. Una distribuzione CloudFront impiega tipicamente **15–20 minuti** per la propagazione iniziale completa (pochi minuti per le modifiche successive). Non vedere il sito funzionante subito dopo il primo deploy è comportamento atteso, non un errore di configurazione.

3. Configurazione Custom Error Response (403/404 → `index.html`).
4. Build di Vite con `VITE_API_BASE_URL` relativo, caricamento e invalidazione cache.

27.8 Storage Applicativo

1. Creazione bucket S3 per export report (privato, policy limitata al Task Role del backend, Lifecycle 30 giorni).

27.9 Osservabilità e Budget

1. Creazione Log Group CloudWatch e resource policy per Bedrock invocation logging.

Attenzione. Un CloudWatch Log Group ha di default retention **infinita** (*Never Expire*), con costo di storage che cresce indefinitamente. Impostare una retention esplicita su ogni Log Group creato:

```
const logGroup = new logs.LogGroup(this, 'BackendLogs', {
  retention: logs.RetentionDays.TWO_WEEKS,
});
```

Listing 11: Retention esplicita sul Log Group

2. Abilitazione Container Insights su ECS.
3. Creazione Argomento SNS (`codeguardian-alerts`) e sottoscrizione email del team.

Attenzione. La sottoscrizione email a SNS richiede un **click di conferma** nella mail ricevuta. Se nessuno lo effettua, gli allarmi non verranno mai recapitati e il problema resta invisibile finché non serve davvero. Verificare lo stato **Confirmed** della subscription in console SNS prima di considerare questo punto completato.

4. Creazione allarmi operativi CloudWatch verso SNS.
5. Calcolo stima tramite AWS Pricing Calculator e creazione allarmi AWS Budgets verso SNS.

28 Perimetro e Scope

28.1 Servizi AWS per Area (MVP)

Tabella 20: Mappa dei servizi AWS dell'MVP.

Area	Servizi AWS Configurati
Rete e Sicurezza	VPC dedicata (2 AZ), Subnet pubbliche/private, 1 NAT Gateway, VPC Endpoints, Security Group dedicati.
Compute	Amazon ECS su Fargate, Amazon ECR, ALB, AWS Cloud Map (Service Discovery).
Dati e Caching	MongoDB Atlas (cluster M10, via AWS PrivateLink), Amazon ElastiCache Redis (1 nodo).
LLM	Amazon Bedrock (Famiglia Qwen3 in <code>eu-south-1</code>).
Storage e Frontend	Amazon S3 (2 bucket: artefatti e build React), Amazon CloudFront (distribuzione condivisa).
Segreti	AWS Secrets Manager, Parameter Store, 1 Chiave AWS KMS.
Osservabilità & Costi	Amazon CloudWatch Logs/Insights, AWS Budgets, SNS.

28.2 Fuori Perimetro

Le seguenti architetture **non** fanno parte dell'MVP (Parte V: Compromessi e Rischi Accettati):

- Auto-scaling dinamico per ECS.
- Alta disponibilità avanzata / Multi-Region (Single NAT e nodo Redis singolo accettati; il cluster Atlas M10 è già multi-AZ nella regione).
- Dominio Custom (Route53) e relativi certificati validati.
- AWS WAF (Web Application Firewall).

28.3 Mappatura Componenti PoC → AWS

L'unità di deploy (immagine Docker) e il codice applicativo **non subiscono modifiche** infrastrutturali (Parte V: Principio di Migrazione).

Tabella 21: Mappatura operativa PoC verso AWS.

Componente PoC	Target AWS	Azione / Vincolo per i Dev
Container backend e agents	ECS Fargate	Sostituire variabili d'ambiente. Vincolo: i prompt non possono usare bind mount; richiedono bake-in e nuova build dell'immagine per le modifiche (§ 41).
Frontend Vite (Dev Server)	S3 + CloudFront	Modificare configurazione Vite: compilazione statica con URL base relativo. Nessuna gestione CORS necessaria (Parte V: ADR-AWS-9).
Rete Docker (agents:8000)	Cloud Map	URL aggiornato a <code>agents.codeguardian.local</code> .
LLM API Esterna (Dash-Scope)	Amazon Bedrock	Passaggio da chiave API statica a permessi IAM (Task Role).
MongoDB	MongoDB Atlas (via PrivateLink)	Aggiornare <code>MONGO_URI</code> con la <i>connection string</i> del Private Endpoint fornita da Atlas. Nessuna validazione di compatibilità necessaria (MongoDB nativo).
Redis	ElastiCache	Aggiornare <code>REDIS_URL</code> . Doppio uso (coda + cache) mantenuto sulla stessa istanza.
MinIO	Amazon S3	Aggiornare l'endpoint SDK nel backend. Passaggio ad autenticazione tramite Task Role.

29 Rete e Sicurezza (VPC)

29.1 Parametri VPC base

- **Strumento IaC:** AWS CDK (TypeScript).
- **Regione:** eu-south-1 (Milano).
- **CIDR VPC:** 10.0.0.0/16.
- **Availability Zones:** eu-south-1a, eu-south-1b.
- **Impostazioni DNS:** `enableDnsSupport=true`, `enableDnsHostnames=true`.

29.2 Subnet e Routing

Tabella 22: Parametri di Subnetting.

Subnet	CIDR	Tipo	Risorse allocate
public-a	10.0.0.0/24	Pubblica	ALB, NAT Gateway (Rischio Single-NAT, § 40)
private-a	10.0.1.0/24	Privata	ECS, ElastiCache, VPC Endpoints (incl. Atlas PrivateLink)
public-b	10.0.2.0/24	Pubblica	ALB (nodo AZ secondaria)
private-b	10.0.3.0/24	Privata	ECS, ElastiCache, VPC Endpoints (incl. Atlas PrivateLink)

Tabella 23: Route table della VPC e relative associazioni.

Route table	Associazione	Regola
rt-public	public-a, public-b	0.0.0.0/0 → Internet Gateway
rt-private	private-a, private-b	0.0.0.0/0 → NAT Gateway (public-a)

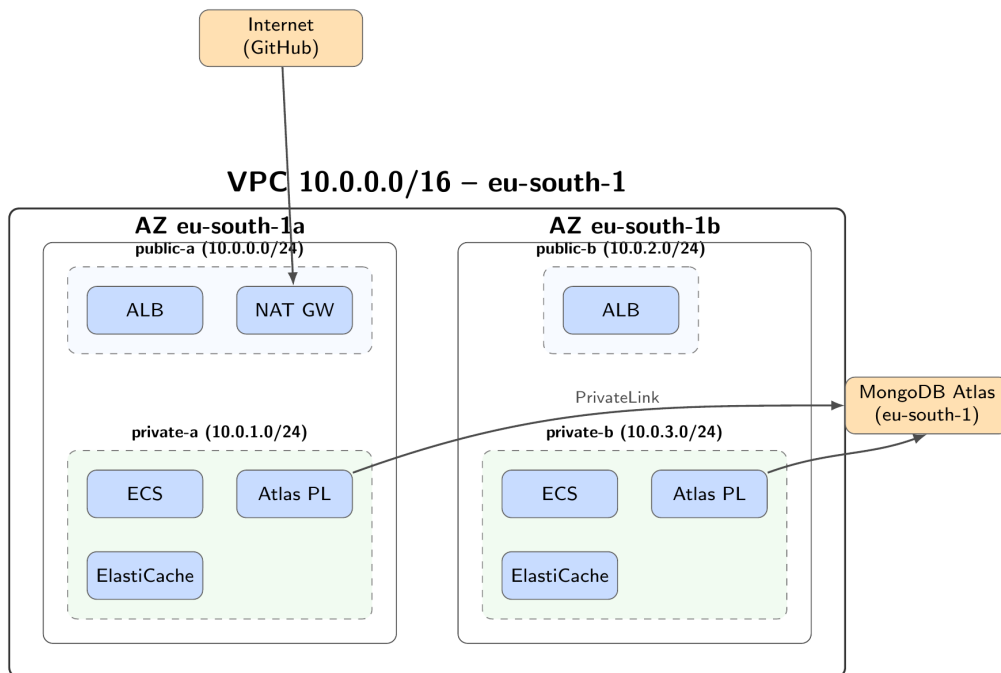


Figura 17: VPC del MVP: due Availability Zone, subnet pubbliche e private, NAT Gateway unico in public-a. Il database MongoDB Atlas è esterno alla VPC e raggiunto via AWS PrivateLink (Interface Endpoint “Atlas PL” nelle subnet private). Gli altri VPC Endpoint verso i servizi AWS interni sono omessi da questo diagramma.

29.3 VPC Endpoints (PrivateLink)

Tutto il traffico AWS interno deve restare nella VPC. Il NAT serve esclusivamente per GitHub. Tutti gli Endpoint “Interface” richiedono l’abilitazione del Private DNS.

Tabella 24: Elenco VPC Endpoints da configurare.

Servizio AWS	Tipo	Note Operative
S3	Gateway	Agisce su Route Table, no Private DNS necessario.
Bedrock Runtime	Interface	Indispensabile per sicurezza Prompt (ADR-AWS-6, § 39).
Secrets Manager	Interface	Interrogato dai task all'avvio.
ECR (api, dkr)	Interface	Pull immagini senza uscita web.
CloudWatch Logs	Interface	Traffico streaming logs continuo.
SSM, SSM Msgs, EC2 Msgs	Interface	Necessari per ECS Exec senza Bastion Host (ADR-AWS-7, § 39).
MongoDB Atlas (PrivateLink)	Interface	Endpoint verso l' <i>endpoint service</i> di Atlas (non <code>com.amazonaws.*</code>). Usare la <i>connection string del Private Endpoint</i> fornita da Atlas per la risoluzione DNS.

Nota Implementativa. Ogni VPC Endpoint di tipo *Interface* viene fatturato per ora **per ogni AZ** in cui è distribuito. Con 7 endpoint Interface (incluso Atlas PrivateLink) su 2 AZ si tratta di 14 ENI fatturati continuativamente, da includere nella stima con AWS Pricing Calculator (§ 10.2) insieme al NAT Gateway.

29.4 Matrice Security Groups (Regole di Rete)

Ogni componente possiede un SG dedicato. Il traffico non in elenco è implicitamente negato.

Tabella 25: Matrice delle Regole Security Group. *L'endpoint Bedrock accetta ingress solo da `sg-agents`.

Security Group	Ingress (In Entrata)	Egress (In Uscita)
<code>sg-alb</code>	TCP 80 dal Prefix List AWS <code>com.amazonaws.global.cloudfront.origin-facing</code>	TCP 3000 verso <code>sg-backend</code>
<code>sg-backend</code>	TCP 3000 da <code>sg-alb</code>	TCP 27017 verso <code>sg-atlas</code> (endpoint Atlas PrivateLink); TCP 6379 verso <code>sg-redis</code> ; TCP 443 verso <code>sg-vpce</code> ; Porta interna verso <code>sg-agents</code> ; TCP 443 verso <code>0.0.0.0/0</code> (Solo NAT/GitHub)
<code>sg-agents</code>	Porta interna da <code>sg-backend</code>	Porta interna verso <code>sg-backend</code> ; TCP 443 verso <code>sg-vpce</code> ; NESUNA regola verso <code>0.0.0.0/0</code> (ADR-AWS-4, § 39)
<code>sg-atlas</code>	TCP 27017 da <code>sg-backend</code>	(Nessuna regola)
<code>sg-redis</code>	TCP 6379 da <code>sg-backend</code>	(Nessuna regola)
<code>sg-vpce</code>	TCP 443 da <code>sg-backend</code> ; TCP 443 da <code>sg-agents</code> *	(Nessuna regola)

Attenzione. Il Security Group `sg-atlas` è associato all'*Interface VPC Endpoint* di

Atlas PrivateLink, non a un'istanza database nella VPC. La porta esatta (27017 o l'eventuale range indicato da Atlas) va confermata dalla *connection string* del *Private Endpoint* generata in fase di pairing e usata come sorgente di verità per le regole di egress di `sg-backend`.

30 Compute e Orchestrazione (ECS su Fargate)

Il deploy dell'applicativo si basa sulle stesse immagini Docker validate nel PoC (Parte I), senza modifiche al codice sorgente (Parte V: Principio di Migrazione).

30.1 Cluster ECS e Task Definition

- **Capacity Provider:** Esclusivamente FARGATE (nessun FARGATE_SPOT).
- **Modalità di Rete:** awsvpc (interfacce dedicate nelle subnet private).
- **Log Driver:** awslogs (inoltre standard output/error a CloudWatch Logs).

Tabella 26: Dimensionamento e configurazione Task ECS per l'MVP.

Servizio ECS	vCPU	Memoria	Desired Count	Configurazioni Task
backend	0.5	1 GB	1 (§ 39)	Health check breve su <code>/health</code> . Vincolo implementativo: la generazione PDF deve usare una libreria di composizione programmatica (pdfkit), non rendering via browser headless; dimensionamento da confermare con test di carico su export concorrenti prima del rilascio (§ 40, Tabella 39).
agents	1.0	2 GB	1 (§ 39)	Valori di base, aggiornabili dopo test di carico su LangGraph e Bedrock.

30.2 Gestione Immagini Container (Amazon ECR)

Tabella 27: Configurazione dei registri ECR.

Parametro	Valore / Configurazione
Repository	codeguardian/backend, codeguardian/agents
Tagging	Uso obbligatorio dello SHA breve del commit (es. <code>:a1b2c3d</code>). Tag <code>latest</code> solo come puntatore di comodo in dev.
Security Scan	Scansione automatica base abilitata al push.
Lifecycle Policy	Mantenere ultime 20 immagini taggate; eliminare immagini untagged vecchie di > 1 giorno.
Permessi IAM Push	Ruolo CI/CD (<code>codeguardian-ci-role</code>).
Permessi IAM Pull	Task Execution Role di ECS.

30.3 Application Load Balancer (ALB)

Nessun dominio custom o certificato TLS è installato sull'ALB (ADR-AWS-8, § 39).

Tabella 28: Configurazione ALB.

Parametro	Valore Operativo
Posizionamento	Subnet pubbliche (<code>public-a</code> , <code>public-b</code>).
Listener	Protocollo HTTP, Porta 80.
Target Group	Tipo <code>ip</code> , Porta 3000, Protocollo HTTP (puntato ai task del backend). Path Health Check: <code>/health</code> .
Idle Timeout	3600 secondi (Necessario per connessioni WebSocket).
Stickiness	Abilitata, cookie <code>AWSALB</code> , durata 3600 secondi.
Deregistration	30 secondi (Ridotto per deploy più rapidi).

30.4 Service Discovery (AWS Cloud Map)

Usata per la comunicazione interna (Backend ↔ Agents).

- **Namespace Privato:** `codeguardian.local` (associato alla VPC).
- **TTL Record DNS:** 10 secondi.

Tabella 29: Record Service Discovery Cloud Map.

Servizio DNS	Porta	Tipo	Scopo
<code>agents.codeguardian.local</code>	8000	A	Chiamato dal backend per avvio run.
<code>backend.codeguardian.local</code>	3000	A	Chiamato dagli agenti per tool-calls (GitHub read-only) e callback di progresso (ADR-AWS-3, § 39).

30.5 Flussi Applicativi End-to-End

Il diagramma di rete (Figura 17) mostra il posizionamento delle risorse, non i flussi dati. La Figura 18 integra le due viste, mostrando chi chiama chi e con quale protocollo.

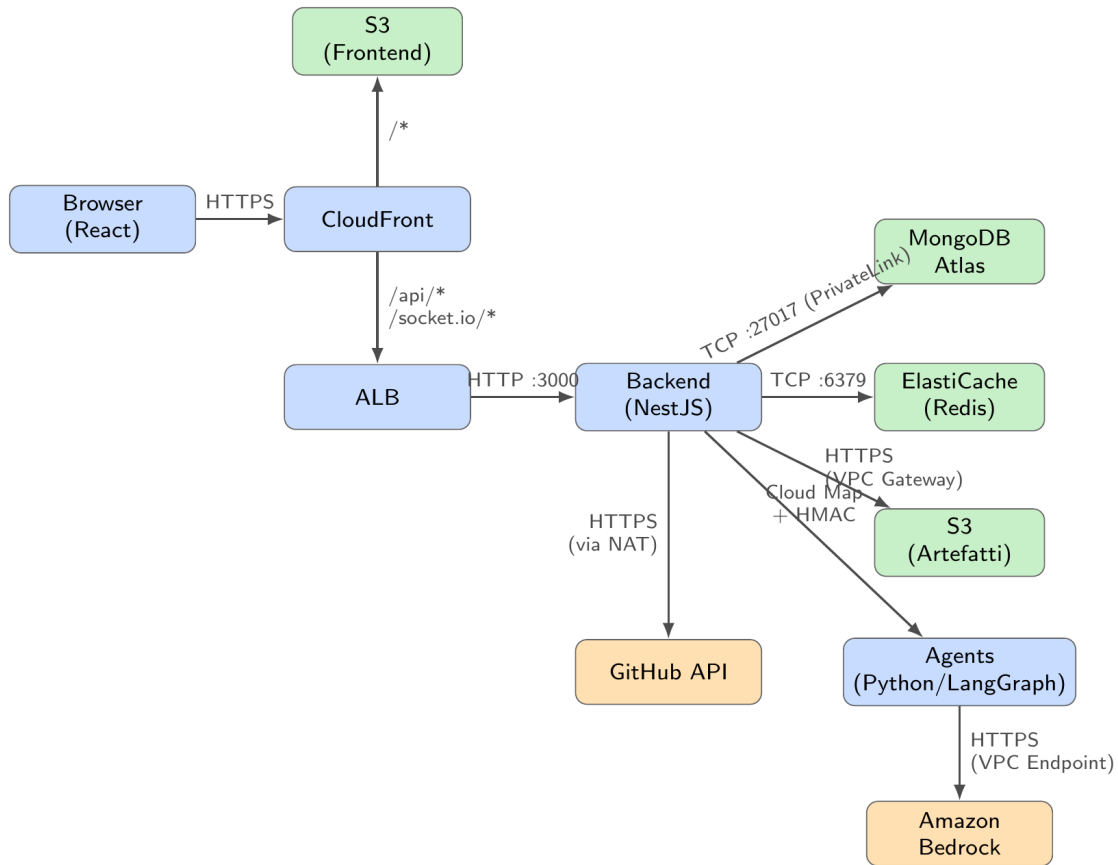


Figura 18: Flussi applicativi principali con protocollo e direzione. Il traffico agents→GitHub non esiste: ogni lettura passa dalla facade del backend (§ 6, ADR-AWS-1).

31 Livello Dati e Caching

31.1 MongoDB Atlas (via AWS PrivateLink)

Sostituisce il container MongoDB del PoC con un cluster **MongoDB Atlas** gestito, deployato su AWS nella regione **eu-south-1** e raggiunto esclusivamente tramite **AWS PrivateLink** (nessun transito su Internet pubblico e nessuna dipendenza dal NAT Gateway per il livello dati).

Tabella 30: Configurazione MongoDB Atlas.

Parametro	Valore Operativo
Tier	M10 dedicato (minimo che abilita PrivateLink).
Topologia	Replica set a 3 nodi distribuiti sulle AZ della regione (alta disponibilità per costruzione, gestita da Atlas).
Regione	AWS eu-south-1 (Milano), per data residency coerente col resto dello stack.
Versione	Ultima major MongoDB supportata da Atlas nella regione.
Connettività	AWS PrivateLink (Interface Endpoint su private-a/private-b); <i>Network Access List</i> Atlas ristretta al Private Endpoint.
Rete e Sicurezza	Porta MongoDB (27017), SG sg-atlas sull'endpoint (Ingresso solo da sg-backend).
Cifratura	A riposo gestita da Atlas (AES-256); BYOK via KMS fuori scope MVP. In transito TLS obbligatoria (CA pubblica).
Backup	Atlas Cloud Backup (snapshot gestiti da Atlas, non da AWS Backup).

Attenzione. Con PrivateLink la connessione deve usare la *connection string* del *Private Endpoint* generata da Atlas (host che risolvono a IP privati), **non** la connection string standard `mongodb+srv://` pubblica: quest'ultima risolverebbe a IP pubblici non instradabili senza NAT, con connessione in timeout senza un errore esplicito. Atlas usa certificati TLS da CA pubblica: **non** serve alcun CA bundle custom (a differenza di DocumentDB, che imponeva il bundle RDS).

```
// MONGO_URI = connection string del PRIVATE ENDPOINT
// (Atlas -> Connect -> Private Endpoint), es.:
// mongodb+srv://<user>:<pwd>@<cluster>-pl-0.<hash>.mongodb.net/<db>?
//   retryWrites=true

mongoose.connect(process.env.MONGO_URI, {
  // TLS implicito con mongodb+srv; nessun tlsCAFile custom (CA pubblica)
})
```

Listing 12: Connessione Mongoose a MongoDB Atlas (Private Endpoint)

Azione Dev: Configurare Private Endpoint e Network Access List su Atlas e verificare la connessione dal task **backend**. Lo schema design della base dati resta di competenza del team backend (RV.21), senza spike di compatibilità.

31.2 Amazon ElastiCache (Redis)

Duplica uso applicativo: coda BullMQ (`bull:*`) e cache GitHub (`repo@sha:*`).

Tabella 31: Configurazione ElastiCache (Redis).

Parametro	Valore Operativo
Topologia	Nodo singolo (No cluster, no repliche) (§ 40).
Subnet Group	Subnet private (private-a , private-b).
Rete e Sicurezza	Porta 6379, SG sg-redis (Ingresso solo da sg-backend).
Cifratura	A riposo abilitata (Stessa chiave KMS), in transito (in-transit encryption) abilitata.

Attenzione. Abilitare **in-transit encryption** su ElastiCache **non è trasparente** lato applicativo: il client (**ioredis** / **node-redis**) deve connettersi specificando esplicitamente l'opzione TLS, altrimenti la connessione fallisce in timeout senza un messaggio di errore chiaro.

```
const redis = new Redis({
  host: process.env.REDIS_HOST,
  port: 6379,
  tls: {}, // richiesto se in-transit encryption e' abilitata
});
```

Listing 13: Connessione ioredis con TLS abilitato

Rate Limiting Applicativo (RQ.8). Oltre agli usi già previsti (coda BullMQ, cache letture GitHub), l'istanza Redis supporta un **rate limiter applicativo** lato backend NestJS con **sliding window**, con chiave per utente (**userId**): il limite è fissato a **60 richieste al minuto per utente** sugli endpoint che invocano l'API GitHub (variabile d'ambiente **RATE_LIMIT_GITHUB_RPM=60**). Questo copre RQ.8 con un controllo architetturale reale, senza affidarsi ai soli limiti di default dell'account Bedrock (insufficienti da soli a soddisfare il requisito in modo verificabile). Non introduce nuovi servizi AWS: riusa l'istanza ElastiCache già provisionata.

32 Integrazione Amazon Bedrock

Tutte le comunicazioni con Bedrock avvengono via VPC Endpoint, garantendo che il codice analizzato non transiti su Internet pubblico (Conformità RS.5, § 39).

32.1 Modelli e Accessi

L'accesso ai modelli va richiesto **immediatamente** tramite console Bedrock (sezione *Model access*) nella regione **eu-south-1**.

Tabella 32: Assegnazione Modelli e Richieste di Accesso Bedrock.

Agente	Modello Selezionato	Priorità di Richiesta
Docs, Changelog	Qwen3-32B (dense)	Alta (Assegnazione primaria).
Security	Qwen3-Coder-30B-A3B-Instruct	Alta (Assegnazione primaria).
N/A (Scalata)	Qwen3-235B-A22B	Media (Richiedere subito come fallback in caso di accuratezza insufficiente in fase di validazione) (§ 39).

Attenzione. Verificare in console Bedrock (sezione *Model access* / *Cross-region inference*) se i modelli Qwen3 richiedono un **Inference Profile ARN** invece dell'ARN

diretto del foundation model per l'invocazione on-demand. Se richiesto, l'ARN nella policy IAM e nel codice applicativo va aggiornato di conseguenza; l'errore tipico in caso di mismatch è `ValidationException: Invocation of model ID ... is not supported`.

32.2 Autenticazione e Policy IAM

Nessuna chiave API statica (ADR-AWS-1, § 39). L'invocazione avviene tramite il Task Role del servizio `agents`.

Tabella 33: Configurazione IAM per l'accesso a Bedrock.

Parametro Policy	Valore Operativo
Azioni Ammesse	<code>bedrock:InvokeModel</code> , <code>bedrock:InvokeModelWithResponseStream</code>
Risorse (ARNs)	<code>arn:aws:bedrock:eu-south-1::foundation-model/qwen.*</code> (Scoping limitato alla famiglia Qwen3 per impedire l'uso accidentale di modelli costosi).
Condizioni	<code>aws:RequestedRegion == eu-south-1</code> .

32.3 Model Invocation Logging

- **Cosa registrare:** Solo i metadati (Modello invocato, timestamp, token consumati).
- **Cosa NON registrare:** Corpo del prompt e della risposta (per policy di sicurezza sul codice).
- **Vincolo:** Aggiungere resource policy al Log Group per permettere al principal `bedrock.amazonaws.com` la scrittura.

33 Storage e Comunicazioni Esterne

33.1 Amazon S3 (Artefatti Applicativi)

Sostituisce MinIO per il salvataggio dei report generati.

Tabella 34: Configurazione Bucket S3 (Artefatti).

Parametro	Valore Operativo
Accesso di Rete	Nessun accesso pubblico (<code>Block Public Access = True</code>). Accesso solo via VPC Endpoint Gateway.
Permessi IAM	Concessi in lettura/scrittura esclusivamente al Task Role del servizio <code>backend</code> .
Cifratura & Versioning	Cifratura KMS abilitata. Versioning disabilitato.
Lifecycle Policy	Eliminazione automatica oggetti > 30 giorni.

34 Frontend Web Hosting

Il frontend React (Vite) viene compilato come sito statico. Un'unica distribuzione CloudFront serve sia il Frontend sia le API, eliminando la necessità di configurare policy CORS (ADR-AWS-9, § 39).

34.1 Amazon S3 (Hosting Statico)

- **Accesso Pubblico:** Bloccato. Il Bucket NON deve usare la funzione "Static Website Hosting" nativa.
- **Accesso in Lettura:** Esclusivo per CloudFront tramite Origin Access Control (OAC).

34.2 Distribuzione Amazon CloudFront

Tabella 35: Behavior di CloudFront per l'MVP condiviso.

Behavior Path	Origin Target	Configurazione e Policy
/* (Default)	Bucket S3	Viewer Protocol: Redirect HTTP to HTTPS. Cache: Abilitata.
/api/* e /socket.io/*	ALB (Backend)	Viewer Protocol: HTTP Only (L'ALB non ha certificato TLS). Cache: Disabilitata. Origin Request Policy: Forward All (Inoltro header Upgrade, Connection, e Authorization).

34.3 Gestione SPA Routing e Build Vite

- **Custom Error Response (CloudFront):** Errori 403/404 generati da S3 devono essere redirezionati al file `index.html` con Status Code 200 (necessario per TanStack Router).
- **Variabili Runtime:** Configurare `VITE_API_BASE_URL` con un path relativo (es. `/api/v1`) durante la build.
- **Cache Invalidation:** Il ruolo CI/CD deve avere i permessi `cloudfront:CreateInvalidation` per svuotare la cache ad ogni deploy.

35 Segreti e Configurazione (Variabili d'Ambiente)

35.1 Mappatura Variabili PoC → AWS

Integrazione trasparente: il codice applicativo continua a leggere le variabili tramite `process.env` / `os.environ`. La configurazione viene iniettata nativamente da ECS al momento del bootstrap.

Tabella 36: Mappatura delle Variabili d'Ambiente.

Variabile (PoC)	Servizio	Meccanismo AWS	Permessi IAM	Lettura
MONGO_URI	backend	Secrets Manager	Task Execution	(back-end)
JWT_SECRET	backend	Secrets Manager	Task Execution	(back-end)
CREDENTIAL_MASTER_KEY	backend	Secrets Manager	Task Execution	(back-end)
INTERNAL_SHARED_SECRET	Entrambi	Secrets Manager	Task Exec	(backend & agents)
REDIS_URL	backend	Parameter Store	Task Execution	(back-end)
BACKEND_BASE_URL	agents	Parameter Store	Task Execution	(agents)
LLM_PROVIDER	agents	Parameter Store	Task Execution	(agents)
<i>Variabili Rimosse o Modificate</i>				
LLM_API_KEY, S3_KEY	Tutti	ELIMINATE	Sostituite da IAM Task Role.	
PROMPTS_DIR	agents	Bake-in Docker	Niente mount locali in Fargate.	

35.2 Cifratura (AWS KMS)

Creare un'unica chiave **KMS Customer Managed Key** per l'MVP.

- **Utilizzatori per cifratura:** Servizio ElastiCache, Secrets Manager, S3. *MongoDB Atlas cifra a riposo con chiavi proprie, fuori da questa CMK.*
- **Utilizzatori per decifratura:** Task Execution Roles di **backend** e **agents** (per l'iniezione sicura all'avvio del container).

36 Budget e Osservabilità Infrastrutturale

36.1 Metriche e Allarmi Base

Tutti gli allarmi vanno instradati verso un unico Topic SNS (**codeguardian-alerts**), condiviso dal team di sviluppo via email.

Tabella 37: Matrice degli allarmi infrastrutturali base.

Allarme / AWS Service	Metrica	Soglia di Attivazione
ECS Fargate	Task in esecuzione	< 1 (Desired Count) per > 5 minuti.
ECS Fargate	Utilizzo CPU / Memoria	> 80% per > 5 minuti.
ALB	Tasso di errore HTTP 5xx	> 5% su finestra di 5 minuti.
ElastiCache	Utilizzo CPU Cache	> 80% per > 5 minuti.
Amazon Bedrock	Invocation Throttling	≥ 1 invocazione limitata in 5 minuti.

Nota Implementativa. Le metriche del cluster **MongoDB Atlas** (CPU, connessioni, IOPS, replication lag) non sono esposte nativamente su CloudWatch: gli allarmi relativi si configurano con l'*alerting integrato di Atlas*, instradato verso lo stesso canale del team (email o, opzionalmente, integrazione Atlas → SNS/webhook). CloudWatch resta autoritativo per ECS, ALB, ElastiCache e Bedrock.

Correlazione dei Log tra Servizi. Ogni log strutturato emesso da **backend** e **agents** deve includere il campo **taskId** (già presente nel modello di dominio del PoC) in ogni riga, per l'intera durata dell'esecuzione di una Task. Questo consente di filtrare CloudWatch Logs Insights per una singola run end-to-end (avvio, tool-call verso GitHub, invocazione Bedrock, persistenza del Report) senza introdurre un sistema di tracing distribuito dedicato, non giustificato nei tempi dell'MVP.

36.2 Gestione Costi (AWS Budgets)

Azione Dev: Usare AWS Pricing Calculator per ottenere il Costo Mensile Stimato basato sui parametri di questo documento (1 NAT, 1 ElastiCache, 7 VPC Endpoint — incluso Atlas PrivateLink —, Compute ECS). **Il costo del cluster MongoDB Atlas M10 è fatturato separatamente da MongoDB** (non compare in AWS Pricing Calculator) e va sommato alla stima; l'Interface Endpoint di PrivateLink è invece un costo AWS.

- **Soglia 50%:** Informativa (Nessuna azione richiesta).
- **Soglia 80%:** Attenzione (Indagare eventuali loop di esecuzione).
- **Soglia 100%:** Superamento budget (Nessun blocco automatico configurato per evitare downtime bloccanti).

37 Tracciamento Requisito → Decisione AWS

Questa tabella mappa i requisiti funzionali, di vincolo e qualitativi (definiti nell'Analisi dei Requisiti) sulle scelte operative AWS implementate in questa configurazione. I requisiti puramente applicativi restano di competenza dello sviluppo software.

Tabella 38: Mappatura Requisiti - Decisioni Infrastrutturali AWS.

Requisito	Descrizione Sintetica	Decisione Infrastrutturale (AWS)	Rif.
RV.15	Architettura e hosting basati su AWS.	Compute e servizi core su AWS: VPC, ECS (Fargate), ElastiCache, Bedrock, S3, CloudFront. Il database è MongoDB Atlas, servizio gestito di terze parti <i>deployato su AWS</i> in eu-south-1 e raggiunto via PrivateLink: hosting su AWS e data residency preservati, con una dipendenza da fornitore esterno (§ 40).	Tutto il doc.
RV.12	Database non relazionale basato su MongoDB.	Adozione di MongoDB Atlas (cluster M10, MongoDB nativo) in eu-south-1 , connesso via AWS PrivateLink.	§ 5
RV.13	Interfacciamento con LLM tramite AWS Bedrock.	Invocazione nativa Bedrock tramite Task Role IAM del container agents , via VPC Endpoint.	§ 6

(continua)

Tabella 38 (segue dalla pagina precedente)

Requisito	Descrizione Sintetica	Decisione Infrastrutturale (AWS)	Rif.
RV.21	Schema Design della base dati non relazionale.	Demandato al team backend. Nessuno spike di compatibilità richiesto: Atlas è MongoDB nativo (driver Mongoose pienamente supportato).	§ 5
RV.4	Supporto a Repository privati, oltre ai pubblici.	Token GitHub cifrato in AWS Secrets Manager (RS.2), letto dal backend via Task Execution Role; l'accesso in HTTPS verso l'API GitHub avviene tramite il NAT Gateway, indipendentemente dalla visibilità del repository.	§ 3, § 9
RV.14	Integrazione nativa con GitHub e GitHub Issues.	Nessun servizio AWS dedicato: il backend raggiunge le API GitHub in HTTPS in uscita (sg-backend → NAT Gateway), unico canale esterno oltre a Bedrock.	§ 3
RS.2	Token e credenziali memorizzati in formato cifrato.	Segreti instradati su AWS Secrets Manager, cifrati con KMS e iniettati nativamente da ECS al bootstrap.	§ 9
RS.3	Divieto di scrittura autonoma agenti su Git.	Il Security Group sg-agents blocca in uscita tutto il traffico Internet (0.0.0.0/0). Letture e scritture (apertura Pull Request) sono eseguite esclusivamente dal backend, mai dagli agenti.	§ 3, § 13
RS.5	Non utilizzo codice per addestramento LLM terzi.	Transito tramite VPC Endpoint e invocazione via Bedrock, coperto dalla garanzia contrattuale AWS.	§ 6
RQ.4	Disaccoppiamento prompt dalla logica backend.	I prompt restano file esterni, ma il vincolo Fargate impone il bake-in nell'immagine Docker a ogni modifica.	§ 9, § 41
RQ.5	Accuratezza risposte agenti $\geq 85\%$.	Configurazione di modelli adeguati (Qwen3-Coder) e predisposizione di percorsi di scalata (Qwen3-235B).	§ 6
RQ.6	Tempo di risposta agente ≤ 5 minuti.	Adozione di ECS su Fargate al posto di AWS Lambda, incompatibile con lunghe esecuzioni o WebSocket.	§ 4
RV.6	Limitazione budget API.	Alert AWS Budgets su 3 soglie; limitazione della Policy IAM Bedrock alla sola famiglia Qwen3.	§ 10
RQ.8	Rate Limiting token API.	Rate limiter applicativo su Redis (sliding window, 60 req/min per utente) lato backend, a protezione delle chiamate verso l'API GitHub; i limiti di account Bedrock restano una difesa di secondo livello, non il controllo primario.	§ 5

Requisiti Fuori Perimetro Infrastrutturale

I seguenti requisiti non compaiono in tabella poiché la loro soddisfazione dipende interamente dal codice sorgente o dalle policy di progetto, e non richiedono specifiche risorse o configurazioni AWS:

- **RV.1:** Natura deterministica dell'orchestratore.
- **RV.7:** Analisi supportata per TypeScript, JavaScript, Python.

- **RS.1:** Hashing password utente con **argon2id** (raccomandazione OWASP 2023; supportato in NestJS tramite il pacchetto **argon2**).
- **RS.4:** Validità logica delle credenziali Git esterne.
- **RV.16 – RV.20:** Vincoli di documentazione di progetto, Swagger, e versionamento codice.

Parte V

Infrastruttura AWS — Motivazioni e Decisioni Architettureali

Questa parte raccoglie le argomentazioni, i principi guida e le decisioni architetturali che hanno portato alla configurazione descritta nella Parte IV.

38 Principi Guida e Vincoli di Progetto

Le scelte architetturali di questo MVP sono state guidate da un set di vincoli e principi applicati trasversalmente a tutti i livelli dell'infrastruttura. Invece di ribadirli per ogni singolo servizio, vengono dichiarati qui come fondamento delle decisioni successive.

- **Vincolo Tempo/Risorse (Il costo operativo):** Il team di sviluppo è composto da 6 persone con meno di due settimane per l'intera implementazione (infrastruttura inclusa). Questo ha imposto la scelta sistematica di servizi gestiti (es. Fargate al posto di EC2) e di topologie minimamente valide (es. singola istanza dati, singolo NAT) per abbattere il tempo di configurazione e manutenzione.
- **Principio di Migrazione (“Code-Freeze” infrastrutturale):** Nessuna sostituzione infrastrutturale deve toccare il codice applicativo o la logica di business. Le immagini Docker di **backend** e **agents** validate nel PoC (Parte I) restano le unità di deploy. L'adattamento avviene esclusivamente tramite iniezione di variabili d'ambiente e interfacce compatibili (es. MongoDB Atlas gestito al posto del container MongoDB del PoC, con lo stesso driver Mongoose).
- **Sicurezza Garantita per Costruzione vs Dichiarata:** Se un vincolo applicativo impone una restrizione (es. divieto di scrittura su GitHub per gli agenti), questo viene garantito a livello di rete (Security Groups, Routing) e non affidato unicamente alla correttezza del codice Python.
- **Infrastructure as Code (AWS CDK in TypeScript):** CDK è stato scelto rispetto a Terraform/HCL poiché il team possiede già una buona padronanza di TypeScript (React, NestJS). Inserire un nuovo linguaggio dichiarativo avrebbe impattato negativamente sul tempo di sviluppo.

39 Decisioni Architettureali (ADR) e Motivazioni

Di seguito vengono espanso e contestualizzate le decisioni architetturali cardine (ADR-AWS), raggruppate per dominio di competenza.

39.1 Identità, Accessi e Segreti

- **ADR-AWS-1 — Nessuna chiave statica a lungo termine:** L'autenticazione verso i servizi AWS (Bedrock, S3, ECR) avviene esclusivamente tramite **IAM Task Roles**. È un miglioramento netto rispetto al PoC: le credenziali sono temporanee, auto-ruotanti e mai esposte nel codice. Si separano inoltre i ruoli di esecuzione (Task Execution Role per pull immagini/segreti) da quelli applicativi (Task Role).
- **ADR-AWS-5 — Singola Chiave KMS Condivisa:** Cifratura a riposo applicata a ElastiCache, S3 e Secrets Manager tramite un'unica chiave gestita dal cliente (MongoDB Atlas cifra a riposo con chiavi proprie, fuori da questa CMK). Un'architettura a chiavi isolate per servizio offrirebbe una limitazione del danno superiore, ma moltiplicherebbe la complessità delle policy IAM, compromesso non giustificato per un MVP di questa scala.
- **ADR-AWS-10 — Scrittura su GitHub: esclusivamente dal backend, mai dagli agenti:** La generazione automatica di Pull Request (RF.82, RF.83, RF.86) richiede un token GitHub con scope di scrittura (repo). Per non violare RS.3, l'apertura della PR non è mai eseguita dall'agente Python: l'agente produce solo una **Proposal** (diff) tramite la facade read-only esistente; è il backend NestJS, ricevuto l'output dell'agente, a invocare l'endpoint di scrittura delle API GitHub. L'isolamento di rete di **sg-agents** (zero egress) resta invariato: il confine di sicurezza non è lo scope del token ma il fatto che gli agenti non possiedono alcun percorso di rete verso l'esterno, nemmeno per la scrittura. La validazione umana della modifica proposta avviene interamente su GitHub (UC26.2.4/RF.63): il sistema non implementa un flusso di approvazione interno per questa operazione.

39.2 Rete, Sicurezza e Isolamento

- **ADR-AWS-2 e ADR-AWS-4 — Isolamento del livello dati e degli Agenti:** Redis (ElastiCache) risiede in subnet private senza IP pubblico; il database MongoDB Atlas è esterno alla VPC ma raggiunto esclusivamente via **AWS PrivateLink** (Interface Endpoint privato nelle subnet, nessun transito su Internet pubblico né dipendenza dal NAT). Il container **agents** non possiede alcuna route (nemmeno tramite NAT) verso 0.0.0.0/0. Questo rende impossibile per un agente contattare autonomamente GitHub o server esterni, garantendo il vincolo **RS.3** direttamente a livello di rete.
- **ADR-AWS-6 — Bedrock via PrivateLink:** Il modello LLM viene interrogato via VPC Endpoint Interface. Questo garantisce che il codice sorgente contenuto nei prompt non transiti mai su Internet pubblico, soddisfacendo la conformità sulla data residency e privacy.
- **ADR-AWS-7 — Debugging via ECS Exec:** L'accesso shell ai container per il debug avviene tramite AWS Systems Manager (SSM) Session Manager. Non vengono creati Bastion Host né utilizzate chiavi SSH. Questo elimina la vulnerabilità classica di istanze intermedie dimenticate aperte.
- **Scelta di Routing (Singolo NAT e 2 AZ):** Si adottano due Availability Zone poiché rappresentano il minimo tecnico richiesto dall'ALB. Si adotta un singolo NAT Gateway in **public-a** per contenere i costi continui. In caso di fail della AZ-A, il backend perderebbe temporaneamente l'accesso a GitHub. È un compromesso calcolato e accettato per la scala dell'MVP.

39.3 Compute, Orchestrazione e Frontend

- **Perché Fargate e non Lambda/EC2:** EC2 è stato escluso per azzerare il tempo di amministrazione OS/host. Lambda è stato escluso poiché gli agenti hanno timeout lunghi

(fino a 5 min) con stati conversazionali complessi (LangGraph), e il backend richiede connessioni WebSocket persistenti, pattern incompatibili col paradigma Lambda.

- **Desired Count pari a 1:** Mantenere 1 task per servizio semplifica drasticamente l'MVP. Scaling orizzontale del backend richiederebbe l'introduzione di sticky-sessions sull'ALB o un adapter Redis per la sincronizzazione dei WebSocket.
- **ADR-AWS-3 — Traffico interno via Cloud Map:** Il traffico tra backend e agenti avviene internamente tramite Service Discovery (namespace `.local`) protetto da HMAC. Questo evita di esporre API riservate del backend sull'Application Load Balancer pubblico.
- **ADR-AWS-8 e ADR-AWS-9 — ALB dietro CloudFront condiviso:** Il perimetro esclude l'acquisto di un dominio custom. L'ALB non può quindi ottenere un certificato SSL valido e ascolta in chiaro su porta 80. Viene posto dietro una distribuzione CloudFront (che fornisce TLS nativo gratis su dominio `*.cloudfront.net`) e l'ALB accetta traffico unicamente dai prefix-list di AWS Edge. Servendo sia S3 (Frontend) che ALB (Backend) sotto lo stesso dominio CloudFront, si rende il traffico *Same-Origin*, azzerando totalmente le configurazioni e i bug relativi alle policy CORS.

39.4 Scelte sul Livello Dati

- **MongoDB Atlas (gestito, esterno ad AWS):** Sostituisce il container MongoDB locale del PoC. Essendo MongoDB nativo, elimina il rischio di compatibilità sul driver e lo spike di validazione che DocumentDB avrebbe imposto. Il compromesso si sposta sulla natura del servizio: Atlas è un *data processor* di terze parti, mitigato deployandolo su AWS in `eu-south-1` e raggiungendolo via PrivateLink (traffico sul backbone privato AWS, data residency preservata). Resta una dipendenza da fornitore esterno con fatturazione separata.
- **Disponibilità del livello dati:** Il cluster Atlas M10 è un replica set a 3 nodi multi-AZ, quindi **HA per costruzione**: il database non è più un SPOF a istanza singola come con DocumentDB. Resta invece single-node **ElastiCache (Redis)**, senza repliche in questa fase; l'alta disponibilità della cache è esclusa dallo scope per motivi di budget e tempo.

39.5 Scelta dei Modelli LLM (Bedrock)

- **Esclusione di Llama 3.3 70B:** Il modello usato nel PoC non ha endpoint di inferenza nativi nella regione `eu-south-1` su Bedrock. Abilitarlo implicherebbe il routing del codice sorgente analizzato verso regioni USA (*cross-region inference*), violando i principi di data residency e incrementando i costi operativi.
- **Adozione di Qwen3 (32B e 30B Coder):** Si adotta la famiglia Qwen3, nativa su Milano e in linea con il budget (\$0.15/\$0.60 per 1M token). L'agente Security sfrutta il modello `Qwen3-Coder-30B-A3B-Instruct` (MoE) che vanta un reliability score del 76.6% sul benchmark BFCL-v2 per il tool-calling, un requisito fondamentale per far sì che l'agente decida autonomamente quali file interrogare.
- **Fallback Plan:** Qualora il modello da 30B fallisse i test di accuratezza sul golden set (Piano di Qualifica), il team effettuerà la scalata su `Qwen3-235B-A22B`, sempre disponibile in `eu-south-1` per mantenere la conformità, con un modesto aumento dei costi.

40 Compromessi e Rischi Accettati

Il vincolo temporale per lo sviluppo dell'MVP ha reso necessario bilanciare la perfezione architetturale con il pragmatismo del rilascio. Invece di disseminare queste giustificazioni in ogni

capitolo, vengono consolidate nella seguente tabella, che mappa i compromessi infrastrutturali accettati e il relativo Single Point of Failure (SPOF) o limite di sicurezza.

Tabella 39: Matrice dei Compromessi e Rischi Accettati per l'MVP.

Risorsa / Scelta	Compromesso Architetture	Rischio Accettato
NAT Gateway	Deployment di un singolo NAT Gateway allocato nella subnet public-a invece di uno per ogni Availability Zone.	Se la AZ eu-south-1a dovesse subire un disservizio, gli agenti perderebbero la connettività per leggere da GitHub. Lato frontend, l'errore è gestito come fallimento generico di orchestrazione (UC23), distinto da un errore di credenziale.
Ridondanza Compute	Subnet e ALB predisposti su 2 AZ, ma Desired Count: 1 per ogni servizio ECS: ogni servizio gira comunque su una sola istanza, in una sola AZ, alla volta. La topologia multi-AZ è quindi predisposizione di rete, non protezione attiva contro il fallimento del task.	In caso di crash del task o di disservizio sull'AZ ospitante, il servizio è irraggiungibile fino al riavvio automatico di ECS (nessun failover immediato su AZ alternativa). Rischio della stessa natura di quello già accettato per Auto-Scaling, qui esplicitato separatamente perché riguarda la distribuzione geografica, non solo il numero di repliche.
ElastiCache (Redis)	Nodo singolo (nessuna replica, no modalità cluster). Il DB su Atlas M10 è invece multi-AZ e non è più un SPOF.	SPOF residuo limitato a cache e coda BullMQ. Cluster ElastiCache Multi-AZ non giustificato per un carico da validazione MVP.
MongoDB Atlas (fornitore esterno)	Database gestito da terze parti, fuori dall'account AWS, con fatturazione separata e piano di controllo non gestito via CDK.	Dipendenza da un provider esterno e da PrivateLink per la connettività. Data residency preservata (deploy su AWS eu-south-1); un disservizio di Atlas o del Private Endpoint interrompe il livello dati.
AWS KMS	Utilizzo di un'unica chiave crittografica condivisa per ElastiCache, S3 e Secrets Manager.	Una compromissione della chiave esporrebbe questi livelli di persistenza AWS (MongoDB Atlas cifra a riposo separatamente). Si evitano complessità e moltiplicazione delle policy IAM.
Application Load Balancer	ALB senza certificato TLS pubblico e senza dominio custom, in ascolto solo su HTTP/80 (dietro CloudFront). Conseguenza diretta dell'esclusione del Dominio Custom (Route53) dal perimetro MVP (§ 2.2): senza un dominio proprio non è possibile emettere un certificato ACM valido per l'ALB.	Il tratto di rete tra l'edge CloudFront e l'ALB non è cifrato (traffico comunque ristretto alla rete privata AWS, non instradato su Internet pubblico). Il rischio è accettato in coerenza con la decisione di non acquistare un dominio per l'MVP (ADR-AWS-8); introdurlo richiederebbe di riaprire quella decisione, non solo di configurare un certificato.

(continua)

Tabella 39 (segue dalla pagina precedente)

Risorsa / Scelta	Compromesso Architettuale	Rischio Accettato
Auto-Scaling	Desired count fisso a 1 per i servizi ECS. Nessuna policy di autoscaling configurata.	Irraggiungibilità momentanea in caso di task failure durante il tempo di riavvio (Drop SLA). La resilienza (retry, risincronizzazione post-riconnessione) è responsabilità applicativa backend/frontend, da definire nell'MVP riusando l'approccio del PoC.
AWS WAF	Nessun Web Application Firewall configurato a livello Edge (CloudFront/ALB).	Nessuna protezione infrastrutturale contro DDoS Layer 7 o abusi di throttling applicativo da client anomali.
Token GitHub	Token unico con scope di scrittura (repo), condiviso tra letture e apertura PR. Nessuna segmentazione read/write a livello di credenziale.	Una compromissione del backend (unico detentore del token decifrato) consentirebbe scritture sul repository. Il rischio è mitigato dall'isolamento di rete degli agenti e dalla cifratura KMS del token in Secrets Manager, ma non eliminato: non c'è un secondo token a permessi ridotti per le sole letture.
Backend — Sizing task per PDF	Task backend dimensionato per un servizio REST leggero (0.5 vCPU / 1 GB, Tabella 26), esteso senza revisione dedicata a generare e streammare PDF nello stesso container.	Non verificato dal team se il carico di generazione PDF (specialmente in concorrenza con task WebSocket attivi) rientri nel budget di memoria. Rischio contenuto se la generazione usa una libreria non basata su browser headless; da confermare con un test di carico mirato prima del rilascio, non con un ridimensionamento preventivo.

41 Limiti di Piattaforma (Fargate e Prompt)

Il PoC consentiva la modifica in tempo reale dei prompt passati ai modelli LLM grazie al montaggio di volumi (*bind mount*) da file system locale su Docker Compose. **AWS Fargate non supporta i bind mount da directory locali.** Per evitare l'introduzione di un file system condiviso (Amazon EFS), che comporterebbe costi continui e overhead di configurazione ingiustificati per l'MVP, **i file dei prompt devono essere inclusi staticamente (bake-in) all'interno dell'immagine Docker.**

Conseguenza operativa: Il disaccoppiamento logico richiesto da **RQ.4** resta soddisfatto (i prompt sono file YAML isolati dal codice Python), ma ogni modifica al testo di un prompt richiederà ora la ricompilazione dell'immagine e un nuovo deploy del servizio **agents**.

Il tuning effettivo dei prompt avviene comunque in locale via Docker Compose (bind mount, come nel PoC): il bake-in impatta solo la promozione di una versione già validata verso l'ambiente Fargate, non il ciclo di iterazione. Un'alternativa a costo contenuto (prompt su S3, scaricati a runtime) resta valutabile in una fase successiva, se il tuning dovesse richiedere iterazione diretta contro Fargate/Bedrock.

Parte VI

Progettazione Backend MVP

42 Perimetro e obiettivo

Questa parte documenta la progettazione del backend per la versione MVP di Code Guardian, a cura di Sara Ristovic (2026/08/14). Estende e sostituisce il contratto PoC descritto nella Parte I per tutti i componenti portati a maturità nell'MVP: autenticazione reale, cifratura delle credenziali di servizio, evoluzione del contratto `CreateContextDto`, validazione del contesto in otto passi e il meccanismo di interazione asincrona tra l'agente Changelog e l'utente.

Il backend è implementato in NestJS (Node.js/TypeScript). Gli agenti Python (LangGraph) comunicano con il backend tramite HTTP interno su endpoint `/internal/*` autenticati via HMAC, come già stabilito nel PoC (Parte I, § 6).

42.1 Cosa cambia dal PoC all'MVP

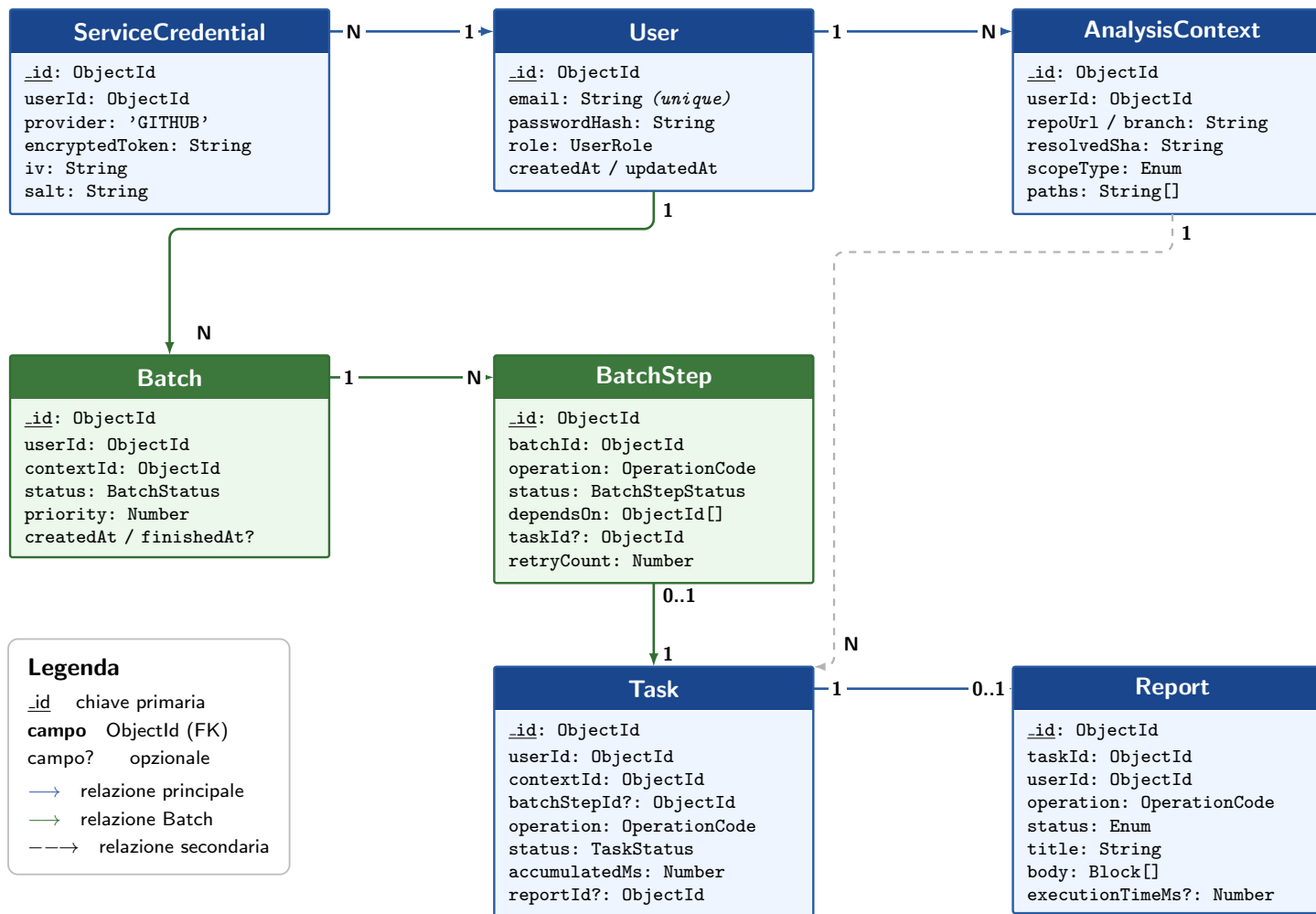
Tabella 40: Mappatura delle evoluzioni PoC → Backend MVP.

PoC	MVP
Auth stub (JWT statico, nessun utente reale)	Auth reale: Argon2id + JWT HS256, registrazione e login effettivi (BE-ADR-1, BE-ADR-5)
Nessuna cifratura credenziali applicativa	AES-256-GCM con chiave derivata via HKDF (BE-ADR-2)
<code>CreateContextDto</code> : <code>repoOwner</code> + <code>repoName</code> + <code>ref</code>	<code>CreateContextDto</code> : <code>repoUrl</code> + <code>branch</code> + <code>commitSha?</code> (BE-ADR-4)
Nessuna validazione del contesto	Sequenza di 8 passi di validazione su <code>POST /contexts</code>
<code>AnalysisContextDto</code> senza <code>branch</code>	<code>AnalysisContextDto</code> include il campo <code>branch</code>
4 eventi WebSocket	5 eventi WebSocket (aggiunto <code>task.inputRequired</code>)
Nessun input utente a runtime	<code>POST /tasks/:id/input</code> + meccanismo <code>pause/resume</code> LangGraph
3 <code>OperationCode</code> nell' <code>AgentRegistry</code>	7 <code>OperationCode</code> nell' <code>AgentRegistry</code>

43 Schema MongoDB (Modello di Persistenza)

Il backend usa Mongoose come ODM su MongoDB. Le sette collection dell'MVP sono descritte di seguito con i campi, i tipi e i vincoli reali. Le collection `batches` e `batchsteps` (in verde nel diagramma) supportano il meccanismo di esecuzione multipla a DAG dell'orchestratore di produzione (§ 12). I nomi dei campi seguono la convenzione `camelCase`.

43.1 Diagramma Entità-Relazione



Note: *task.contextId* e

batch.contextId puntano entrambi ad *analysiscontexts._id* (mostrata solo la seconda per chiarezza). *BatchStep.dependsOn* è un array di riferimenti ad altri *BatchStep* dello stesso batch (DAG intra-batch, non mostrato). *Report.context* è uno snapshot denormalizzato, non una FK.

43.2 Collection users

```
@Schema({ timestamps: true })
class UserDocument {
  @Prop({ required: true, trim: true, maxlength: 40 })
  firstName: string

  @Prop({ required: true, trim: true, maxlength: 40 })
  lastName: string

  @Prop({ required: true, unique: true, lowercase: true, trim: true })
  email: string // Indice unique: true (lookup login)

  @Prop({ required: true })
  passwordHash: string // Hash Argon2id (mai esposto via API)

  @Prop({ required: true, enum: ['DEVELOPER', 'SECURITY_AUDITOR', 'PROJECT_MANAGER'] })
  role: UserRole

  // timestamps: true aggiunge createdAt e updatedAt automaticamente
}
```

Listing 14: Schema Mongoose della collection `users`. Nessun campo sensibile (es. token GitHub) e' memorizzato qui.

43.3 Collection servicecredentials

```
@Schema({ timestamps: true })
class ServiceCredentialDocument {
  @Prop({ type: MongooseSchema.Types.ObjectId, ref: 'User', required:
    true, index: true })
  userId: ObjectId // Indice: lookup per utente

  @Prop({ required: true, enum: ['GITHUB'] })
  provider: string // Ora solo GITHUB; estendibile

  @Prop({ required: true })
  encryptedToken: string // AES-256-GCM ciphertext (base64)

  @Prop({ required: true })
  iv: string // Initialization vector per-record (base64, 12 byte)

  @Prop({ required: true })
  salt: string // Salt HKDF per-record (base64, 32 byte)

  // timestamps: true aggiunge createdAt e updatedAt
  // Vincolo applicativo: un solo record per (userId, provider)
}
```

Listing 15: Schema Mongoose per le credenziali cifrate (token GitHub). La chiave master non e' nel documento: vive in AWS Secrets Manager.

43.4 Collection analysiscontexts


```

@Schema({ timestamps: true })
class AnalysisContextDocument {
  @Prop({ type: MongooseSchema.Types.ObjectId, ref: 'User', required:
    true, index: true })
  userId: ObjectId

  @Prop({ required: true })
  repoUrl: string          // Es. "https://github.com/org/repo"

  @Prop({ required: true })
  repoOwner: string

  @Prop({ required: true })
  repoName: string

  @Prop({ required: true })
  branch: string

  @Prop({ required: true })
  resolvedSha: string      // SHA commit al momento della creazione (
    immutabile)

  @Prop({ required: true, enum: ['FULL_REPOSITORY', 'FILES', 'DIRECTORIES',
    ''] })
  scopeType: string

  @Prop({ type: [String], default: [] })
  paths: string[]
}

```

Listing 16: Schema Mongoose per `AnalysisContext`. `resolvedSha` e' immutabile: viene scritto una volta sola al POST `/contexts` e non aggiornato mai.

43.5 Collection batches

```

@Schema({ timestamps: true })
class BatchDocument {
  @Prop({ type: MongooseSchema.Types.ObjectId,
    ref: 'User', required: true, index: true })
  userId: ObjectId

  @Prop({ type: MongooseSchema.Types.ObjectId,
    ref: 'AnalysisContext', required: true, index: true })
  contextId: ObjectId

  @Prop({ required: true,
    enum: ['DRAFT', 'SCHEDULED', 'RUNNING',
      'COMPLETED', 'PARTIALLY_FAILED', 'FAILED', 'CANCELLED'],
    default: 'DRAFT' })
  status: BatchStatus

  @Prop({ default: 0 })
  priority: number          // Priorita' di scheduling (0 = default)

  @Prop({ default: null })
  finishedAt: Date | null
}

```

```
// timestamps: true aggiunge createdAt
}
```

Listing 17: Batch: unita' di pianificazione. Contiene un DAG di BatchStep; il suo stato e' derivato dall'aggregazione degli stati degli step da BatchCoordinator.

Stati validi e transizioni di Batch.

Transizione	Trigger
DRAFT → SCHEDULED	Il frontend conferma il batch
SCHEDULED → RUNNING	Almeno uno step passa a RUNNING
RUNNING → COMPLETED	Tutti gli step in COMPLETED
RUNNING → PARTIALLY_FAILED	Almeno uno step FAILED, altri COMPLETED
RUNNING → FAILED	Tutti i rami bloccati da step FAILED
* → CANCELLED	POST /batches/:id/cancel

43.6 Collection batchsteps

```
@Schema({ timestamps: true })
class BatchStepDocument {
  @Prop({ type: MongooseSchema.Types.ObjectId,
    ref: 'Batch', required: true, index: true })
  batchId: ObjectId

  @Prop({ required: true,
    enum: ['DOCS_README', 'DOCS_INLINE', 'DOCS_API',
      'SECURITY_OWASP', 'SECURITY_POLICY',
      'CHANGELOG_TECHNICAL', 'CHANGELOG_BUSINESS'] })
  operation: OperationCode

  @Prop({ required: true,
    enum: ['PENDING', 'READY', 'RUNNING',
      'COMPLETED', 'FAILED', 'SKIPPED', 'CANCELLED'],
    default: 'PENDING' })
  status: BatchStepStatus

  @Prop({ type: [MongooseSchema.Types.ObjectId],
    ref: 'BatchStep', default: [] })
  dependsOn: ObjectId[] // Step predecessori dello stesso batch (
    DAG)

  @Prop({ type: MongooseSchema.Types.ObjectId,
    ref: 'Task', default: null })
  taskId: ObjectId | null // Task generata al momento dell'esecuzione

  @Prop({ default: 0 })
  retryCount: number

  @Prop({
    type: {
      maxAttempts: { type: Number, default: 3 },
      backoffMs: { type: Number, default: 5000 },
      retryOnlyKinds: { type: [String], default: [] },
    },
  })
```

```

    _id: false, default: () => ({}))
  })
  retryPolicy: RetryPolicy
}

```

Listing 18: BatchStep: singolo nodo del DAG. `dependsOn` contiene gli `_id` degli step predecessori (stesso batch) che devono essere `COMPLETED` prima che questo diventi `READY`. `taskId` e' valorizzato quando lo step passa a `RUNNING`.

Stati validi e transizioni di BatchStep.

Transizione	Trigger
PENDING → READY	Tutti i predecessori in <code>COMPLETED</code>
READY → RUNNING	<code>SchedulerService</code> preleva lo step
RUNNING → COMPLETED	Task associata in <code>COMPLETED</code>
RUNNING → FAILED	Task in <code>FAILED</code> e retry esauriti
FAILED/PENDING → SKIPPED	Policy skip su ramo bloccato
* → CANCELLED	Batch cancellato

43.7 Collection tasks

```

@Schema({ timestamps: true })
class TaskDocument {
  @Prop({ type: MongooseSchema.Types.ObjectId,
    ref: 'User', required: true, index: true })
  userId: ObjectId

  @Prop({ type: MongooseSchema.Types.ObjectId,
    ref: 'AnalysisContext', required: true })
  contextId: ObjectId

  @Prop({ type: MongooseSchema.Types.ObjectId,
    ref: 'BatchStep', default: null, index: true })
  batchStepId: ObjectId | null // null = task avviata singolarmente

  @Prop({ required: true,
    enum: ['DOCS_README', 'DOCS_INLINE', 'DOCS_API',
      'SECURITY_OWASP', 'SECURITY_POLICY',
      'CHANGELOG_TECHNICAL', 'CHANGELOG_BUSINESS'] })
  operation: OperationCode

  @Prop({ required: true,
    enum: ['PENDING', 'RUNNING', 'WAITING_INPUT', 'COMPLETED', 'FAILED',
      'CANCELLED'],
    default: 'PENDING' })
  status: TaskStatus

  @Prop({ min: 0, max: 100, default: 0 })
  progressPercent: number

  @Prop({ type: Object, default: null })
  pendingInput: PendingInput | null // Vedi sez.~\ref{sec:be-input-utente}
}

```

```

// lgThreadId: ID del thread LangGraph per il meccanismo pause/resume.
// Valorizzato solo per task Changelog; null per tutte le altre
operazioni.
@Prop({ default: null })
lgThreadId: string | null

// accumulatedMs: somma dei segmenti di tempo macchina gia' completati
.
// Copiato in Report.executionTimeMs al completamento della task.
@Prop({ default: 0 })
accumulatedMs: number

@Prop({ type: MongooseSchema.Types.ObjectId, ref: 'Report', default:
  null })
reportId: ObjectId | null // Valorizzato quando status in {COMPLETED,
  FAILED}

@Prop({ type: Object, default: null })
error: { code: string; message: string; stage: string } | null

@Prop({ default: null })
finishedAt: Date | null
}

```

Listing 19: Schema Mongoose per Task. Il campo `batchStepId` sostituisce il precedente `batchId`: la relazione ora punta allo step del DAG che ha generato la task, non direttamente al batch. Le task avviate singolarmente hanno `batchStepId = null`.

Stati validi e transizioni di Task.

Transizione	Trigger
PENDING → RUNNING	Il TaskProcessor preleva la task dalla coda
RUNNING → WAITING_INPUT	L'agente (o il backend) solleva un <code>pendingInput</code>
WAITING_INPUT → RUNNING	L'utente risponde con <code>PROCEED</code>
RUNNING → COMPLETED	L'agente risponde <code>status:"completed"</code>
RUNNING → FAILED	Errore di agente o timeout
WAITING_INPUT → CANCELLED	L'utente risponde con <code>CANCEL</code>

43.8 Collection reports

```

@Schema({ timestamps: false }) // generatedAt gestito manualmente
class ReportDocument {
  @Prop({ type: MongooseSchema.Types.ObjectId, ref: 'Task', required:
    true, index: true })
  taskId: ObjectId

  @Prop({ type: MongooseSchema.Types.ObjectId, ref: 'User', required:
    true, index: true })
  userId: ObjectId

  @Prop({ required: true,
    enum: ['DOCS_README', 'DOCS_INLINE', 'DOCS_API',
      'SECURITY_OWASP', 'SECURITY_POLICY',
      'CHANGELOG_TECHNICAL', 'CHANGELOG_BUSINESS'] })

```

```

operation: OperationCode

@Prop({ required: true, enum: ['COMPLETED', 'FAILED'] })
status: 'COMPLETED' | 'FAILED'

@Prop({ required: true })
title: string // Deterministico, mai null (RF.51)

@Prop({ default: null })
summary: string | null

@Prop({ required: true })
generatedAt: Date

@Prop({ default: null })
executionTimeMs: number | null // = Task.accumulatedMs al
    completamento

@Prop({
    type: {
        repoOwner: { type: String, required: true },
        repoName: { type: String, required: true },
        repoUrl: { type: String, required: true },
        branch: { type: String, required: true },
        resolvedSha: { type: String, required: true },
        scopeType: { type: String, required: true },
        paths: { type: [String], default: [] },
    },
    required: true, _id: false,
})
context: ReportContext // Snapshot denormalizzato (non FK)

@Prop({ type: [Object], default: [] })
body: Block[] // TextBlock | FindingBlock |
    // ComplexityWarningBlock |
    // PolicyViolationBlock

@Prop({ type: Object, default: null })
proposal: {
    targetPath: string
    diffUnified: string
    language: string
    pullRequestUrl: string | null
} | null

@Prop({ type: Object, default: null })
error: { code: string; message: string; stage: string } | null
}

```

Listing 20: Il campo `context` e' denormalizzato: snapshot di `AnalysisContext` al momento della creazione, mai aggiornato. `executionTimeMs` e' copiato da `Task.accumulatedMs` al completamento.

43.9 Indici e vincoli di integrità

Tabella 41: Indici MongoDB rilevanti per le query critiche dell'MVP.

Collection	Campo	Tipo e motivazione
users	email	Unique — lookup al login
servicecredentials	userId	Non-unique — recupero token per utente
analysiscontexts	userId	Non-unique — lista contesti (GET /contexts)
batches	userId	Non-unique — lista batch (GET /batches)
batches	contextId	Non-unique — batch per contesto
batchsteps	batchId	Non-unique — step di un batch
tasks	userId	Non-unique — lista task (GET /tasks)
tasks	batchStepId	Non-unique — task generata da uno step
reports	userId	Non-unique — lista report (GET /reports)
reports	taskId	Non-unique — recupero report da task

La coerenza referenziale (es. `reportId` su `Task` punta a un `Report` esistente, `taskId` su `BatchStep` punta a una `Task` esistente) è garantita a livello applicativo nel `TaskProcessor` e nel `BatchCoordinator` di NestJS: se la scrittura di un documento derivato fallisce, lo stato del documento padre viene riportato a `FAILED` dallo stesso handler.

44 Autenticazione e Autorizzazione

L'MVP implementa un sistema di autenticazione stateless basato su Argon2id per la verifica della password e JWT HS256 per i token di sessione. Non esiste un server-side token store: il logout è stateless (il client elimina il token). I ruoli utente sono codificati nel claim JWT e verificati da una `RolesGuard` NestJS.

44.1 Data Transfer Object

```
// --- Body di POST /auth/register ---
interface RegisterDto {
  firstName: string // 1-40 caratteri
  lastName: string // 1-40 caratteri [campo nuovo MVP]
  email: string // email valida, univoca
  password: string // min 8 char, almeno 1 lettera + 1 cifra
  role: UserRole // scelto alla registrazione, non modificabile
}

type UserRole = 'DEVELOPER' | 'SECURITY_AUDITOR' | 'PROJECT_MANAGER'

// --- Risposta di POST /auth/register (201, senza token) ---
interface UserProfileDto {
  id: string
  firstName: string
  lastName: string
  email: string
  role: UserRole
}

// --- Risposta di POST /auth/login e GET /auth/me ---
```

```
// Login restituisce solo il token; il profilo si ottiene con GET /auth/me
interface AuthTokenDto {
    accessToken: string // JWT HS256, scadenza 8h
}

// UserProfileDto e' la risposta anche di GET /auth/me (stesso schema sopra)

// --- Body di POST /auth/login ---
interface LoginDto {
    email: string
    password: string
}

// --- POST /auth/logout: nessun body, risponde 204 ---
// L'endpoint esiste per consentire al frontend di gestire
// errori di rete espliciti al logout; non ha effetto lato server.
```

Listing 21: DTO di autenticazione. `lastName` è un campo obbligatorio nuovo rispetto al PoC; lo schema Mongoose deve includere `@Prop({required:true, trim:true}) lastName: string`. La risposta di login restituisce solo `accessToken`; il profilo completo si ottiene con `GET /auth/me`.

44.2 Hashing delle password

Le password vengono hashate con **Argon2id** prima della persistenza. I parametri scelti sono: memoria 65 536 KB (64 MB), iterazioni 3, parallelismo 4. La scelta di Argon2id rispetto a bcrypt è motivata dalla resistenza agli attacchi side-channel (ibrido memory-hard + data-independent) e dal fatto che OWASP la raccomanda come prima scelta per il 2024.

44.3 JSON Web Token

Il token JWT è firmato con l'algoritmo **HS256** usando la chiave `JWT_SECRET` (iniettata da AWS Secrets Manager in produzione). Il payload include: `sub` (`userId`), `email`, `role`, `iat`, `exp` (8 ore dall'emissione). Non è previsto il refresh token (BE-ADR-5): alla scadenza l'utente effettua nuovamente il login.

Il ruolo viene verificato a ogni richiesta autenticata dalla `RolesGuard`: non è necessario interrogare il database perché il ruolo è già nel claim.

45 Cifratura delle Credenziali di Servizio

I token delle credenziali esterne (es. Personal Access Token GitHub) vengono cifrati prima della persistenza su MongoDB. La chiave di cifratura non è derivata dalla password utente ma da una chiave master gestita da AWS Secrets Manager (`CREDENTIAL_MASTER_KEY`).

45.1 Schema di cifratura

L'algoritmo è **AES-256-GCM** con chiave derivata per record tramite **HKDF** (RFC 5869). Per ogni credenziale vengono generati e persistiti quattro campi:

```
interface EncryptedCredential {
    ciphertext: Buffer // testo cifrato
    iv: Buffer // initialization vector (12 byte, casuale)
    salt: Buffer // sale per HKDF (32 byte, casuale per record)
```

```

    authTag: Buffer          // tag di autenticazione GCM (16 byte)
  }

```

Listing 22: Struttura dati persistita per ogni credenziale cifrata.

La derivazione della chiave avviene come segue: $k = \text{HKDF-SHA256}(\text{IKM} = \text{CREDENTIAL_MASTER_KEY}, \text{salt} = \text{salt}, \text{info} = \text{"credential-enc"})$, con output a 256 bit. La scelta di HKDF al posto di Argon2id è giustificata in BE-ADR-2: la master key è già ad alta entropia (generata da AWS Secrets Manager), quindi non è necessario il key-stretching di Argon2id; HKDF è più adeguato per derivare chiavi da materiale già entropico.

45.2 DTO delle credenziali

```

// --- Body di POST /credentials ---
interface CreateCredentialDto {
  provider: string    // es. 'GITHUB'
  token: string       // Personal Access Token in chiaro (cifrato prima
                      // della persistenza)
}

// --- Risposta di GET /credentials e POST /credentials ---
interface ServiceCredentialDto {
  id: string
  provider: string
  connectedAt: string // ISO 8601
}

```

Listing 23: DTO esposti al frontend per la gestione delle credenziali.

46 GitHub Client — Estensioni MVP

Il `GitHubClient` del PoC espone già i tre tool di lettura (`tree`, `file`, `issues`). L'MVP aggiunge il metodo `compareCommits`, necessario per la validazione RF.22 (un commit dichiarato nel `CreateContextDto` deve appartenere al branch indicato).

```

// Firma del metodo
compareCommits(
  userId: string,
  owner: string,
  repo: string,
  base: string,    // branch
  head: string     // commitSha fornito dal client
): Promise<CompareResult>

interface CompareResult {
  status: 'ahead' | 'behind' | 'identical' | 'diverged'
}

// 'ahead': head e' un antenato di base (commit appartiene al branch) ->
// valido
// 'identical': stesso commit -> valido
// 'behind' | 'diverged': commit non appartiene al branch -> errore RF
// .22

```

Listing 24: `compareCommits` non è un tool `LangGraph` ma un metodo interno del `GitHubClient`, usato dal backend durante la validazione di `POST /contexts`.

Il token GitHub per questa operazione viene decifrato in memoria dal backend a ogni chiamata; non è mai esposto agli agenti. La scelta del singolo scope `repo` per tutti i ruoli (incluse le sole letture) è documentata in BE-ADR-3.

47 Creazione del Contesto di Analisi

47.1 Evoluzione del DTO

```
// PoC (superato):
// interface CreateContextDto {
//   repoOwner: string; repoName: string; ref: string
//   scopeType: ...; paths?: string[]; sprintId?: string
// }

// MVP:
interface CreateContextDto {
  repoUrl: string           // es. https://github.com/owner/repo
                           // owner e repo estratti lato backend (non dal
                           // client)
  branch: string           // nome del branch di riferimento
  commitSha?: string       // opzionale: SHA specifico (se assente, usa
                           // HEAD del branch)
  scopeType: 'FULL_REPOSITORY' | 'FILES' | 'DIRECTORIES'
  paths?: string[]         // richiesto se scopeType != FULL_REPOSITORY
}

// Risposta di POST /contexts (MVP, aggiunto campo branch):
interface AnalysisContextDto {
  id: string
  repoOwner: string
  repoName: string
  isPrivate: boolean
  branch: string           // campo nuovo MVP
  resolvedSha: string      // SHA risolto (HEAD del branch se commitSha
                           // non fornito)
  scopeType: string
  detectedLanguages: string[]
  estimatedFileCount: number
}
```

Listing 25: Evoluzione di `CreateContextDto` dal PoC all'MVP (BE-ADR-4).

Nota: `sprintId` è stato rimosso da `CreateContextDto` e spostato a livello Task: viene fornito dall'utente in risposta all'evento `task.inputRequired` (kind `SPRINT_ID`) durante l'esecuzione dell'agente Changelog, non al momento della creazione del contesto.

47.2 Sequenza di validazione in 10 passi

La `POST /contexts` esegue i seguenti passi in ordine, interrompendosi al primo fallimento. I passi 1–7 si riferiscono al riferimento temporale (`repoUrl`, `branch`, `commitSha?`); i passi 8–9 all'ambito di analisi (`scopeType`, `paths`):

1. **Sintassi URL:** verifica che `repoUrl` sia un URL `https://github.com/:owner/:repo` ben formato (RFC 3986).
2. **Estrazione owner/name:** parsing dell'URL per estrarre `owner` e `repo`; fallisce se il formato non è esattamente `github.com/:owner/:repo`.

3. **Raggiungibilità e accesso:** chiama GitHub API per verificare che il repository esista e che il token dell'utente abbia i permessi necessari (RF.19–RF.20).
4. **Esistenza del branch:** verifica che il branch dichiarato esista nel repository remoto (RF.16).
5. **Membership del commit** (RF.22): se `commitSha` è fornito, chiama `compareCommits` per verificare che il commit appartenga al branch (stato `ahead` o `identical`); se non appartiene, risponde 422.
6. **Ancoraggio SHA:** risolve il commit SHA definitivo (`resolvedSha`) — il `commitSha` fornito se valido, altrimenti `HEAD` del branch (RF.17).
7. **Rilevamento linguaggi:** legge l'albero del repository a `resolvedSha` e determina i linguaggi presenti tramite euristica sull'estensione dei file (RF.25); calcola `estimatedFileCount`.
8. **Ambito non vuoto** (RF.29): per `FILES` o `DIRECTORIES`, `paths` deve contenere almeno un elemento dopo normalizzazione (strip slash iniziale, rimozione `./` e `../`, deduplicazione); per `FULL_REPOSITORY`, `paths` deve essere omesso o vuoto.
9. **Esistenza dell'ambito** (RF.30): ogni percorso in `paths` viene verificato contro l'albero già letto al passo 7; un percorso inesistente, o un file dichiarato come `DIRECTORIES` (o viceversa), produce 422.
10. **Persistenza:** salva l'`AnalysisContext` su MongoDB e restituisce l'`AnalysisContextDto` (RF.31).

I passi 3, 5, 7 e 9 coinvolgono chiamate a GitHub API; tutti gli altri sono validazioni locali. La lettura dell'albero al passo 7 è condivisa col passo 9 tramite cache Redis (chiave `repo@sha:path`): se `GET /repositories/tree` è stato chiamato di recente con lo stesso SHA, il passo 9 è gratuito.

48 Contratto API MVP

Questa sezione costituisce il contratto vincolante per l'implementazione. Integra e sostituisce la tabella PoC (§ 7) per tutti i punti indicati.

48.1 Endpoint REST MVP

Rispetto al PoC, le modifiche principali sono: autenticazione reale (endpoint `/auth/*` non più stub), endpoint `POST /tasks/:id/input` nuovo. Il prefisso `/api/v1` e l'autenticazione via `Authorization: Bearer <token>` restano invariati.

Tabella 42: Endpoint REST MVP — delta rispetto al PoC (§ 3).

Metodo e Path	Scopo (MVP)	Stato
<i>Autenticazione</i>		
POST /auth/register	Registrazione reale con Argon2id; accetta RegisterDto (include lastName); risponde 201 UserProfileDto (profilo senza token — il token si ottiene con il login successivo)	Realizzato MVP
POST /auth/login	Login reale; verifica password Argon2id; risponde AuthTokenDto (JWT HS256, scadenza 8h)	Realizzato MVP
POST /auth/logout	Nuovo. Stateless: nessun effetto lato server. Risponde 204. Esiste perché il frontend deve gestire stati di errore di rete al logout	Nuovo MVP
GET /auth/me	Profilo dell'utente autenticato; risponde UserProfileDto	Invariato
<i>Avvio ed Esecuzione</i>		
POST /tasks/:id/input	Risposta dell'utente a un pendingInput sospeso; body: SubmitInputDto; risponde 200 o 409 se nessun input atteso	Nuovo MVP

48.2 Canale realtime — 5 eventi WebSocket

L'MVP aggiunge un quinto evento rispetto al PoC (§ 4). Gli altri quattro sono invariati.

Tabella 43: Cinque eventi WebSocket emessi dal backend verso il frontend (MVP completo).

Evento	Payload	Uso
task.updated	{ taskId, status, reportId? }	Aggiorna il badge di stato in dashboard
task.progress	{ taskId, stage, percent }	Muove la barra di avanzamento
task.failed	{ taskId, error }	Mostra toast di errore isolato
batch.completed	{ batchId, completed, failed }	Abilita i controlli successivi (RF.47)
task.inputRequired	PendingInput (vedi § 49)	Sospende la task e mostra il pannello di input all'utente

48.3 Tipo PendingInput e SubmitInputDto

```
// Payload di task.inputRequired e campo pendingInput su TaskDto
// FONTE DI VERITA': questi nomi si applicano sia al backend che al
// frontend.
type PendingInput =
  | { kind: 'SPRINT_ID' }
  | { kind: 'INCOMPLETE_TASKS'; taskIds: string[] }
  | { kind: 'BUSINESS_CONFIRMATION'; technicalReportId: string }
  | null

// Nota: INCOMPLETE_TASKS porta taskIds (ID delle Issue/User Story
// GitHub con
```

```
// metadati insufficienti). BUSINESS_CONFIRMATION porta
technicalReportId
// (ID del report tecnico già persistito che l'utente deve leggere
prima
// di confermare) -- non il testo in linea, per evitare duplicazione.

// Body di POST /tasks/:id/input
type SubmitInputDto =
  | { kind: 'SPRINT_ID'; sprintId: string }
  | { kind: 'INCOMPLETE_TASKS'; action: 'PROCEED' | 'CANCEL' }
  | { kind: 'BUSINESS_CONFIRMATION'; action: 'PROCEED' | 'CANCEL' }
```

Listing 26: Tipo PendingInput (payload di task.inputRequired) e SubmitInputDto (body di POST /tasks/:id/input). I kind name qui definiti sono la fonte di verità: il frontend aderisce a questa nomenclatura.

49 Meccanismo Input Utente: Pause e Ripresa dell'Agente Changelog

L'agente Changelog richiede input dall'utente fino a tre volte durante la sua esecuzione (Sprint ID, qualità dei work item, conferma del changelog tecnico). Questo richiede un meccanismo di **sospensione e ripresa** dell'esecuzione LangGraph, che non è un comportamento predefinito di FastAPI e va progettato esplicitamente.

49.1 Soluzione architetturale

La soluzione adottata si basa su tre componenti:

1. **LangGraph Native Interrupts + Checkpointer MongoDB:** si usa la primitiva `interrupt()` nativa di LangGraph per sospendere il grafo. Lo stato del grafo viene persistito su MongoDB tramite `langgraph-checkpoint-mongodb`, senza aggiungere nuova infrastruttura (MongoDB è già nel progetto).
2. **Campo `lgThreadId` su Task:** ogni Task che avvia l'agente Changelog riceve un UUID univoco (`lgThreadId`) che identifica il thread LangGraph. Questo campo è salvato nel documento MongoDB del Task e costituisce il mapping tra Task ID e checkpoint LangGraph.
3. **Comunicazione NestJS ↔ Python via HTTP interno:** il servizio Python espone due endpoint interni (non pubblici) per avvio e ripresa:

```
// Avvia una nuova esecuzione (sincrono: blocca fino al prossimo
interrupt o fine)
POST /internal/agent/start
Body: { taskId: string, threadId: string, operationCode: string, payload
      : object }
Response: AgentStepResult

// Riprende un'esecuzione sospesa
POST /internal/agent/resume
Body: { threadId: string, inputValue: string | object }
Response: AgentStepResult

interface AgentStepResult {
  status: 'interrupted' | 'completed' | 'failed'
```

```

    pendingInput?: PendingInput    // presente solo se status === '
        interrupted'
    result?: object                  // presente solo se status === '
        completed'
    error?: string                   // presente solo se status === 'failed'
}

```

Listing 27: Endpoint HTTP interni del Python agent service per il meccanismo pause/resume. Non sono esposti pubblicamente; il frontend non li conosce.

49.2 Implementazione LangGraph

Il nodo LangGraph che richiede input usa `interrupt()` di LangGraph, che internamente salva il checkpoint e restituisce il controllo:

```

from langgraph.types import interrupt, Command
from langgraph.checkpoint.mongodb import MongoDBSaver

# Configurazione checkpointer a bootstrap del servizio
checkpointer = MongoDBSaver(mongo_client, db_name="codeguardian")
changelog_graph = build_changelog_graph().compile(checkpointer=
    checkpointer)

# Nodo che richiede lo Sprint ID
def node_richiedi_sprint_id(state: ChangelogState):
    sprint_id = interrupt({"kind": "SPRINT_ID"}) # pausa; salva
        checkpoint
    return {"sprint_id": sprint_id}

# Nodo che segnala elementi con metadati insufficienti
def node_verifica_metadati(state: ChangelogState):
    insufficient_ids = calcola_ids_insufficienti(state.work_items)
    if insufficient_ids:
        action = interrupt({
            "kind": "INCOMPLETE_TASKS",
            "taskIds": insufficient_ids # ID delle Issue GitHub
                insufficienti
        })
        if action == "CANCEL":
            return {"status": "cancelled"}
    return {"status": "ok"}

# Ripresa con il valore fornito dall'utente
def ripresa(thread_id: str, valore_utente):
    config = {"configurable": {"thread_id": thread_id}}
    return changelog_graph.invoke(Command(resume=valore_utente), config=
        config)

```

Listing 28: Pattern di sospensione nei nodi del grafo Changelog. Il checkpointer MongoDB salva lo stato automaticamente prima di ogni interrupt.

49.3 Flusso completo end-to-end — CHANGELOG_BUSINESS (caso peggiore: tre sospensioni)

CHANGELOG_BUSINESS è l'unica operazione che può toccare tutti e tre i punti di sospensione. Il flusso per CHANGELOG_TECHNICAL è identico ma si ferma al passo 6.

1. **Creazione Task:** NestJS genera `lgThreadId = uuid()` e lo salva su `Task`; porta la `Task` in stato `RUNNING`.
2. **Sospensione pre-avvio** — `SPRINT_ID`: NestJS controlla se lo `Sprint ID` è noto *prima* di invocare l'agente. Se non lo è, valorizza `Task.pendingInput = {kind:"SPRINT_ID"}` ed emette `task.inputRequired` via `WebSocket`. L'agente non viene avviato.
3. **Input utente (Sprint ID):** l'utente risponde via `POST /tasks/:id/input {kind:"SPRINT_ID", sprintId:"SP-42"}`. NestJS azzerava `pendingInput`.
4. **Prima invocazione agente:** NestJS segna $t_0 = \text{now}()$ e chiama `POST /internal/agent/start {taskId, threadId, operationCode, payload:{sprintId:"SP-42",...}}` con `timeout HTTP = timeout_s[operationCode] + 5 s`.
5. **Sospensione su INCOMPLETE_TASKS (eventuale):** se `ChangelogLoader` trova `Issue` con metadati insufficienti, `interrupt()` salva il checkpoint e Python risponde subito con `{status:"interrupted", pendingInput:{kind:"INCOMPLETE_TASKS", taskIds:[...]}}`. NestJS cumula $\text{now}() - t_0$ in `Task.accumulatedMs`, valorizza `pendingInput`, emette `task.inputRequired`.
6. **Input utente (PROCEED/CANCEL):** l'utente sceglie via `POST /tasks/:id/input {kind:"INCOMPLETE_TASKS", action:"PROCEED"}`. Se `CANCEL`: `Task` → `CANCELLED`, flusso terminato. Se `PROCEED`: NestJS azzerava `pendingInput`, segna $t_1 = \text{now}()$ e chiama `POST /internal/agent/resume {threadId, resumeValue:"PROCEED"}`.
7. **Sospensione su BUSINESS_CONFIRMATION:** l'agente produce il changelog tecnico, lo restituisce nel risultato intermedio insieme a `status:"interrupted"` e `pendingInput:{kind:"BUSINESS_CONFIRMATION", technicalReportId:"..."}`. NestJS: cumula $\text{now}() - t_1$ in `Task.accumulatedMs`; persiste il Report tecnico (`status:COMPLETED`); valorizza `pendingInput`; emette `task.inputRequired`.
8. **Input utente (PROCEED/CANCEL):** l'utente legge il Report tecnico su `GET /reports/:id` e risponde via `POST /tasks/:id/input {kind:"BUSINESS_CONFIRMATION", action:"PROCEED"}`. Se `CANCEL`: `Task` → `CANCELLED`. Se `PROCEED`: NestJS azzerava `pendingInput`, segna $t_2 = \text{now}()$ e chiama `POST /internal/agent/resume {threadId, resumeValue:"PROCEED"}`.
9. **Completamento:** l'agente produce il changelog di business e risponde `{status:"completed", result:{...}}`. NestJS cumula $\text{now}() - t_2$; persiste il Report business; scrive `Report.executionTimeMs = Task.accumulatedMs` (somma dei tre segmenti di tempo macchina, senza i periodi di attesa utente); porta `Task` → `COMPLETED`; emette `task.updated` e `batch.completed`.

Timeout delle chiamate interne. NestJS usa un `HttpService` dedicato per le chiamate a `/internal/*`, configurato con `timeout = timeout_s[operationCode] + 5 s`. Se il timeout scatta, NestJS porta la `Task` a `FAILED` con `error.code = 'INTERNAL_ERROR'`: il container Python non ha risposto, probabilmente per OOM o processo bloccato. Questo timeout è indipendente dal timeout del gateway pubblico (§51).

49.4 Perché questa soluzione

Tabella 44: Confronto tra le alternative valutate per il meccanismo pause/resume.

Alternativa	Vantaggio	Problema
In-memory state Python	Nessun overhead	Stato perso al riavvio del container
Redis Pub/Sub	Disaccoppiamento forte	Infrastruttura aggiuntiva (Redis già presente solo per BullMQ)
MongoDB Checkpointer + HTTP	Nessuna infrastruttura nuova; LangGraph-native; semplice	Richiede il package <code>langgraph-checkpoint-mongodb</code>

50 Decisioni Architetture Backend (BE-ADR)

Le sigle BE-ADR sono proprie di questa sezione e si distinguono dagli ADR del PoC (§ 10) e dell'infrastruttura AWS (§ 39).

BE-ADR-1 — Argon2id per l'hashing delle password. Tra le opzioni valutate (bcrypt, scrypt, Argon2id), Argon2id è la scelta raccomandata da OWASP per il 2024 per le nuove implementazioni. A differenza di bcrypt (massimo 72 byte di input, nessuna protezione memory-hard lato side-channel) e di scrypt (data-dependent memory access, vulnerabile ad alcuni attacchi timing), Argon2id combina la resistenza agli attacchi GPU (memory-hard) con la protezione dai side-channel (data-independent memory access nella prima metà). I parametri scelti (memoria 64 MB, iterazioni 3, parallelismo 4) rispettano le raccomandazioni minime OWASP e producono un tempo di hashing accettabile per il backend (< 500 ms su hardware da produzione).

BE-ADR-2 — AES-256-GCM + HKDF per la cifratura delle credenziali. La chiave master (`CREDENTIAL_MASTER_KEY`) è generata da AWS Secrets Manager e ha già alta entropia. Usare Argon2id per derivarla sarebbe inappropriato (key-stretching applicato a materiale già entropico, costo computazionale ingiustificato). HKDF (RFC 5869) è lo standard per derivare chiavi crittograficamente forti da materiale già entropico: produce un salt per-record (garantisce che chiavi distinte cifrano record distinti) e include un campo *info* per separare i domini di utilizzo ("`credential-enc`"). AES-256-GCM fornisce sia riservatezza sia integrità del ciphertext (authenticated encryption).

BE-ADR-3 — Scope GitHub repo unico per tutti i ruoli. Il principio del least-privilege richiederebbe token con scope diversi per lettura e scrittura. Tuttavia, GitHub non offre un scope intermedio tra `public_repo` (sola lettura pubblica) e `repo` (lettura/scrittura privata e pubblica): un token a sola lettura non riesce ad accedere ai repository privati dell'utente. Poiché Code Guardian deve operare anche su repository privati, il team ha accettato il compromesso di usare un unico scope `repo` per tutti i ruoli, con la mitigazione che il token è custodito cifrato nel backend e mai esposto agli agenti in chiaro.

BE-ADR-4 — URL-first per la creazione del contesto (repoUrl). Nel PoC il client forniva `repoOwner` e `repoName` separatamente. Il passaggio a `repoUrl` nel corpo di `POST /contexts` (con estrazione server-side di owner/name) è motivato da: (a) l'URL è l'identificatore naturale che l'utente copia dal browser; (b) il parsing server-side elimina la possibilità che owner e name

siano inconsistenti; (c) l'URL contiene già l'host, il che permette di validarne la provenienza (deve essere `github.com`).

BE-ADR-5 — No refresh token, no server-side token store. La scelta di un'architettura stateless (JWT con scadenza 8h, nessun refresh token) è coerente con il perimetro MVP: non introduce Redis come session store, non richiede endpoint di revoca token, e il numero di utenti concorrenti attesi non giustifica la complessità aggiuntiva. Il compromesso è che una sessione compromessa non può essere invalidata prima della scadenza naturale; il rischio è mitigato dall'intervallo breve (8h) e dall'assenza di dati particolarmente sensibili nel JWT (il ruolo non è un segreto).

BE-ADR-6 — Scrittura su GitHub esclusivamente dal backend NestJS. L'agente Python continua a produrre solo una **Proposal** (diff) attraverso la facade read-only invariata. Una volta ricevuto e persistito quell'output nel Report, è il backend NestJS a invocare l'endpoint di scrittura delle API GitHub (branch + commit + apertura PR), usando un token con scope **repo** dedicato. L'apertura è automatica e immediata al termine dell'esecuzione dell'agente: Code Guardian non implementa un passaggio di conferma umana preliminare. La validazione della modifica proposta avviene interamente su GitHub, tramite il normale flusso di review/merge della PR.

Questa decisione recepisce ADR-AWS-10 e ADR-FE-2. Perché RS.3 resta rispettato: a livello applicativo, il token di scrittura non lascia mai NestJS; a livello di rete, il container agents non ha alcuna rotta verso l'esterno (zero egress), quindi non potrebbe raggiungere GitHub nemmeno se il token gli fosse passato per errore.

51 Timeout e Limiti di Esecuzione

51.1 Gateway pubblico (RQ.6)

Il gateway pubblico (AWS API Gateway o Load Balancer) ha un timeout massimo configurabile. RQ.6 richiede che il backend rispetti un limite di **300 s** per il completamento di una richiesta. Il gateway è configurato a **310 s** (300 s + 10 s di buffer per overhead di rete e serializzazione). Superato il timeout, il gateway restituisce 504 al client; NestJS porta la Task a **FAILED** con `error.code = 'INTERNAL_ERROR'` e `error.stage = 'EXECUTION'`.

Nota PoC: nel PoC il gateway è configurato a 100 s; questa è una configurazione di sviluppo, non di produzione. Il valore corretto per l'MVP è 310 s.

51.2 Timeout per operazione (`timeout_s`)

Il campo `timeout_s` vive nella configurazione di `AgentRegistry` (non nella firma di `run()`). È il timeout che NestJS applica alla singola chiamata HTTP verso `/internal/agent/start` o `/internal/agent/resume`. Se il timeout scatta, NestJS porta la Task a **FAILED** con `error.code = 'INTERNAL_ERROR'`.

Tabella 45: Timeout configurati per operazione nell'MVP. I valori valgono per la singola chiamata a `/internal/agent/*`, non per la Task complessiva.

OperationCode	timeout_s	Nota
DOCS_INLINE	90	Validato nel PoC
DOCS_README	150	Esplorazione ampia del repository
DOCS_API	150	Esplorazione ampia del repository
SECURITY_OWASP	180	Validato nel PoC; scansione completa
SECURITY_POLICY	120	Scope piu' ristretto (solo POLICY.md)
CHANGELOG_TECHNICAL	90	Validato nel PoC
CHANGELOG_BUSINESS	120	Per singola fase, non per la Task totale

Relazione con il gateway: il timeout NestJS per le chiamate `/internal/*` è configurato come `timeout_s[operationCode] + 5 s` (5 s di buffer per latenza di rete intra-VPC). Il gateway pubblico a 310 s è sempre superiore alla somma dei timeout per-fase di qualunque operazione, poiché `CHANGELOG_BUSINESS` è l'unica operazione con piu' fasi e la somma delle sue fasi (`SPRINT_ID` sincro + `INCOMPLETE_TASKS` 120 s + `BUSINESS_CONFIRMATION` 120 s) non supera i 310 s del gateway.

52 Catalogo delle Operazioni e Avvio

52.1 OperationDescriptorDto

`GET /operations` è invariato dalla PoC. Il tipo di risposta, `OperationDescriptorDto`, era nominato nel contratto PoC ma mai definito:

```
interface OperationDescriptorDto {
  code: OperationCode          // identificativo stabile usato in Task e
    Report
  displayName: string           // etichetta leggibile (italiano)
  description: string           // descrizione sintetica dell'operazione
  agent: 'DOCS' | 'SECURITY' | 'CHANGELOG'
}

type OperationCode =
  | 'DOCS_README'
  | 'DOCS_INLINE'
  | 'DOCS_API'
  | 'SECURITY_OWASP'
  | 'SECURITY_POLICY'
  | 'CHANGELOG_TECHNICAL'
  | 'CHANGELOG_BUSINESS'
```

Listing 29: `OperationDescriptorDto`: schema restituito da `GET /operations`.

La risposta contiene solo le operazioni permesse al ruolo del chiamante (claim `role` del JWT). L'elenco è costruito dalle voci di `AgentRegistry`: un'operazione non registrata non compare mai nel catalogo.

52.2 Controlli prima dell'accodamento (POST `/tasks`)

Prima di accodare un batch, NestJS esegue tre controlli in ordine. Il fallimento di uno solo respinge l'intero batch (nulla viene accodato parzialmente):

1. **Selezione non vuota** (RF.41): `operations[]` deve contenere almeno un elemento dopo deduplicazione; valori non nell'enumerazione chiusa sono respinti con 400.
2. **Prerequisiti** (RF.42): `contextId` deve esistere e appartenere al chiamante; una credenziale GitHub deve essere configurata. Lo scope del token è già stato verificato al salvataggio (RF.13) e non viene reverificato qui.
3. **Ruolo** (RF.7): se anche una sola operazione non è permessa al ruolo del chiamante, l'intero batch è respinto con 403.

Limite di utilizzo (RF.66, desiderabile): il controllo del tetto mensile per utente (contato a livello di Task creata, non di invocazione) avviene dentro `POST /tasks` *prima* dell'accodamento: la richiesta è respinta con 429 e codice `USAGE_LIMIT_EXCEEDED`.

53 Dettaglio degli Agenti

Questa sezione descrive, operazione per operazione, cosa fa il backend *prima* e *dopo* aver invocato ciascun agente. Il funzionamento interno degli agenti appartiene al servizio Python; qui rileva solo il contratto tra i due servizi.

53.1 Agente Docs

L'Agente Docs è l'unico dei tre che produce una `Proposal` (diff): il backend, ricevuta la `Proposal`, apre automaticamente una Pull Request su GitHub (BE-ADR-6) e ne scrive l'URL nel campo `proposal.pullRequestUrl`. L'agente non scrive mai direttamente sul repository.

53.1.1 DOCS_README (RF.82)

Instradamento: `AgentRegistry` → Agente Docs. Il backend invoca l'agente con il contesto e attende la `Proposal`. Ricevuta, persiste il Report, apre la PR (BE-ADR-6, RF.63) e ne salva l'URL. Un README non ancora esistente è gestito nativamente: il `diffUnified` rappresenta un file nuovo come diff contro il vuoto.

53.1.2 DOCS_INLINE (RF.83, RF.84)

Stesso meccanismo di `DOCS_README`, con la `Proposal` contenente diff di commenti `JSDoc/-docstring`. RF.84 (commenti disallineati) e RF.83 (commenti assenti) condividono lo stesso `OperationCode`, la stessa voce di `AgentRegistry` e lo stesso ciclo invoca→ricevi→apri PR. L'agente emette un `ComplexityWarningBlock` (RF.85, RF.61) quando il codice è troppo intricato: il backend lo persiste nel corpo del Report senza ulteriori elaborazioni.

53.1.3 DOCS_API (RF.86)

Nuovo `OperationCode`, nuova voce in `AgentRegistry`, stessa sequenza `Proposal`→PR→URL. Cosa costituisca “documentazione degli endpoint” è una decisione del servizio agenti.

53.2 Agente Security

L'Agente Security è *flag-only*: individua e documenta, non propone modifiche al repository. Il backend non apre mai Pull Request per queste operazioni.

53.2.1 SECURITY_OWASP (RF.87–RF.91)

Instradamento: `AgentRegistry` → Agente Security. Il backend invoca l'agente e persiste il Report ricevuto (elenco di `FindingBlock`) senza ulteriori passi. Ogni `FindingBlock` porta `category` (OWASP), `severity`, `filePath`, `lineStart/lineEnd?` e `remediation` (discriminante SNIPPET | TEXT). Il backend riceve e persiste questi valori così come arrivano.

53.2.2 SECURITY_POLICY (RF.92, RF.93)

Stessa meccanica di SECURITY_OWASP, instradato su SECURITY_POLICY. L'assenza di `POLICY.md` emerge come `CONTEXT_RESOURCE_MISSING` (RF.70) dall'agente, propagata dal backend come qualsiasi altro fallimento.

53.3 Agente Changelog

L'Agente Changelog non apre mai Pull Request: il suo output è testo. È l'unico agente che richiede input interattivi durante l'esecuzione.

53.3.1 Flusso di input per le operazioni Changelog

Entrambe le operazioni Changelog richiedono lo Sprint ID *prima* dell'invocazione dell'agente. Il backend intercetta questo passo:

1. **Sospensione pre-avvio** (`SPRINT_ID`): se lo Sprint ID non è noto, NestJS valorizza `pendingInput = {kind: "SPRINT_ID"}` ed emette `task.inputRequired` senza invocare l'agente. L'utente risponde via `POST /tasks/:id/input`; solo allora l'agente viene avviato con lo Sprint ID nel payload.
2. **Sospensione durante esecuzione** (`INCOMPLETE_TASKS`): il `ChangelogLoader` (agente) rileva Issue con metadati insufficienti. Invece di procedere autonomamente, si sospende con `interrupt()` e restituisce al backend `{kind: "INCOMPLETE_TASKS", taskIds: [...]}`. Il backend valorizza `pendingInput` e notifica il frontend. L'utente sceglie `PROCEED` (escludi gli elementi) o `CANCEL` (termina la Task in `CANCELLED`).
3. **Sospensione durante esecuzione** (`BUSINESS_CONFIRMATION`, solo `CHANGELOG_BUSINESS`): il backend, dopo aver persistito il changelog tecnico come Report `COMPLETED`, valorizza `pendingInput = {kind: "BUSINESS_CONFIRMATION", technicalReportId: "<id>"}` e sospende. L'utente legge il report tecnico (tramite `GET /reports/:id`), poi sceglie `PROCEED` (avvia fase business) o `CANCEL` (Task in `CANCELLED`).

53.3.2 CHANGELOG_TECHNICAL (RF.94–RF.96)

Il backend riceve lo Sprint ID da `POST /tasks/:id/input`, avvia l'agente con quel dato, persiste il Report tecnico ricevuto (un `TextBlock` in Markdown) e porta la Task a `COMPLETED`.

53.3.3 CHANGELOG_BUSINESS (RF.94–RF.105)

Stessa sequenza di CHANGELOG_TECHNICAL per la prima fase (changelog tecnico). Dopo la persistenza del Report tecnico, il backend sospende su `BUSINESS_CONFIRMATION`. Alla conferma, riprende l'agente per la fase business; al rifiuto, porta la Task a `CANCELLED`. Una Task `CHANGELOG_BUSINESS` che rifiuti la conferma non produce mai un changelog di business: chi vuole solo il changelog tecnico avvia CHANGELOG_TECHNICAL.

53.3.4 Leggibilità del Changelog (RQ.7)

L'agente calcola l'indice Flesch Reading Ease sul proprio output. Se l'output non supera la soglia entro il budget di tentativi, l'agente restituisce `READABILITY_TOO_LOW`: il backend propaga il fallimento come ogni altro errore rilevato dall'agente.

54 Gestione degli Errori

54.1 Il meccanismo riusato dalla PoC (RF.65)

Il contratto della PoC copre già il percorso di errore di base: timeout e parsing failure valorizzano `error` e portano `status` a `FAILED` senza crash (PoC §5). Restano invariati: il campo `error: {code, message, stage}` | `null` su `TaskDto` e `ReportDto`; lo stato terminale `FAILED`; l'evento `task.failed`; l'isolamento dei fallimenti nel `TaskProcessor`.

Correzione al contratto PoC: `reportId` su `TaskDto` è valorizzato quando `status` è `COMPLETED` oppure `FAILED` (non solo `COMPLETED` come indicato in PoC §7.4). Una Task fallita produce sempre un Report con `status: FAILED` e corpo vuoto, raggiungibile a `GET /reports/:id`: è ciò che permette al frontend di aprire il report da una Task fallita.

54.2 Codici d'errore (enumerazione chiusa)

Tabella 46: Enumerazione `error.code`: 11 valori per l'MVP. I messaggi mostrati all'utente restano quelli della Progettazione Frontend, Tabella 7.

<code>error.code</code>	Req.	Rilevato da	Azione secondaria (Frontend)
<code>USAGE_LIMIT_EXCEEDED</code>	RF.66	backend	Nessuna
<code>LLM_TIMEOUT</code>	RF.67	agente	“Riprova”
<code>LLM_RATE_LIMITED</code>	RQ.8	agente	Da definire
<code>OUTPUT_UNPARSABLE</code>	RF.68	agente	“Riprova”
<code>CONTEXT_TOO_LARGE</code>	RF.69	agente	Rimanda a <code>/select</code>
<code>CONTEXT_RESOURCE_MISSING</code>	RF.70	agente	Nessuna
<code>CONTEXT_RESOURCE_INVALID</code>	RF.71	agente	Nessuna
<code>READABILITY_TOO_LOW</code>	RQ.7	agente	Da definire
<code>PR_CREATION_FAILED</code>	RF.72	backend	Rimanda a <code>/credentials</code>
<code>CREDENTIAL_INVALID</code>	RF.13, RS.4	backend	Rimanda a <code>/credentials</code>
<code>INTERNAL_ERROR</code>	RF.65	backend	“Riprova” (fallback)

Regole di classificazione: `INTERNAL_ERROR` è il valore di ripiego per qualsiasi fallimento non previsto. Un'irraggiungibilità di rete verso GitHub ricade sempre in `INTERNAL_ERROR` e mai in `CREDENTIAL_INVALID` (per evitare falsi avvisi di credenziale non valida in caso di caduta del NAT Gateway). `error.stage` è sempre valorizzato con la fase in cui l'esecuzione si è interrotta per i codici rilevati durante l'esecuzione di una Task; i codici che nascono prima che una Task esista (`USAGE_LIMIT_EXCEEDED`, `CREDENTIAL_INVALID` su `POST /credentials`) viaggiano nell'involucro di errore REST, non nel campo `error` di una Task.

Responsabilità di rilevamento : gli agenti rilevano RF.67–RF.71, RQ.7–RQ.8 e restituiscono il codice al backend; il backend riceve, classifica e propaga (porta la Task a `FAILED`,

valorizza `error`, persiste il Report fallito, emette `task.failed`). Il backend rileva in proprio solo RF.66 (limite utilizzo) e RF.72 (apertura PR).

Fallimento dell'apertura PR (RF.72) : `PR_CREATION_FAILED` per rifiuto di autorizzazione GitHub (`error.stage = 'PULL_REQUEST'`); `INTERNAL_ERROR` per irraggiungibilità di rete. Il Report è persistito con il diff dell'agente prima del tentativo di apertura; quando il tentativo fallisce, il Report è aggiornato a status `FAILED` con l'errore. Il diff è conservato nei dati ma non offerto nell'interfaccia.

54.3 Enumerazione `ErrorKind` lato agente (Python)

Il backend riceve il codice di errore nell'oggetto `AgentStepResult.error`. Internamente, il servizio Python mappa ogni classe di fallimento su un `ErrorKind` prima di rispondere. L'MVP estende l'enumerazione del PoC (che copriva solo `TIMEOUT` e `PARSING`) con i codici aggiuntivi richiesti dai nuovi requisiti:

```
from enum import Enum

class ErrorKind(str, Enum):
    # Valori storici (PoC)
    TIMEOUT = "TIMEOUT"      # Timeout LLM (RF.67) -> error.code
    LLM_TIMEOUT
    PARSING = "PARSING"      # Output LLM non parsabile (RF.68) ->
    OUTPUT_UNPARSABLE
    UPSTREAM = "UPSTREAM"    # Errore generico upstream -> INTERNAL_ERROR

    # Valori nuovi (MVP)
    CONTEXT_TOO_LARGE = "CONTEXT_TOO_LARGE"      # RF.69
    CONTEXT_RESOURCE_MISSING = "CONTEXT_RESOURCE_MISSING" # RF.70
    CONTEXT_RESOURCE_INVALID = "CONTEXT_RESOURCE_INVALID" # RF.71
    READABILITY_TOO_LOW = "READABILITY_TOO_LOW"    # RQ.7
    RATE_LIMITED = "RATE_LIMITED"                  # RQ.8 ->
    LLM_RATE_LIMITED
```

Listing 30: `ErrorKind`: enumerazione estesa per l'MVP nel servizio agenti Python. I tre valori storici del PoC sono invariati; i cinque nuovi valori coprono i requisiti RF.69–RF.71, RQ.7–RQ.8.

La mappatura da `ErrorKind` a `error.code` (backend) è biunivoca: `TIMEOUT` → `LLM_TIMEOUT`, `PARSING` → `OUTPUT_UNPARSABLE`, `RATE_LIMITED` → `LLM_RATE_LIMITED`, gli altri tre nuovi valori mantengono il nome invariato come `error.code`.

55 Report: Storico, Dettaglio ed Esportazione

55.1 Contratto `ReportDto` esteso (RF.54–RF.64)

Il Report estende l'envelope PoC con nuovi campi. Nessun campo esistente viene rimosso o rinominato:

```
interface ReportDto {
    id: string
    taskId: string
    operation: OperationCode
    status: 'COMPLETED' | 'FAILED'
    title: string          // RF.51, sempre presente, deterministico
    summary: string | null // null su report FAILED (nessun output
        agente)
    generatedAt: string    // ISO 8601 con offset (RF.52, RF.58)
```

```

executionTimeMs: number | null    // RF.57; null se nessun agente
    invocato
context: {                        // RF.55-56, denormalizzato al momento del
    Report
    repoOwner: string
    repoName: string
    repoUrl: string
    branch: string
    resolvedSha: string
    scopeType: 'FULL_REPOSITORY' | 'FILES' | 'DIRECTORIES'
    paths: string[]
}
body: Block[]
proposal?: {
    targetPath: string
    diffUnified: string
    language: string
    pullRequestUrl: string | null    // RF.63; null per Security e
        Changelog
}
pendingAction: {                  // RF.64, proiezione read-only di
    pendingInput
    kind: 'BUSINESS_CONFIRMATION'
    taskId: string
    actions: ('PROCEED' | 'CANCEL')[]
} | null
error?: { code: string; message: string; stage: string }
}

```

Listing 31: ReportDto: schema completo di GET /reports/:id per l'MVP.

Note implementative:

- **Titolo** (RF.51): composto deterministicamente dal backend nella forma “*Nome operazione — owner/repo@branch*” (es. “Scansione OWASP — SkynetUniGroup/-Code-Guardian@main”). Non generato da LLM; sempre presente anche su Report FAILED; letto dallo stesso catalogo di GET /operations.
- **Contesto denormalizzato**: i valori di `context` sono copiati nel documento Report al momento dell’assemblaggio, non risolti dinamicamente. Un report è un registro immutabile: resta leggibile anche se il contesto di analisi viene rimosso.
- **executionTimeMs** (RF.57): misura il solo *tempo macchina*, cioè la somma dei segmenti di esecuzione dell’agente. Esclude: attesa in coda prima dell’invocazione, generazione PDF, e soprattutto i periodi di attesa dell’utente su `pendingInput` (che possono essere arbitrariamente lunghi). Il campo `Task.accumulatedMs` (campo interno, non esposto nell’API) accumula la durata di ciascuna chiamata `/internal/agent/*`; al completamento della Task, `Report.executionTimeMs` è scritto uguale a `Task.accumulatedMs`. Per le operazioni con una sola chiamata all’agente, `executionTimeMs = t.response - t.call`; per CHANGELOG.BUSINESS, è la somma dei tre segmenti:

$$\text{executionTimeMs} = (t_1 - t_0) + (t_2 - \text{risposta INCOMPLETE_TASKS}) + (t_3 - t_2)$$

dove t_0, t_1, t_2, t_3 sono i timestamp di chiamata e risposta di ciascuna invocazione `/internal/agent/*`.

- **pendingAction**: proiezione in sola lettura di `Task.pendingInput`. La risposta dell’utente viaggia sempre su POST `/tasks/:id/input`, mai tramite il Report.

- **Su Report FAILED:** `executionTimeMs` e `summary` valgono `null` se nessun agente è stato invocato. `body` è vuoto. `title` è sempre presente.

55.2 Storico e sintesi (GET /reports)

```
interface ReportSummaryDto {
  id: string          // RF.50
  operation: OperationCode
  status: 'COMPLETED' | 'FAILED'
  title: string        // RF.51
  generatedAt: string  // RF.52
}
```

Listing 32: `ReportSummaryDto`: proiezione restituita da `GET /reports`.

`body`, `proposal` ed `error` sono esclusi dalla proiezione. L'endpoint è protetto da JWT; il guard confronta `Report.userId` con il claim `sub`: risposta 404 (non 403) se il report non appartiene al chiamante, per non confermare l'esistenza di report altrui.

55.3 Blocchi del corpo (RF.59–RF.62)

Due nuovi tipi di blocco, aggiuntivi rispetto ai `TextBlock` della PoC:

```
// Avviso di complessità eccessiva (DOCS_INLINE, DOCS_API; mai DOCS_README)
interface ComplexityWarningBlock {
  kind: 'COMPLEXITY_WARNING'
  category: 'EXCESSIVE_COMPLEXITY'
  severity: 'LOW' | 'MEDIUM' | 'HIGH' | 'CRITICAL' // valore fisso, non LLM
  filePath: string
  lineStart: number
  lineEnd: number
  reason: string
}

// Tipo di rimedio (union discriminata su kind)
// Usato in FindingBlock e PolicyViolationBlock
type Remediation =
  | { kind: 'SNIPPET'; language: string; code: string }
  | { kind: 'TEXT'; text: string }

// Risccontro di sicurezza OWASP (SECURITY_OWASP)
interface FindingBlock {
  kind: 'FINDING'
  category: string // categoria OWASP
  severity: 'LOW' | 'MEDIUM' | 'HIGH' | 'CRITICAL'
  filePath: string
  lineStart: number
  lineEnd?: number
  description: string
  remediation: Remediation
}

// Violazione di policy (solo SECURITY_POLICY, RF.92--RF.93)
// Differisce da FindingBlock: porta rule_id/rule_text anziché category OWASP.
```

```
// severity e' determinato dal modello (non valore fisso come in
// ComplexityWarningBlock).
interface PolicyViolationBlock {
  kind: 'POLICY_VIOLATION'
  ruleId: string // identificatore
    della regola violata
  ruleText: string // testo della
    regola (estratto da POLICY.md)
  filePath: string
  lineStart?: number
  lineEnd?: number
  severity: 'LOW' | 'MEDIUM' | 'HIGH' | 'CRITICAL' // RF.61;
    determinato dal modello
  explanation: string
  remediation: Remediation
}
```

Listing 33: Nuovi tipi di blocco per l'MVP.

Il Markdown nei `TextBlock` è normalizzato dal backend al momento dell'assemblaggio: blocchi HTML grezzi rimossi, destinazioni `javascript:` e `data:` rifiutate. La normalizzazione avviene una volta sola, al confine, così resa a schermo ed esportazione PDF consumano la stessa stringa.

55.4 Esportazione PDF (RF.73–RF.74)

Endpoint: `GET /reports/:id/export?format=pdf` (stesso percorso della PoC, formato md decade). Protetto da JWT con la stessa regola di proprietà di `GET /reports/:id` (404 se non appartiene al chiamante). Restituisce 409 su report FAILED (corpo vuoto per costruzione).

Implementazione: il PDF è composto programmaticamente (`pdfkit/pdf-lib`) a partire dal `ReportDto` persistito — mai dal DOM renderizzato. Viene archiviato nel bucket S3 privato (lifecycle 30 giorni) e restituito in streaming con `Content-Type: application/pdf`, `Content-Disposition: attachment`, nome file `code-guardian-<operation>-<reportId>.pdf`. La composizione avviene interamente in memoria prima di scrivere nel corpo della risposta (per evitare file troncati in caso di errore). Le URL pre-firmate S3 sono vietate dalla Progettazione Frontend §13.5: lo streaming è sempre same-origin.

Fallimento dell'esportazione (RF.74): 500 con `{code: 'EXPORT_FAILED', message}`. Nessun evento `WebSocket`; né `Task.status` né `Report.status` vengono modificati. `EXPORT_FAILED` non compare nell'enumerazione di § 54.2 perché non è un fallimento di esecuzione `Task` (UC27).

Nota di rischio: la Progettazione AWS (Tabella 20) annota che il task backend da 0,5 vCPU e 1 GB è dimensionato per un servizio REST leggero e la generazione PDF non è stata validata con un test di carico su esportazioni concorrenti. Tale test è da eseguire prima del rilascio.